

Unique NIZKs and Steganography Detection

Willy Quach *

LaKyah Tyner [†]

Daniel Wicks [‡]

Abstract

Non-interactive zero-knowledge (NIZK) proofs tend to be randomized and there are many possible proofs for any fixed NP statement. Can we have NIZKs with only a single *unique* valid proof per statement? Such NIZKs are known under strong cryptographic assumptions (indistinguishability obfuscation), and are conversely known to require strong cryptographic assumptions (witness encryption). In this work, following Lepinski, Micali, and Shelat (TCC '05), we consider the following relaxed notion of unique NIZKs (UNIZKs):

1. We only require (computationally) unique proofs for NP statements with a (computationally) unique witness; an adversary that can produce two distinct proofs must also know two distinct witnesses.
2. We consider NIZKs with prover setup, where a potentially malicious prover initially publishes a public key pk and keeps a corresponding secret key sk , which it uses to produce arbitrarily many NIZK proofs π in the future. While the public key pk is not required to be unique, once it is fixed, all the subsequent proofs π that the prover can produce should be unique.

We show that *both* of these relaxations are needed to avoid witness encryption. Prior work constructed such UNIZKs under the quadratic residuosity assumption, and it remained an open problem to do so under any other assumptions. Here, we give a new construction of UNIZKs under the *learning with errors* (LWE) assumption. We also identify and fix a subtle circularity issue in the prior work.

UNIZKs are a non-interactive version of steganography-free zero-knowledge of Abdolmaleki et al. (TCC '22). As an application of UNIZKs, we get a general steganography detection mechanism that can passively monitor arbitrary functionalities to detect steganographic leakage.

*CISPA Helmholtz Center for Information Security. Email: willy.quach@cispa.de.

[†]Northeastern University. Email: tyner.l@northeastern.edu.

[‡]Northeastern University and NTT Research. Email: wicks@ccs.neu.edu. Research supported by NSF CNS-2349972 and CNS-2055510.

Contents

1	Introduction	3
2	Technical Overview	5
3	Preliminaries	10
4	Unique NIZKs	18
4.1	Relaxations of UNIZKs and Additional Properties	20
5	Single Theorem UNIZK	21
6	Adding Bootstrappability	26
7	Multi-Theorem UNIZK	32
8	Security against Fully Malicious Provers	39
9	Necessity of Relaxations	41

1 Introduction

Zero-Knowledge: Interaction, Randomness and Uniqueness. Zero-knowledge proofs [GMR85] allow a prover to convince a verifier that an NP statement holds without revealing anything about the corresponding witness. Such proofs¹ typically rely on *interaction and randomness* and have *many valid proof transcripts* for any fixed NP statement. To what extent are interaction, randomness and non-uniqueness necessary? Non-interactive zero-knowledge proofs (NIZKs) [BFM88] remove interaction at the cost of requiring a trusted *common reference string* (CRS). In almost all NIZK constructions under various assumptions (e.g., [BFM88, FLS90, CHK03, GOS06, CCH⁺19, PS19]), the prover is still randomized and there are still many potential valid proofs for any NP statement. Interestingly, we do have a NIZK construction with a deterministic prover and unique proofs, albeit only under very strong cryptographic hardness assumptions. In particular, the NIZK of Sahai and Waters [SW14] from indistinguishability obfuscation (iO) and one-way functions, has a deterministic prover, and, for any fixed true NP statement x , the honest prover produces the same NIZK proof π no matter what witness w for x it started with. Furthermore, it is possible to ensure that even a dishonest prover can only find at most a single accepting proof π for any true statement x .² Unfortunately, it is also known [CPW23] that unique proofs (even for an honest prover, even without zero-knowledge) require strong assumptions – namely, the existence of witness PRFs [Zha16], which in turn imply witness encryption [GGSW13], but are thought to be a stronger primitive that lies somewhere between witness encryption and iO.

UNIZKs and Our Results. In this work, we focus on a relaxed notion called *unique NIZKs* (UNIZKs), introduced by Lepinski, Micali, and Shelat [LMs05].

1. We require (computationally) unique proofs π for NP statements x with a (computationally) unique witness w ; a polynomial-time adversary that can produce two distinct accepting proofs $\pi \neq \pi'$ for x must also know two distinct witnesses $w \neq w'$, as formalized using the extraction paradigm. For example, we can ensure unique proofs when proving that the output of some deterministic computation over a committed/encrypted/hashed input is correct.
2. We consider UNIZKs with *prover setup* where, in addition to having a CRS, we also have the prover initially publish a public key pk and keep a corresponding secret key sk , which it uses to produce arbitrarily many NIZK proofs π for various NP statements x in the future. The verifier uses the CRS and pk to verify the proofs. While the generation of the public key pk is randomized and pk is not required to be unique, once pk is fixed, we require that the subsequent proofs π that a prover can produce are unique as defined in (1). Both soundness and uniqueness need to hold even if a prover generates pk maliciously. In particular, while allowing for prover setup relaxes uniqueness since pk need not be unique, UNIZKs with prover setup still imply standard NIZKs without prover setup by making pk a part of the proof.

The work of [LMs05] constructed UNIZKs from the *quadratic residuosity* (QR) assumption, and left it as an explicitly open problem to build UNIZKs from other assumptions. This problem has remained open for close to 20 years. Moreover, they only provided informal descriptions of their construction without a full proof of security. In fact, we uncovered a subtle bug in their construction, which we fix in our work. We refer to the technical overview for more details.

In this work, we construct UNIZKs from leveled FHE, dual-mode extractable/equivocal commitments, and standard NIZKs, all of which follow from the learning with errors (LWE) assumption.³ Our construction is modular, with formal scheme descriptions and proofs of security.

¹We use the term “proofs” without distinguishing between statistical or computational soundness (AKA arguments).

²In [SW14], a proof is a pre-image of a one-way function f on some output y determined by the NP statement. By using a universal one-way hash function (UOWHF) instead of a one-way function, we get the above uniqueness guarantee. By using injective one-way functions we could even ensure statistically unique proofs.

³In fact, our construction is statistically sound, and achieves a statistical version of the uniqueness property described above.

As a secondary novel contribution of our work, we also show that removing *either* relaxation (1) *or* (2) would imply witness PRFs and therefore witness encryption. This strongly motivates UNIZKs as a natural cryptographic object to study if we want to avoid strong assumptions.

Steganography-Free Zero-Knowledge. This study of UNIZKs is inspired by the notion of *steganography-free (interactive) zero-knowledge* (SF-ZK) of [AFG⁺22].⁴ In SF-ZK, both the prover and the verifier have a non-unique setup phase where they output some public keys pk_P, pk_V respectively. As long as at least one of the two is honest during the setup phase, an SF-ZK ensures (computationally) unique proof transcripts π for any NP statement x with a (computationally) unique witness w , analogously to our requirement (1). Moreover, the soundness and uniqueness of the proof transcript can be checked by an external observer. Beyond having interaction, there is another major difference between UNIZKs and SF-ZKs. In UNIZKs, after the initial setup where the prover published pk , it can generate arbitrarily many proofs for arbitrary statements to arbitrary verifiers. In contrast, [AFG⁺22] considers two variants of SF-ZK. In the *single-execution* variant, which is constructed from sub-exponentially secure injective one-way functions, the setup can only be used once to prove a single statement to a single verifier. In the *multi-execution* variant, which additionally relies on leveled fully homomorphic encryption, the setup can be used to prove arbitrarily many statements, but still to only a single verifier that maintains state between proofs. In particular, in contrast to UNIZK, SF-ZK does not allow a prover with some fixed public key pk_P to later execute arbitrarily many unique proof protocols with arbitrary verifiers.

General Steganography Detection. The work of [AFG⁺22] introduced SF-ZK in the context of detecting steganographic leakage. We observe that UNIZKs enable a new form of steganography detection.

We consider the problem of detecting steganographic communication from a compromised device storing secret sensitive data. For example, consider a certificate authority that stores a secret signing key on a specialized device meant to generate DSA signatures. Or, an AI company that stores a private large-language model (LLM) on a specialized device and uses it to respond to user queries. In both cases, there are some specific computations over the secret data that the device is intended to perform (e.g., generate signatures, answer queries to the LLM), but we do not want to reveal any other aspects of the secret data (e.g., the signing key itself, internal parameters of the LLM). A particular worry is that the device may be compromised (by an external party or even by the manufacturer) and may perform the computations incorrectly causing the outputs to steganographically leak information about the secret data, beyond what would be revealed by the honest executions. For example, DSA signatures are randomized, making it easy for a compromised device to choose signatures that look legitimate but steganographically leak the secret key.

Can we passively monitor all of the incoming/outgoing communication of the device to detect such steganographic leakage? A simple solution is to commit to all the secret data as well as a seed for a pseudorandom function (PRF) before placing them (along with the decommitments) on the device when it is being setup. These commitments are public, and we do not need to store any secret data outside of the device. On each input x , the device performs the intended computation using the PRF of x to generate any local randomness and then gives the output y along with a NIZK proof π that it performed the computation correctly. The hiding of the commitment and the ZK property of the NIZK ensure that nothing beyond the intended outputs is leaked if the device is honest. On the other hand, if the device is malicious, then the soundness of the NIZK already ensures that the produced output y is correct. However, the proof π itself may be used for steganographic communication. By using UNIZKs (where the initial setup to generate pk, sk is also performed when the device is being setup), we ensure that even a malicious device can only generate a unique accepting proof π on each input x .⁵ Therefore, an external passive monitor that checks the

⁴The notion of SF-ZK is somewhat more cumbersome to define than UNIZK due to its interactive nature. We give an overview here for the sake of comparison, but it is not needed to understand our results.

⁵Technically, we need a commitment scheme where the decommitment is computationally unique, meaning that it is hard to find two different decommitments of the same commitment.

outputs and the UNIZK proofs can detect any deviation from honest behavior and therefore any attempt at steno-graphic communication.

A similar application and solution concept was briefly mentioned in [AFG⁺22], where the device would use SF-ZKs to interactively prove to the user that it performed the computation correctly, with an external passive monitor checking such proofs for uniqueness. However, this comes with two major caveats. Firstly, it requires a participating end-user who interacts with the device. More importantly, since SF-ZKs require a dedicated setup for each verifier, this would mean that we would need separate setup for each end-user of the system (e.g., each user of the LLM). By using UNIZKs instead of SF-ZKs, we avoid both these issues.

2 Technical Overview

We now describe our notion of UNIZK, and give a high level overview of our construction.

Defining UNIZKs. The syntax of a UNIZK is similar to the that of standard NIZKs,⁶ except for the following modification. Given a common reference string crs , the prover samples a random pair of keys (pk, sk) , where sk is used to deterministically generate proofs π for various statements x given a corresponding witness w , and pk (along with the crs) is used to verify these proofs.

We consider a notion of (*computational*) *uniqueness*: any (efficient) prover who generates two distinct valid proofs $\pi \neq \pi'$ for the same statement x that both verify with respect to some crs, pk , must know two different witnesses $w \neq w'$ for x , as formalized via an extraction experiment.⁷ Additionally, the UNIZK should satisfy *adaptive soundness* and *adaptive multi-theorem zero-knowledge*. Uniqueness and soundness should both hold even if the public key pk is maliciously generated by the cheating prover. Unfortunately, we cannot hope to ensure uniqueness of the public key pk itself.⁸ Instead, we aim to ensure that, while pk itself does not have any meaningful uniqueness guarantees, once it is selected, subsequent proofs verified with respect to pk must satisfy the notion of uniqueness above. We require multi-theorem adaptive zero-knowledge, where an honest prover can reuse the setup (pk, sk) to generate arbitrarily many proofs of different statements x (chosen adaptively) and all such proofs should be efficiently simulatable.⁹

Relaxing UNIZKs: Single-Theorem ZK and Honest Prover Setup. Towards constructing the above notion of UNIZKs, we start by constructing a significantly simpler variant. Firstly, we relax the requirement that the setup (pk, sk) is reusable, and only insist on single-theorem zero-knowledge where the honest prover only proves a single statement x using such a setup. Secondly, we relax the requirement that soundness and uniqueness holds even if the setup (pk, sk) is generated fully maliciously, and instead only require these to hold when the setup is performed honestly, but with potentially maliciously chosen random coins (AKA semi-maliciously).¹⁰

The construction closely follows that of single-theorem designated-prover NIZKs of [BGT20] with small modifications to ensure uniqueness. A variant of this construction was also used as a building block in the SF-ZK protocol of [AFG⁺22]. Suppose our NP statements are represented as circuits C , and valid witnesses are inputs w to such circuits such that $C(w) = 1$. The crs consists of a commitment key. To generate a pair of

⁶Our definition is conceptually very similar to the one of [LMs05], albeit with some small technical differences. We refer to the end of this overview for a more detailed comparison.

⁷We additionally consider a stronger version that we call *statistical uniqueness* where the above holds for any potentially inefficient provers. We achieve this stronger notion assuming the hardness of LWE, and refer to the technical sections for more details.

⁸This would imply UNIZK without a prover setup, which we show implies witness encryption. See the end of the technical overview and Section 9 for more details.

⁹One minor subtlety is that we do not allow the verifier to see two different proofs of the same statement x . Since the prover is deterministic and the proofs depend on the witness, doing so would reveal whether the prover uses the same witness or different witnesses. This restriction is natural in settings where witnesses are (computationally) unique and hence this scenario would never occur. Alternately, we can always have a stateful prover remember the first time it generates a proof π for some statement x and reuse it whenever it wants to prove the statement x again in the future, regardless of which witness it is given.

¹⁰Note that this relaxed variant of UNIZK no longer implies standard NIZKs and is closer in spirit to designated-prover NIZKs (DP-NIZKs) [KW18, BGT20].

keys (pk, sk) , garble a universal circuit U , which on input $(C||w)$ outputs $U(C||w) = C(w)$, obtaining garbled circuits $\tilde{U}$ and labels $lab_{i,b}$. Generate commitments $com_{i,b}$ of the labels $lab_{i,b}$ of the garbled circuit, along with a commitment com_r to the randomness used to generate the garbled circuit. For the indices i corresponding to the bits of C we reveal which commitment $com_{i,b}$ corresponds to which bit b (e.g., we give the ordered tuple $(com_{i,0}, com_{i,1})$), while for the indices i corresponding to the bits of w we do not reveal this (e.g., we only reveal the set $\{com_{i,0}, com_{i,1}\}$ ordered randomly). The resulting public key is $pk = (\tilde{U}, \{com_{i,b}\}, com_r)$ and the secret key consists of the labels $lab_{i,b}$ along with all the commitment openings. To generate a proof for a statement C and a witness w , we decommit to the labels corresponding to the input $C||w$ of the garbled universal circuit. To verify a proof, we simply check that the opened commitments correspond to the correct statement C , then evaluate the garbled circuit in pk with the provided labels and check that the output is 1.

Single-theorem zero-knowledge follows by hiding of the commitment and security of the garbled circuit.¹¹ (Adaptive) soundness for honestly (or semi-maliciously) generated pk follows by binding of the commitment and correctness of the garbled circuit. In fact, if the underlying commitment is extractable, the construction is knowledge sound for honestly generated keys: an extraction trapdoor for the commitment allows us to retrieve the randomness r from com_r , and thus the whole garbled circuit and its labels, from which one can read off the bits of the witness given the labels in the proof.

It remains to argue (adaptive) uniqueness for honestly (or semi-maliciously) generated pk . To do so, we assume that our commitment scheme has (computationally) *unique openings*: no efficient adversary can provide different openings for a single commitment.¹² Suppose an efficient cheating prover provides two different proofs $\pi \neq \pi'$ of a single statement C ; we argue that the knowledge extractor from the previous paragraph then extracts two witnesses $w \neq w'$. Since the proofs cannot open the same commitment differently, the only way they can differ is by opening different label commitments $com_{i,0}, com_{i,1}$ for some indices i corresponding to the purported bits of w_i , in which case the knowledge extractor extracts two different valid witnesses, by correctness of the garbled circuit.

Naturally, our next step is to remove the relaxations that (1) the public key pk can only be used to securely generate a single proof (in terms of zero-knowledge) and (2) that soundness and uniqueness only hold for honest (or semi-malicious) rather than fully malicious public keys. We first tackle (1).

Boosting to Multi-Theorem ZK. We bootstrap our construction above — let’s call it 1T.UNIZK for convenience — from one-time to many-time security. We do so using a tree-based construction, similar to the Naor-Yung [NY89] transformation from one-time to many-time signatures. Indeed, this idea was also used by [BGT20] in a similar context to ours to boost single-theorem zero-knowledge of designated prover NIZKs to multi-theorem zero-knowledge. Consider a binary tree (of say depth $|C|$, where C denotes the statement we want to prove), and associate every node in the binary tree with a pair of one-time keys (pk, sk) . For a parent node associated with keys (pk, sk) , one generates a *fixed* set of keys for its children in the tree $(pk^{(0)}, sk^{(0)})$, $(pk^{(1)}, sk^{(1)})$, and proves using the 1T.UNIZK that the two pairs $(pk^{(0)}, sk^{(0)})$, $(pk^{(1)}, sk^{(1)})$ are generated correctly. For zero-knowledge, we crucially rely here on the fact that the keys $(pk^{(\beta)}, sk^{(\beta)})$ where $\beta \in \{0, 1\}$ are fixed once for all, so that we only need one proof to ever be generated using the parent keys (pk, sk) . We ensure this by generating $(pk^{(\beta)}, sk^{(\beta)})$ pseudorandomly using a PRF whose key we commit to as part of pk , and proving that $(pk^{(\beta)}, sk^{(\beta)})$ were indeed generated correctly from the committed PRF key using the 1T.UNIZK. Finally, to produce a proof for the statement/witness C, w we provide all the public keys and proofs along the path from the root to the leaf node indexed by C , and then generate the proof for the statement/witness C, w using the key pair at the leaf node of the path.

Summing up, in the new construction, the new public key consists of a one-time public key at the root node, and a commitment to a PRF key, and the new secret key consists of the associated one-time secret key

¹¹The above only yields selective security. We can obtain *adaptive* single-theorem zero-knowledge with a simple modification: instead of providing the garbled circuit $\tilde{U}$ in the clear as part of pk , we commit to it through an equivocal commitment and give the opening as part of the proof. We refer to Section 5 for more details.

¹²Commitments with all the needed properties can be instantiated from LWE, and in fact can satisfy a stronger form of statistically unique openings, along with statistical binding, which down the line translate to a stronger statistical version of uniqueness, along with statistical soundness; we refer to technical sections for more details, and ignore this distinction for the rest of this overview.

for the root node, along with the PRF key and openings to the commitment. Proofs consist of all the public keys pseudorandomly generated along the path to the leaf node indexed by the statement C , along with (1) 1T.UNIZK proofs, for every non-leaf node on the path, that the keys $(pk^{(\beta)}, sk^{(\beta)})$ of its children nodes were generated correctly using the committed PRF, and (2) a 1T.UNIZK proof of the statement generated using the secret key at the leaf node. Verification checks that all the proofs verify, and rejects if not.

As long as no leaf node is ever reused, multi-theorem zero-knowledge follows from single-theorem zero-knowledge of 1T.UNIZK, security of the PRF, and hiding of the commitment, where we switch pseudorandomly generated internal keys to fresh truly random keys, layer by layer.¹³ Here we crucially use the fact that only a single proof is generated for a given statement C , so that only a single proof is generated from a given key at a leaf node.

(Adaptive, knowledge) soundness of the protocol also follows readily: by adaptive soundness of 1T.UNIZK, one can iteratively argue, layer by layer, that all intermediate keys are pseudorandomly (and therefore semi-maliciously) generated. Therefore, so are the ones at the leaf nodes and (adaptive, knowledge) soundness of 1T.UNIZK at the leaf nodes ensures soundness of the construction.

Arguing uniqueness is slightly more involved. Suppose that an efficient cheating prover provides two valid proofs for a single statement C . We first argue that all the intermediate component proofs on the path determined by C (corresponding to internal nodes in the tree) have to be equal. In particular, we need to argue that (1) all the keys pk_i generated along the tree path are necessarily equal, and that (2) all the one-time proofs of well-formedness of the intermediate keys are also equal. We argue these layer by layer, starting from the root node and finishing at the leaf layer. Intuitively, to argue (1), if the public keys corresponding to a parent node are honestly (or semi-maliciously) generated, then by soundness of 1T.UNIZK and binding of the commitment, the public keys of the children have to be deterministically generated using the PRF, which ensures uniqueness of pk_i . To argue (2), we naturally rely on the uniqueness of 1T.UNIZK. To do so, we argue that the auxiliary statements underlying the intermediate 1T.UNIZK proofs have (computationally) unique witnesses. Recall that the witness here is a PRF key, along an opening to a commitment to this PRF key. By binding of the commitment, the PRF key that can be opened to is computationally unique. More generally, committing to a witness using a commitment with (computationally resp. statistically) unique openings allows to convert a statement into one with (computationally resp. statistically) unique witnesses. Now, by (1) and (2), the proofs have to differ at the leaf node, and we conclude by leveraging uniqueness of 1T.UNIZK at the leaf node.

Unfortunately, we skimmed over a very important subtlety in this exposition of this tree-based transformation, and the transformation cannot be instantiated as is. The issue is that the size of the public and secret keys of 1T.UNIZK, as well as the complexity of generating them, might grow in general with the size of the statements supported. Suppose for instance that proving statements of size n with 1T.UNIZK requires keys of size n . Then, to prove that the keys at both children nodes are generated correctly, one would need to have keys of size (at least) $2n$ for the parent node! Thus, applying the transformation above on such 1T.UNIZKs might result in a blow-up in the size of keys that is exponential in the depth of the tree. Note, in fact, that the construction of 1T.UNIZK sketched above *does* have its key grow with the statement size (as it is garbling the statement seen as a circuit).

As a result, to make the transformation work, we need to rely on a 1T.UNIZK with some specific *efficiency* property, which we describe next.

1T-UNIZK with Efficient Prover Setup. Recall that, in the tree-based transformation described above, the 1T.UNIZK scheme is used at each node to prove statements about (two) correct executions of the 1T.UNIZK setup for its children nodes. Thus, in order to make the tree-based transformation efficient and not have any blow up in parameters, it suffices to have a single instantiation of a 1T.UNIZK with some fixed parameters that would support proving such statements about its own setup.

To do so, we use an FHE-based transformation originally suggested by [Gen09] and also used in [GGI⁺15, BGT20], which allow us to generically improve the efficiency of NIZKs. In our case, we will instead view

¹³For technical reasons, we actually need to use a different PRF key for every layer of the tree. See paragraph “Comparison with [LMs05]” and Section 7 for more details.

them as a generic way to *expand* the set of statements supported by a given setup. Starting with a 1T.UNIZK, we defined a more efficient version as follows (while guaranteeing the same security properties). The prover setup phase will now consist in generating both the setup (pk, sk) of the 1T.UNIZK, along with keys $(FHE.pk, FHE.sk)$ for an FHE scheme. To prove a statement C with witness w , first encrypt w with the FHE, giving an FHE ciphertext ct_w . Compute homomorphically C on ct_w : this gives an FHE ciphertext $ct_{C(w)}$. We now prove using 1T.UNIZK that the ciphertext $ct_{C(w)}$ decrypts to 1 using as a witness $FHE.sk$. The proof now consists of ct_w and the one-time proof. Verification proceeds by computing homomorphically C on ct_w to obtain $ct_{C(w)}$, and checking that the one-time proof verifies.

However, the transformation described above does not guarantee uniqueness of proofs, so we need to modify it slightly. There are two potential barriers to uniqueness here: first, the generation of ct_w is randomized. This can be fixed by additionally committing during the prover setup to some randomness ρ_{Enc} to be used to encrypt, namely $ct_w = FHE.Enc(w; \rho_{Enc})$, and use 1T.UNIZK to prove that the ct_w is indeed an encryption of some w with randomness ρ_{Enc} . The second issue is that ρ_{Enc} and $FHE.sk$ (which are parts of the witnesses of the one-time proof) are not unique either in general. Similarly to our construction of 1T.UNIZK, this can be fixed by committing to these values via a commitment scheme with computationally unique openings. Overall, the public keys now include a public key for 1T.UNIZK, an FHE public key and commitments to ρ_{Enc} and the FHE secret key; the secret key consists of the secret key for 1T.UNIZK, the FHE secret key, ρ_{Enc} and openings to the commitments. We then use 1T.UNIZK to prove that (1) ct_w is generated from w and the ρ_{Enc} committed, and (2) that $ct_{C(w)}$ decrypts to 1 under the FHE secret key that is committed.

Soundness of the construction intuitively follows from soundness of 1T.UNIZK, binding of the commitment and (perfect) correctness of the FHE, and single-theorem zero-knowledge follows from security of the FHE, hiding of the commitment and one-time zero-knowledge of 1T.UNIZK.

Crucially, now 1T.UNIZK is only used to prove statements about FHE decryption, encryption, and validity of commitments, which might grow with the witness size but not the statement size.¹⁴ On the other hand, the well-formedness of public keys, seen as a statement, can be made to have small witness size, by using pseudorandom coins instead. Setting parameters appropriately, we obtain 1T.UNIZK that can prove statements about (two) correct executions of their own setup, as desired.

Plugging this efficient form of one-time UNIZK into the tree-based construction described above gives a multi-theorem UNIZK.

Security against Malicious Prover Setups. Last, we have so far argued soundness and uniqueness for *honest* or *semi-malicious* prover setup. To remove this restriction to handle arbitrarily malicious setup, we simply attach a standard (non-unique) NIZK to the public key pk , to prove that it was generated honestly using some (potentially maliciously chosen) randomness.

Comparison with [LMs05]. UNIZKs were originally introduced in [LMs05]. Although the notions of UNIZK studied in [LMs05] and this paper are conceptually very similar, there are small differences between the formal definitions. In their work, the property of uniqueness does not allow the adversarial pk to depend on the CRS. In contrast, ours does allow adaptivity. Also, their notion requires a stateful prover that keeps a counter while ours does not. The constructions in both works can be adapted to satisfy either definition.

The work of [LMs05] gives a construction of UNIZKs from the quadratic residuosity (QR) assumption, leaving the question of providing constructions from any other assumptions as an explicit open question. In contrast, our work gives a construction of UNIZKs from LWE.

On a technical level, the single-theorem to multi-theorem UNIZK transformation in our work is also conceptually similar to the transformation described in [LMs05]. Unfortunately, they only (informally) described the transformation and only provided brief sketches of proof. In fact, their approach suffers from a subtle circularity issue, and we do not see how to prove their construction secure. In more details, the construction of [LMs05] uses a single global PRF key to generate the prover setups for each node in the bootstrapping tree. To simulate the single-theorem proof of a parent node, we first need to switch the

¹⁴This overview assumes unleveled FHE for simplicity. However, with a little more work, we show that using leveled FHE from plain LWE in fact suffices.

prover setup of that node from being generated using the PRF key to using fresh randomness. The same PRF key, however, is used inside the single-theorem proofs in children nodes, but we cannot argue zero-knowledge of these children nodes as the well-formedness of their prover setup is proven using their parent node. Summing up, to argue zero-knowledge of a parent node, we first need to argue zero-knowledge of its children; but we can only argue zero-knowledge of a children node after arguing zero-knowledge of its parent node.

We break this circularity by not using a global PRF key for the entire tree, but instead having a separate PRF key for each level. This modification can also be directly implemented in [LMs05] to fix their proof of multi-theorem zero-knowledge.

In [LMs05], UNIZKs were introduced in as a technical tool to build *Fair Zero-Knowledge* (fair ZK). Fair ZK considers a setting of zero-knowledge with many verifiers, and aims to address the risks associated with collusion between a malicious prover and dishonest verifiers, by ensuring that no malicious verifier can obtain more information from an accepting proof than honest verifiers. More precisely, in fair ZK, if an honest verifier accepts a proof, then no other verifier is able to learn more from the proof than that the statement is in the language. In [LMs05], fair ZK protocols are shown to be generically implied by UNIZKs, thus resulting in a construction from QR. Our results also yield a fair ZK protocol from LWE.¹⁵

Comparison with [BGT20]. Overall, the techniques behind our construction of UNIZKs closely follow those of [BGT20] for constructing designated-prover NIZKs from FHE, including: (1) the construction of the one-time scheme 1T.UNIZK from garbled circuits, (2) upgrading the efficiency of the one-time scheme using FHE and (3) going from one-time to many-time using a tree-based construction. Interestingly, while the techniques are similar, the end-goals of the two works are quite different. One of the main contributions of our work is showing that these techniques can be adapted to achieve uniqueness. Doing so induces small technical differences, notably in the manner bootstrapping from single-theorem to multi-theorem zero-knowledge is performed. More precisely, in our construction, the public-keys of our 1T-UNIZK with efficient prover setup do grow with the witness size, which is a crucial modification that ensures uniqueness. Our bootstrapping step then proceeds by carefully analyzing what statements can be proven using a fixed public-key of the 1T-UNIZK, and showing that such statements suffice to perform the tree-based bootstrapping.

Beyond UNIZK. Recall that UNIZKs rely on two relaxations: (1) only requiring unique proofs π for NP statements x with a (computationally) unique witness w , (2) allowing for non-unique prover setup pk . We sketch why removing either of these relaxations implies witness PRFs, which in turn imply witness encryption.

We start with a result of [CPW23], which shows that “unique witness maps (UWMs)” imply witness PRFs. A unique witness map is a non-interactive proof system in the CRS model where a prover $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$ is given a crs , a statement x and a witness w and maps them to a proof π . The proof system should be (selectively) computationally sound and unique, where the latter means that for any statement x and any two witnesses w, w' , the honestly produced proofs $\pi \leftarrow \text{Prove}(\text{crs}, x, w), \pi' \leftarrow \text{Prove}(\text{crs}, x, w')$ should satisfy $\pi = \pi'$ with overwhelming probability. There is no zero-knowledge property. The work of [CPW23] shows that any such UWM implies witness PRFs and therefore witness encryption.¹⁶

Let us briefly sketch the result for witness encryption (WE). To encrypt a message m under an NP statement x , the encryptor chooses a PRG seed r , computes $y = \text{PRG}(r)$ and then constructs a UWM proof π for the statement $\text{st} = \text{“either } x \text{ is true or } y \text{ is in the image of PRG”}$ using the witness r . It then sends y and uses the Goldreich-Levin (GL) hard-core bit of π to one-time pad the message. The decryptor gets y and

¹⁵A small technical difference is that the construction of UNIZKs of [LMs05] uses a uniform common random string, whereas our construction in general uses a structured common reference string. However, a direct inspection of the construction of fair ZK of [LMs05] shows that it suffices that the CRS of the UNIZK is pseudorandom, which is the case in our construction if our commitment and CRHF keys are pseudorandom. These can both be instantiated from LWE ([Reg05, Ajt96]).

¹⁶Formally, they defined a stronger version of UWM where the prover is deterministic and $\text{Prove}(\text{crs}, x, w) = \text{Prove}(\text{crs}, x, w')$ for all witnesses w, w' for x . However, we observe that their result extends to the above version as is without any modifications to the proof.

computes a proof π' for the statement st using the witness w for x , guaranteeing $\pi' = \pi$ w.o.p.. It then uses the hard-core bit of π' to unmask the message. Assume that an adversary breaks WE security when the statement x is false. By GL this implies an adversary that, given $y = \text{PRG}(r)$, can compute the proof π with non-negligible probability. In particular it computes a valid proof π for the statement st . By PRG security, it must also do so if it is given a random y since it cannot distinguish. But in that case st is a false statement and hence the adversary breaks the soundness of the UWM.

Now, suppose we have a UNIZK proof with relaxation (1) but not (2). In particular, it suffices to assume we have a NIZK with a very weak notion of uniqueness: for any true statement x with witness w , if we generate two honest but independent proofs $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$ and $\pi^* \leftarrow \text{Prove}(\text{crs}, x, w)$ using the same witness then we have $\pi = \pi^*$ w.o.p.. Now take any other witnesses w' for x and let $\pi' \leftarrow \text{Prove}(\text{crs}, x, w')$. If for some such x, w, w' we don't have $\pi = \pi'$ w.o.p. then we can break witness indistinguishability and therefore zero-knowledge. In particular, the adversary that knows x, w can distinguish between proofs π generated using w and proofs π' generated using w' by independently generating its own proof π^* using w and comparing it to the received one – in the former case they are equal w.o.p. while in the latter they are not. Therefore, if the proof is a NIZK, it must be the case that for all x, w, w' we have $\pi = \pi'$ w.o.p., in which case it is a UWM and hence it implies witness PRFs and witness encryption by [CPW23].

On the other hand, suppose we have a UNIZK proof with relaxation (2) but not (1). In particular it suffices to assume a very weak notion of uniqueness where, for honestly generated setup (pk, sk) and for any statement x with witnesses w, w' if we generate $\pi \leftarrow \text{Prove}(\text{crs}, \text{sk}, x, w)$, $\pi' \leftarrow \text{Prove}(\text{crs}, \text{sk}, x, w')$ then $\pi = \pi'$ w.o.p.. Then we can convert such NIZK with setup to a standard UWM without setup. In particular, we define the $\text{crs}' = (\text{crs}, \text{pk}, \text{sk})$ of the UWM to include the crs of the UNIZK together with the prover public key and secret key, and the prove and verify algorithms are otherwise the same. Including sk in crs' kills zero-knowledge but it preserves soundness since soundness holds even for a malicious prover that knows sk . Therefore the resulting scheme is a UWM and hence it implies witness PRFs and witness encryption by [CPW23].

Roadmap. In Section 3, we recall basic notations and give definitions of the building blocks we use in our constructions. In Section 4, we define UNIZKs, along with relaxations that are helpful in our construction. In Section 5, we build a single-theorem version of UNIZK against semi-malicious prover setups. In Section 6, we generically improve the efficiency of the previously built weak UNIZK. In Section 7, we build a multi-theorem version of UNIZK starting from an efficient single-theorem one. In Section 8, we show how to upgrade security to handle malicious prover setups. In Section 9, we show how stronger forms of UNIZKs compared to one considered here would imply witness encryption and related primitives.

3 Preliminaries

Notations. When X is a distribution, or a random variable following this distribution, we let $x \leftarrow X$ denote the process of sampling x according to the distribution X . If X is a set, we let $x \leftarrow X$ denote sampling x uniformly at random from X ; if $\text{Alg}(\text{input})$ is a randomized algorithm, we denote by $x \leftarrow \text{Alg}(\text{input})$ the process of sampling an output of $\text{Alg}(\text{input})$ using uniformly random coins. We will also sometimes denote the random coins explicit: we write $x = \text{Alg}(\text{input}; \rho)$ to denote the output of computing $\text{Alg}(\text{input})$ with randomness ρ . We use the notation $[k]$ to denote the set $\{1, \dots, k\}$ where $k \in \mathbb{N}$, and $[0, 1]_{\mathbb{R}}$ to denote the real interval $\{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$. We denote by $\text{negl}(\lambda)$ functions f such that $f(\lambda) = 1/\lambda^{\omega(1)}$. Given two (ensembles of) distributions $X = \{X_\lambda\}, Y = \{Y_\lambda\}$, we will denote $X \approx_s Y$ the statement that the statistical distance between X and Y is negligible, $X \approx_c Y$ the statement that no PPT distinguisher can distinguish X from Y with better than negligible probability. For a string $\text{str} = \text{str}_1 \dots \text{str}_{\text{len}} \in \{0, 1\}^{\text{len}}$ and an index $1 \leq i \leq \text{len}$, we will denote by $\text{str}_{\leq i}$ its i -bit prefix $\text{str}_1 \dots \text{str}_i$.

Definition 3.1 (Pseudorandom Generators (PRGs)). *A PRG is a family of deterministic and efficient to compute functions $G : (\{0, 1\}^n \rightarrow \{0, 1\}^{m(n)})_{n \in \mathbb{N}}$ such that $m(n) > n$, and*

$$G(U_n) \approx_c U_{m(n)}.$$

Definition 3.2 (Pseudorandom Functions (PRFs)). A pseudorandom function is a couple of efficient algorithms (Setup, PRF), where Setup(1^λ) outputs a key K , and $\text{PRF}_K : \{0, 1\}^{m(n)} \leftarrow \{0, 1\}^{p(n)}$ is an efficient deterministic algorithm such that, for all PPT algorithms $\mathcal{A}$:

$$|\Pr[\mathcal{A}^{F_K(\cdot)}(1^n) = 1] - \Pr[\mathcal{A}^{R(\cdot)}(1^n) = 1]| \leq \text{negl}(\lambda),$$

over the randomness of $K \leftarrow \text{Setup}(1^\lambda)$, the internal randomness of $\mathcal{A}$, and where R is a uniformly random function.

We recall here the definition of witness PRFs.

Definition 3.3 (Witness PRFs (WPRFs) [Zha16]). A witness PRF for an NP language L with relation $R_L : \mathcal{X} \times \mathcal{W} \rightarrow \{0, 1\}$ is a triple of algorithms (wPRF.Gen, F_{fk} , wPRF.Eval) where:

- wPRF.Gen($1^\lambda, R_L$) $\rightarrow$ (fk, ek): a randomized algorithm that takes as input a security parameter λ and a relation R_L and outputs a secret function key fk and a public evaluation key ek.
- $F_{\text{fk}}(x) \rightarrow y$: a deterministic algorithm that takes as input a function key fk and a statement $x \in \mathcal{X}$ and outputs a value $y \in \mathcal{Y}$ for some set $\mathcal{Y}$.
- wPRF.Eval_{ek}(x, w) $\rightarrow \{y, \perp\}$: a deterministic algorithm that takes as input an evaluation key ek, a statement $x \in \mathcal{X}$, and a witness $w \in \mathcal{W}$ and outputs a value $y \in \mathcal{Y}$ or $\perp$.

It must satisfy the following properties:

Correctness. For all $x \in \mathcal{X}, w \in \mathcal{W}$,

$$\text{wPRF.Eval}_{\text{ek}}(x, w) = \begin{cases} F_{\text{fk}}(x) & \text{if } R_L(x, w) = 1 \\ \perp & \text{if } R_L(x, w) = 0 \end{cases}$$

Static Instance Interactive Security. We consider the following experiment $\text{EXP}_{\mathcal{A}}^{R_L}(\lambda, b)$ with a PPT adversary $\mathcal{A}$:

1. The adversary $\mathcal{A}$ declares challenge x^* and sends it to the challenger
2. The challenger computes (fk, ek) $\leftarrow$ Gen($1^\lambda, R_L$) and sends ek to $\mathcal{A}$ along with y_b where
 - $y_0 \leftarrow F_{\text{fk}}(x^*)$
 - $y_1 \leftarrow \mathcal{Y}$
3. $\mathcal{A}$ can adaptively make queries on instances $x_i \in X$ to the challenger who responds with $F_{\text{fk}}(x_i)$
4. $\mathcal{A}$ finally sends a bit b' to the challenger
5. The challenger outputs a uniform bit if $x^* \in L$ or if $x^* \in \{x_i\}$ otherwise it outputs b'

We say that a WPRF is static instance interactive secure if for all PPT adversaries $\mathcal{A}$ there exists a function negl such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{EXP}_{\mathcal{A}}^{R_L}(\lambda, 0) = 1] - \Pr[\text{EXP}_{\mathcal{A}}^{R_L}(\lambda, 1) = 1]| < \text{negl}(\lambda).$$

Non-Interactive and Adaptive Security. If we restrict the adversary $\mathcal{A}$ from making any queries to F then, we call the WPRF adaptive instance “non-interactive” secure. If we remove the requirement that the adversary $\mathcal{A}$ declares x^* before seeing ek then, we call the WPRF “adaptive” instance interactive (resp. non-interactive) secure.

Remark 3.4 (Static versus Adaptive Security.). We observe that one can boost a statically-secure witness PRF to one satisfying the adaptive notion of security (where x^* can be chosen after seeing ek), by relying on complexity leveraging. Therefore, all the lower bounds in this paper (Theorems 9.5, 9.6 and 9.9) extend to the non-static setting, assuming sub-exponential soundness and zero-knowledge of the UNIZK-V1, sub-exponentially secure PRFs, and UNIZK-V2, respectively.

Lemma 3.5 (Lattice trapdoors [MP12]). *There exists a PPT algorithm $\text{TrapGen}(1^n, 1^m, q) \rightarrow (\mathbf{A}, \mathsf{td})$, which on input some integers $n \geq 1, q \geq 2$ and $m \geq \Omega(n \log q)$, satisfies the following properties:*

- The distribution of $\mathbf{A}$ is within negligible statistical distance from $\mathcal{U}(\mathbb{Z}_q^{n \times m})$.
- There exists a polynomial-time, deterministic algorithm $\text{Invert}(\mathsf{td}_{\text{Ext}}, \mathbf{A}, \text{com})$, which on input $\text{com} = \mathbf{s}^T \mathbf{A} + \mathbf{e}^T$ where $\mathbf{s} \in \mathbb{Z}_q^m$ and $\mathbf{e} \in \mathbb{Z}_q^m$ such that $\|\mathbf{e}\| < q/\Omega(\sqrt{m})$, outputs $(\mathbf{s}, \mathbf{e})$ with overwhelming probability over the randomness of TrapGen alone.

Without loss of generality, the algorithm only outputs some $(\mathbf{s}, \mathbf{e})$ whenever $\text{com} = \mathbf{s}^T \mathbf{A} + \mathbf{e}^T$ and $\|\mathbf{e}\| < q/O(\sqrt{m})$, as these conditions can be checked efficiently.

Lemma 3.6 ([GPV08] Lemma 5.3). *Let n and q be positive integers with q prime and $m \geq 2n \log q$. Then for all but at most a q^{-n} fraction of $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ the length of the shortest non-zero column vector in the lattice generated by $\mathbf{A}$ is $\geq q/4$.*

Definition 3.7 (Equivocal/extractable commitments). *An equivocal/extractable commitment scheme consists of a tuple PPT algorithms $(\text{Gen}, \text{Commit}, \text{Verify}, \text{Ext}, \text{Gen}_{\text{Equiv}}, \text{Equiv})$ with the following syntax:*

- $\text{Gen}(1^\lambda) \rightarrow (\mathsf{ck}, \mathsf{td}_{\text{Ext}})$: takes as input a security parameter λ and outputs a common commitment key ck and a trapdoor td_{Ext} .¹⁷
- $\text{Commit}(\mathsf{ck}, x) \rightarrow (\text{com}, \text{decom})$: takes as input a commitment key ck and a message x , and outputs a commitment com and an opening decom .
- $\text{Verify}(\mathsf{ck}, x, \text{com}, \text{decom}) \rightarrow b$: takes as input a commitment key ck , a message x , a commitment com , and an opening decom , and outputs a bit $b \in \{0, 1\}$.
- $\text{Ext}(\mathsf{td}_{\text{Ext}}, \text{com}) \rightarrow x$: takes as input a trapdoor td_{Ext} and a commitment com and outputs a message x .
- $\text{Gen}_{\text{Equiv}}(1^\lambda) \rightarrow (\mathsf{ck}_{\text{Equiv}}, \mathsf{td}_{\text{Equiv}})$: takes as input a security parameter λ and outputs a common commitment key $\mathsf{ck}_{\text{Equiv}}$ and a trapdoor $\mathsf{td}_{\text{Equiv}}$.
- $\text{Equiv}(\mathsf{td}_{\text{Equiv}}, \text{com}, \text{decom}, x) \rightarrow \text{decom}'$: takes as input a trapdoor $\mathsf{td}_{\text{Equiv}}$, a commitment com , a decommitment decom , and a target message x , and deterministically outputs an opening decom' .¹⁸

An equivocal/extractable commitment scheme satisfies the following properties.

Perfect correctness. For all security parameters λ , we have, with overwhelming probability over $\mathsf{ck} \leftarrow \text{Gen}(1^\lambda)$, that, for all messages x :

$$\Pr[\text{Verify}(\mathsf{ck}, x, \text{com}, \text{decom}) = 1] = 1,$$

over the randomness of $(\text{com}, \text{decom}) \leftarrow \text{Commit}(\mathsf{ck}, x)$.

¹⁷In cases we do not use the second output td_{Ext} , we might simply omit it from the output of the algorithm. Looking ahead, td_{Ext} will only be used in our proofs of security.

¹⁸For proving the security of our constructions, each commitment we equivocate is honestly generated.

Extractability. For all PPT adversaries $\mathcal{A}$ we have that,

$$\Pr \left[\begin{array}{l} (\text{ck}, \text{td}_{\text{Ext}}) \leftarrow \text{Gen}(1^\lambda) \\ (\text{com}, \text{decom}, x) \leftarrow \mathcal{A}(\text{ck}) \end{array} : \begin{array}{l} \text{Verify}(\text{ck}, x, \text{com}, \text{decom}) = 1 \\ \wedge x \neq \text{Ext}(\text{td}_{\text{Ext}}, \text{com}) \end{array} \right] \leq \text{negl}(\lambda).^{19}$$

If extractability holds for all (potentially inefficient) adversaries $\mathcal{A}$, then we say that the commitment scheme satisfies statistical extractability.

Equivocability. For all messages x , the following distributions are computationally indistinguishable:

$$\left\{ \begin{array}{l} (\text{ck}, \text{td}_{\text{Equiv}}) \leftarrow \text{Gen}_{\text{Equiv}}(1^\lambda) \\ (\text{com}, \text{decom}) \leftarrow \text{Commit}(\text{ck}, x) \end{array} : (\text{td}_{\text{Equiv}}, x, \text{com}, \text{decom}) \right\}$$

and

$$\left\{ \begin{array}{l} (\text{ck}, \widetilde{\text{td}_{\text{Equiv}}}) \leftarrow \text{Gen}_{\text{Equiv}}(1^\lambda) \\ (\widetilde{\text{com}}, \text{decom}) \leftarrow \text{Commit}(\text{ck}, \mathbf{0}) \\ \text{decom}' \leftarrow \text{Equiv}(\text{td}_{\text{Equiv}}, \widetilde{\text{com}}, \text{decom}, x) \end{array} : (\text{td}_{\text{Equiv}}, x, \widetilde{\text{com}}, \text{decom}') \right\}.^{20}$$

If equivocability holds for all (potentially inefficient) adversaries $\mathcal{A}$, then we say that the commitment scheme satisfies statistical equivocability.

Indistinguishability of Modes. The following computational indistinguishability holds:

$$\{(\text{ck}, \text{td}_{\text{Ext}}) \leftarrow \text{Gen}(1^\lambda) : \text{ck}\} \approx_c \{(\text{ck}, \text{td}_{\text{Equiv}}) \leftarrow \text{Gen}_{\text{Equiv}}(1^\lambda) : \text{ck}\}$$

Note that combined with equivocability, indistinguishability of modes also implies hiding of the commitment, namely, for all messages x the following two distributions are computationally indistinguishable:

$$\left\{ \begin{array}{l} (\text{ck}, \text{td}_{\text{Ext}}) \leftarrow \text{Gen}(1^\lambda) \\ (\text{com}, \text{decom}) \leftarrow \text{Commit}(\text{ck}, x) \end{array} : (\text{ck}, \text{com}) \right\} \\ \left\{ \begin{array}{l} (\text{ck}, \text{td}_{\text{Ext}}) \leftarrow \text{Gen}(1^\lambda) \\ (\text{com}, \text{decom}) \leftarrow \text{Commit}(\text{ck}, \mathbf{0}) \end{array} : (\text{ck}, \text{com}) \right\}.$$

Uniqueness. For every PPT algorithm $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{ck}, x, \text{com}, \text{decom}) = \text{Verify}(\text{ck}, x, \text{com}, \text{decom}') = 1 \\ \wedge \text{decom} \neq \text{decom}' \end{array} \right] \leq \text{negl}(\lambda).$$

Given $\text{ck} \leftarrow \text{Gen}(1^\lambda)$, and $(x, \text{com}, \text{decom}, \text{decom}') \leftarrow \mathcal{A}(\text{ck})$.

If uniqueness holds for all (potentially inefficient) adversaries $\mathcal{A}$, then we say that the commitment scheme satisfies statistical uniqueness. The Dual-Regev Encryption Scheme from LWE can be interpreted directly to construct such a scheme.

Lemma 3.8. Assuming LWE with super-polynomial modulus-to-noise ratio, there exists an equivocal/extractable commitment scheme satisfying Definition 3.7 with statistical extractability and statistical uniqueness.

Remark 3.9 (Statistical equivocability). In fact, the construction that we provide below achieves a stronger form of equivocability where the distributions are statistically indistinguishable, though we will not use this stronger property in the rest of the paper.

Below, we give the construction of the commitment scheme along with discussion of the parameters.

¹⁹For our purposes, we need something slightly weaker. We need that extraction is valid for honestly generated commitments where $\mathcal{A}$ chooses the randomness for $(\text{com}, \text{decom}) \leftarrow \text{Commit}(\text{ck}, x)$.

²⁰Note that, combined with correctness, this in particular implies that $\text{Verify}(\text{ck}, x, \widetilde{\text{com}}, \widetilde{\text{decom}}) = 1$ with overwhelming probability.

Parameters. We fix parameters $n, m, q, \beta \in \mathbb{Z}, B$ and, B -bounded LWE error distribution χ over $\mathbb{Z}$, all of which are functions of λ . In particular we set $n = \text{poly}(\lambda)$, $q = 2^{n^\epsilon}$ for $0 < \epsilon < 1$, $m = \theta(n \log q)$, $\beta = q/n^2$, $\beta/B = n^{\omega(1)}$.

Construction.

- $\text{Gen}(1^n, 1^m, q)$: Sample a matrix and lattice trapdoor $(\mathbf{A}, \text{td}_{\text{Ext}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$ (Lemma 3.5) where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\text{td}_{\text{Ext}} \in \mathbb{Z}_q^{n \times m}$, and $\mathbf{r} \leftarrow \mathbb{Z}_q^m$. Set $\text{ck} = (\mathbf{A}, \mathbf{r})$ and output $(\text{ck}, \text{td}_{\text{Ext}})$.
- $\text{Commit}(\text{ck}, b)$: Parse $\text{ck} = (\mathbf{A}, \mathbf{r})$. Sample a vector $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, $\mathbf{e} \leftarrow [-\beta, \beta]^m$. Compute $\text{com} = \mathbf{s}^T \mathbf{A} + \mathbf{e}^T + b \cdot \mathbf{r}^T$ and set $\text{decom} = (\mathbf{s}, \mathbf{e})$. Output $(\text{com}, \text{decom})$.
- $\text{Verify}(\text{ck}, b, \text{com}, \text{decom})$: Parse $\text{ck} = (\mathbf{A}, \mathbf{r})$, $\text{decom} = (\mathbf{s}, \mathbf{e})$. If $\|\mathbf{e}\|_\infty \leq \beta$ and $\text{com} = \mathbf{s}^T \mathbf{A} + \mathbf{e}^T + b \cdot \mathbf{r}^T$ output 1 otherwise, output 0.
- $\text{Ext}(\text{ck}, \text{td}_{\text{Ext}}, \text{com})$: Parse $\text{ck} = (\mathbf{A}, \mathbf{r})$ and compute $(\mathbf{s}, \mathbf{e}) \leftarrow \text{Invert}(\text{td}_{\text{Ext}}, \mathbf{A}, \text{com})$. If $\|\mathbf{e}\|_\infty \leq \beta$ and $\text{com} = \mathbf{s}^T \mathbf{A} + \mathbf{e}$, output 0. Else, compute $(\mathbf{s}, \mathbf{e}) \leftarrow \text{Invert}(\text{td}_{\text{Ext}}, \mathbf{A}, \text{com} - \mathbf{r})$ and if $\|\mathbf{e}\|_\infty \leq \beta$ and $\text{com} - \mathbf{r} = \mathbf{s}^T \mathbf{A} + \mathbf{e}$, output 1. Otherwise, output $\perp$.
- $\text{Gen}_{\text{Equiv}}(1^\lambda) \rightarrow (\text{ck}_{\text{Equiv}}, \text{td}_{\text{Equiv}})$: Sample matrix $\tilde{\mathbf{A}} \in \mathbb{Z}_q^{n \times m}$ and vectors $\tilde{\mathbf{s}} \leftarrow \mathbb{Z}_q^n, \tilde{\mathbf{e}} \leftarrow \chi^m$ where χ is the $(B$ -bounded) LWE error distribution. Compute $\tilde{\mathbf{r}}^T = \tilde{\mathbf{s}}^T \tilde{\mathbf{A}} + \tilde{\mathbf{e}}^T$ and set $\text{ck}_{\text{Equiv}} = (\tilde{\mathbf{A}}, \tilde{\mathbf{r}})$. Define $\text{td}_{\text{Equiv}} = (\tilde{\mathbf{s}}, \tilde{\mathbf{e}})$ and output $(\text{ck}_{\text{Equiv}}, \text{td}_{\text{Equiv}})$
- $\text{Equiv}(\text{td}_{\text{Equiv}}, \text{com}, \text{decom}, b)$: Parse $\text{td}_{\text{Equiv}} = (\tilde{\mathbf{s}}, \tilde{\mathbf{e}})$ and $\text{decom} = (\mathbf{s}, \mathbf{e})$. If $b = 0$, output $(\mathbf{s}, \mathbf{e})$ otherwise, output $(\mathbf{s} + \tilde{\mathbf{s}}, \mathbf{e} + \tilde{\mathbf{e}})$.

We now give the proof to Lemma 3.8.

Proof.

Claim 3.10 (Perfect Correctness). *The construction is perfectly correct.*

Proof. Correctness holds by construction of the commitment. By definition, Commit samples $\mathbf{e}$ with norm $\leq \beta$ so,

$$\begin{aligned} & \Pr \left[\begin{array}{c} \text{ck} \leftarrow \text{Gen}(1^\lambda) \\ (\text{com}, \text{decom}) \leftarrow \text{Commit}(\text{ck}, b) \end{array} : \text{Verify}(\text{ck}, b, \text{com}, \text{decom}) = 1 \right] \\ &= \Pr [\|\mathbf{e}\|_\infty \leq \beta \wedge \text{com} = \mathbf{s}^T \mathbf{A} + \mathbf{e}^T + b \cdot \mathbf{r}^T] = 1 \end{aligned}$$

□

Next, we turn to extractability, which follows from Lemma 3.5 and Lemma 3.6.

Claim 3.11 (Extractability). *The construction satisfies (statistical) extractability.*

Proof. Let $b \in \{0, 1\}$. By the definition of Gen , the construction uses $(\mathbf{A}, \text{td}_{\text{Ext}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$ and $\text{ck} = (\mathbf{A}, \mathbf{r})$ for $\mathbf{r} \leftarrow \mathbb{Z}_q^m$. By definition of Ext ,

$$\Pr \left[\begin{array}{c} \exists (b, \text{com}, \text{decom}) \\ \wedge b \neq \text{Ext}(\text{td}_{\text{Ext}}, \text{com}) \end{array} : \text{Verify}(\text{ck}, b, \text{com}, \text{decom}) = 1 \right]$$

is at most the probability that exists some com such that either:

- (1) $\text{Invert}(\text{td}_{\text{Ext}}, \mathbf{A}, \text{com})$ fails to outputs $(\mathbf{s}, \mathbf{e})$ with $\|\mathbf{e}\|_\infty \leq \beta$ such that $\text{com} = \mathbf{s}^T \mathbf{A} + \mathbf{e}^T$ while there exists decom such that $\text{Verify}(\text{ck}, 0, \text{com}, \text{decom}) = 1$, or $\text{Invert}(\text{td}_{\text{Ext}}, \mathbf{A}, \text{com} - \mathbf{r}^T)$ fails to outputs $(\bar{\mathbf{s}}, \bar{\mathbf{e}})$ with $\|\bar{\mathbf{e}}\|_\infty \leq \beta$ such that $\text{com} - \mathbf{r}^T = \bar{\mathbf{s}}^T \mathbf{A} + \bar{\mathbf{e}}^T$ while there exists decom such that $\text{Verify}(\text{ck}, 1, \text{com}, \text{decom}) = 1$,
- (2) $\text{Invert}(\text{td}_{\text{Ext}}, \mathbf{A}, \text{com})$ succeeds in finding $(\mathbf{s}, \mathbf{e})$ such that $\|\mathbf{e}\|_\infty \leq \beta$ and $\text{com} = \mathbf{s}^T \mathbf{A} + \mathbf{e}^T$ and also there exists $(\bar{\mathbf{s}}, \bar{\mathbf{e}})$ such that $\text{com} - \mathbf{r}^T = \bar{\mathbf{s}}^T \mathbf{A} + \bar{\mathbf{e}}^T$, however there exists decom such that $\text{Verify}(\text{ck}, 1, \text{com}, \text{decom}) = 1$.²¹

For (1), since we set $q/\beta = n^2 > \Omega(\sqrt{m})$, then by Lemma 3.5 we have that for any com :

$$\begin{aligned} \Pr \left[\begin{array}{l} (\mathbf{s}, \mathbf{e}) \leftarrow \text{Invert}(\text{td}_{\text{Ext}}, \mathbf{A}, \text{com}) \\ \text{com} \neq \mathbf{s}^T \mathbf{A} + \mathbf{e}^T \\ \exists \text{decom}, \text{Verify}(\text{ck}, 0, \text{com}, \text{decom}) = 1 \end{array} : \begin{array}{l} \|\mathbf{e}\|_\infty \leq \beta \\ \wedge \end{array} \right] &\leq \text{negl}(\lambda), \\ \Pr \left[\begin{array}{l} (\bar{\mathbf{s}}, \bar{\mathbf{e}}) \leftarrow \text{Invert}(\text{td}_{\text{Ext}}, \mathbf{A}, \text{com} - \mathbf{r}^T) \\ \text{com} - \mathbf{r}^T \neq \bar{\mathbf{s}}^T \mathbf{A} + \bar{\mathbf{e}}^T \\ \exists \text{decom}, \text{Verify}(\text{ck}, 1, \text{com}, \text{decom}) = 1 \end{array} : \begin{array}{l} \|\bar{\mathbf{e}}'\|_\infty \leq \beta \\ \wedge \end{array} \right] &\leq \text{negl}(\lambda), \\ \Pr \left[\perp \leftarrow \text{Invert}(\text{td}_{\text{Ext}}, \mathbf{A}, \text{com}) : \begin{array}{l} \exists \text{decom}, \text{Verify}(\text{ck}, 0, \text{com}, \text{decom}) = 1 \\ \vee \exists \text{decom}, \text{Verify}(\text{ck}, 1, \text{com}, \text{decom}) = 1 \end{array} \right] &\leq \text{negl}(\lambda), \end{aligned}$$

where the probabilities are over the randomness of $(\text{ck}, \text{td}_{\text{Ext}}) \leftarrow \text{Gen}(1^\lambda)$ alone.

The event (2) can only occur if (independently of any decom) if there exists $(\mathbf{s}, \mathbf{e}), (\bar{\mathbf{s}}, \bar{\mathbf{e}})$ such that

$$\begin{aligned} \text{com} &= \mathbf{s}^T \mathbf{A} + \mathbf{e}^T \\ \text{com} - \mathbf{r}^T &= \bar{\mathbf{s}}^T \mathbf{A} + \bar{\mathbf{e}}^T \end{aligned}$$

and $\|\mathbf{e}\|_\infty \leq \beta, \|\bar{\mathbf{e}}\|_\infty \leq \beta$, which can only occur if $\mathbf{r}^T = \tilde{\mathbf{s}}^T \mathbf{A} + \tilde{\mathbf{e}}^T$ for some $\tilde{\mathbf{s}}, \tilde{\mathbf{e}}$ such that $\|\tilde{\mathbf{e}}\|_\infty \leq 2\beta$.

In particular, there would exist a short vector $\tilde{\mathbf{e}}$ (of norm at most 2β) in the lattice generated by the rows of $\mathbf{A}$ and $\mathbf{r}$ (in other words, the vertical concatenation of $\mathbf{A}$ and $\mathbf{r}$). By Lemma 3.6 and our setting of parameters, the probability that such a short vector exists is at most $\leq q^{-n}$, and the conclusion follows. $\square$

Next, we turn to equivocability, which follows from noise flooding and our setting of parameters.

Claim 3.12 (Equivocability). *The construction satisfies (statistical) equivocability.*

Proof. For $b \in \{0, 1\}$ we aim to show that the following two distributions are statistically indistinguishable,

$$\left\{ \begin{array}{l} (\text{ck}_{\text{Equiv}}, \text{td}_{\text{Equiv}}) \leftarrow \text{Gen}_{\text{Equiv}}(1^\lambda) \\ (\text{com}, \text{decom}) \leftarrow \text{Commit}(\text{ck}_{\text{Equiv}}, b) \end{array} : (\text{td}_{\text{Equiv}}, b, \text{com}, \text{decom}) \right\}$$

and

$$\left\{ \begin{array}{l} (\text{ck}_{\text{Equiv}}, \text{td}_{\text{Equiv}}) \leftarrow \text{Gen}_{\text{Equiv}}(1^\lambda) \\ \widetilde{\text{com}} \leftarrow \text{Commit}(\text{ck}_{\text{Equiv}}, 0) \\ \text{decom}' \leftarrow \text{Equiv}(\text{td}_{\text{Equiv}}, \widetilde{\text{com}}, \text{decom}, b) \end{array} : (\text{td}_{\text{Equiv}}, b, \widetilde{\text{com}}, \text{decom}') \right\}.$$

We write $\widetilde{\text{com}} = \mathbf{s}'^T \tilde{\mathbf{A}} + \mathbf{e}'^T$ for $\mathbf{s}' \leftarrow \mathbb{Z}_q^n$ and $\mathbf{e}' \leftarrow [-\beta, \beta]^m$, and $\mathbf{r} = \tilde{\mathbf{s}}^T \tilde{\mathbf{A}} + \tilde{\mathbf{e}}$ for $\tilde{\mathbf{s}} \leftarrow \mathbb{Z}_q^n$ and $\tilde{\mathbf{e}} \leftarrow \chi^m$. We consider two cases: (1) $b = 0$ and (2) $b = 1$.

In the first case, by definition, $\text{Commit}(\text{ck}_{\text{Equiv}}, 0)$ outputs $(\text{com}_0, \text{decom}_0)$ where $\text{com}_0 = \mathbf{s}^T \tilde{\mathbf{A}} + \mathbf{e}^T$ and $\text{decom}_0 = (\mathbf{s}, \mathbf{e})$ for $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ and $\mathbf{e} \leftarrow [-\beta, \beta]^m$. If we equivocate to 0, then $\text{Equiv}(\text{ck}_{\text{Equiv}}, \widetilde{\text{com}}, \text{decom}_0, 0)$ outputs $(\mathbf{s}', \mathbf{e}')$ (defined in the previous paragraph). So, the distribution $\{\text{td}_{\text{Equiv}}, 0, \text{com}_0, (\mathbf{s}, \mathbf{e})\}$ is equal to the distribution $\{\text{td}_{\text{Equiv}}, 0, \widetilde{\text{com}}, (\mathbf{s}', \mathbf{e}')\}$.

²¹This is because if (1) doesn't occur, then if there exists decom such that $\text{Verify}(\text{ck}, 0, \text{com}, \text{decom}) = 1$, then Ext outputs 0.

In the second case, by definition $\text{Commit}(\text{ck}_{\text{Equiv}}, 1)$ outputs $(\text{com}_1, \text{decom}_1)$ where $\text{com}_1 = \mathbf{s}^{*T} \tilde{\mathbf{A}} + \mathbf{e}^{*T} + \tilde{\mathbf{r}}^T$ and $\text{decom}_1 = (\mathbf{s}^*, \mathbf{e}^*)$. If we equivocate to 1, then $\text{Equiv}(\text{ck}_{\text{Equiv}}, \widetilde{\text{com}}, \widetilde{\text{decom}}, 1)$ outputs $(\mathbf{s}^* + \tilde{\mathbf{s}}, \mathbf{e}^* + \tilde{\mathbf{e}})$. We next show that the following two distributions are statistically indistinguishable:

$$\{\text{td}_{\text{Equiv}}, 1, \text{com}_1, (\mathbf{s}', \mathbf{e}')\} \quad \text{and} \quad \{\text{td}_{\text{Equiv}}, 1, \widetilde{\text{com}}, (\mathbf{s}^* + \tilde{\mathbf{s}}, \mathbf{e}^* + \tilde{\mathbf{e}})\}.$$

Both $\mathbf{s}', \mathbf{s}^*, \tilde{\mathbf{s}}$ are sampled uniformly from $\mathbb{Z}^n$ and as such $\mathbf{s}'$ and $\mathbf{s}^* + \tilde{\mathbf{s}}$ are equally distributed. In the first distribution $\mathbf{e}$ is sampled from $[-\beta, \beta]^m$ while in the second we have a sum of $\mathbf{e}^* \in [-\beta, \beta]^m$ and $\tilde{\mathbf{e}} \in \chi^m$ where χ is B -bounded. By noise flooding and our setting of parameters (namely $\beta/B = n^{\omega(1)}$), the distributions of $(\mathbf{e}^* + \tilde{\mathbf{e}})$ is statistically close to that of $\mathbf{e}$. Overall, the distribution of $\text{com}_1 = (\mathbf{s}^* + \tilde{\mathbf{s}})^T \tilde{\mathbf{A}} + (\mathbf{e}^* + \tilde{\mathbf{e}})^T$ is statistically close to that of $\widetilde{\text{com}} = \mathbf{s}'^T \tilde{\mathbf{A}} + \mathbf{e}'^T$. $\square$

Claim 3.13 (Indistinguishability of Modes). *Assuming the hardness of LWE with our parameters, the construction satisfies indistinguishability of modes.*

Proof. Let $(\text{ck}_{\text{Ext}}, \text{td}_{\text{Ext}}) \leftarrow \text{Gen}(1^\lambda)$; we write $\text{ck}_{\text{Ext}} = (\mathbf{A}, \mathbf{r})$ for $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$ and $\mathbf{r} \leftarrow \mathbb{Z}_q^m$. Let $(\text{ck}_{\text{Equiv}}, \text{td}_{\text{Equiv}}) \leftarrow \text{Gen}_{\text{Equiv}}(1^\lambda, \text{ck})$; we write $\text{ck}_{\text{Equiv}} = (\tilde{\mathbf{A}}, \tilde{\mathbf{r}})$ for $\tilde{\mathbf{A}} \leftarrow \mathbb{Z}_q^{n \times m}$ and $\tilde{\mathbf{r}}^T = \tilde{\mathbf{s}}^T \tilde{\mathbf{A}} + \tilde{\mathbf{e}}^T$, $\mathbf{s} \leftarrow \mathbb{Z}^n$, and $\tilde{\mathbf{e}} \leftarrow \chi^m$. By LWE, $(\mathbf{A}, \mathbf{r})$ is computationally indistinguishable from $(\tilde{\mathbf{A}}, \tilde{\mathbf{s}}^T \tilde{\mathbf{A}} + \tilde{\mathbf{e}}^T)$. $\square$

Claim 3.14 (Uniqueness). *The construction satisfies (statistical) uniqueness.*

Proof. Let $b \in \{0, 1\}$ then,

$$\Pr \left[\begin{array}{c} \text{Verify}(\text{ck}, b, \text{com}, \text{decom}) = \text{Verify}(\text{ck}, b, \text{com}, \text{decom}') = 1 \\ \wedge \quad \text{decom} \neq \text{decom}' \end{array} \right] \quad (1)$$

$$= \Pr \left[\begin{array}{c} \exists (\mathbf{s}, \mathbf{e}, \hat{\mathbf{s}}, \hat{\mathbf{e}}) : \\ \wedge \quad \begin{array}{c} (\mathbf{s} \neq \hat{\mathbf{s}}) \vee (\mathbf{e} \neq \hat{\mathbf{e}}) \\ \mathbf{s}^T \mathbf{A} + \mathbf{e} = \hat{\mathbf{s}}^T \mathbf{A} + \hat{\mathbf{e}} \end{array} \end{array} \right] \quad (2)$$

$$= \Pr \left[\begin{array}{c} \exists (\mathbf{s}, \mathbf{e}, \hat{\mathbf{s}}, \hat{\mathbf{e}}) : \\ \wedge \quad \begin{array}{c} (\mathbf{s} \neq \hat{\mathbf{s}}) \vee (\mathbf{e} \neq \hat{\mathbf{e}}) \\ (\mathbf{s} - \hat{\mathbf{s}})^T \mathbf{A} = (\hat{\mathbf{e}} - \mathbf{e}) \end{array} \end{array} \right] \quad (3)$$

$$\leq q^{-n} \quad (4)$$

Line (2) follows from the definition of Verify , and line (4) follows from Lemma 3.6, along the fact that $\mathbf{A}$ is full rank over $\mathbb{Z}_q$ with overwhelming probability (given $m = \Theta(n \log q)$). $\square$

Definition 3.15 (Garbled Circuits). *A garbling scheme consists of two PPT algorithms ($\text{Garble}, \text{Eval}$) with the following syntax:*

- $\text{Garble}(1^\lambda, C; r) \rightarrow (\tilde{C}, \{\text{lab}_{i,b}\}_{i \in [n], b \in \{0,1\}})$: takes as input a security parameter λ , a circuit C , and outputs a garbled circuit $\tilde{C}$ and a set of labels $\{\text{lab}_{i,b}\}$ for $i \in [n]$ and $b \in \{0, 1\}$
- $\text{Eval}(1^\lambda, \tilde{C}, \{\text{lab}_i\}_{i \in [n]}) \rightarrow e$: takes as input a security parameter λ , a garbled circuit, and n labels $\{\text{lab}_i\}_{i \in [n]}$ and outputs a bit $e \in \{0, 1\}$.

A garbling scheme satisfies the following properties.

Perfect correctness. *For every circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}$, every input $x \in \{0, 1\}^n$, and every $(\tilde{C}, \{\text{lab}_{i,b}\}_{i \in [n], b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, C)$,*

$$\Pr[C(x) = \text{Eval}(\tilde{C}, \{\text{lab}_{i,x_i}\}_{i \in [n]})] = 1.$$

Security. *There exists a simulator S such that for every circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}$ and every input $x \in \{0, 1\}^n$, the following distributions are indistinguishable:*

$$(\tilde{C}, \{\text{lab}_{i,b}\}_{i \in [n], b \in \{0,1\}}) \approx_c S(1^\lambda, 1^{|C|}, C(x)).$$

Definition 3.16 (Fully Homomorphic Encryption). *A (leveled) fully homomorphic encryption scheme consists of four PPT algorithms (Gen, Enc, Eval, Dec) with the following syntax:*

- $\text{Gen}(1^\lambda, 1^h) \rightarrow (\text{pk}, \text{sk}, \text{ek})$: takes as input a security parameter λ and a circuit depth bound h , and outputs a public key pk , a secret key sk , and an evaluation key ek .
- $\text{Enc}_{\text{pk}}(x) \rightarrow \text{ct}$: takes as input a message x and outputs a ciphertext ct .
- $\text{Eval}_{\text{ek}}(C, \text{ct}) \rightarrow \text{ct}'$: takes as input an evaluation key ek , a circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}$, and a ciphertext ct and deterministically outputs a ciphertext ct' .
- $\text{Dec}_{\text{sk}}(\text{ct}) \rightarrow x'$: takes as input a secret key sk , and a ciphertext ct and outputs a string x' .

Perfect correctness. *For every (pk, sk) in the support of $\text{Gen}(1^\lambda, 1^h)$, we have that for every circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}$ of depth at most h and every message x :*

$$\Pr[\text{Dec}_{\text{sk}}(\text{Eval}_{\text{ek}}(C, \text{ct})) = C(x) \wedge \text{Dec}_{\text{sk}}(\text{ct}) = x] = 1$$

over the randomness of $\text{ct} \leftarrow \text{Enc}_{\text{pk}}(x)$.

Security. *For any PPT adversary $\mathcal{A}$ and any messages $x_0, x_1 \in \{0, 1\}^n$, we have*

$$|\Pr[\mathcal{A}(\text{pk}, \text{ek}, \text{Enc}_{\text{pk}}(x_0)) = 1] - \Pr[\mathcal{A}(\text{pk}, \text{ek}, \text{Enc}_{\text{pk}}(x_1)) = 1]| = \text{negl}(\lambda)$$

over the randomness of $(\text{pk}, \text{sk}, \text{ek}) \leftarrow \text{Gen}(1^\lambda, 1^h)$.

Compactness. *There exists some polynomial $p = \text{poly}(\lambda, h)$ such that for every security parameter λ and depth bound h , every message x , every circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}$ with depth at most h , every ciphertext computed as $\text{ct} \leftarrow \text{Enc}_{\text{pk}}(x)$ or $\text{ct}' \leftarrow \text{Eval}(C, \text{ct})$ has size at most p .*

We consider additionally the following properties.

Bootstrappability. *There exists a fixed bound $h' = h'(\lambda)$ such that for all $(\text{pk}, \text{sk}, \text{ek}) \leftarrow \text{Gen}(1^\lambda, 1^{h'})$, the decryption circuit $\text{Dec}_{\text{sk}}(\cdot)$ has depth at most $h' - 1$.*

We note that known constructions of FHE from LWE (e.g. [BV11, GSW13]) satisfy all the properties above (up to truncating the gaussians to generate the keys, to obtain perfect correctness). Next, we observe that the algorithm Gen can be generically made more efficient in the following sense.

Claim 3.17 (Efficient Setup for FHE). *Suppose the existence of an FHE with all the properties above. Then there exists an FHE scheme $(\text{Gen}, \text{Enc}, \text{Eval}, \text{Dec})$ with all the properties above such that for all $h = \text{poly}(\lambda)$, $\text{Gen}(1^\lambda, 1^h)$ can be computed in depth $\text{poly}(\lambda)$ for some polynomial poly independent of h . Furthermore, Enc and Dec run in time $\text{poly}(\lambda)$ independent of h .*

We do so by *bootstrapping* a base FHE scheme to support any arbitrary polynomial depth bound $h = \text{poly}(\lambda)$ [Gen09]. We do briefly describe the construction. We compute a chain of independent FHE keys $(\text{pk}_0, \text{sk}_0, \text{ek}_0), \dots, (\text{pk}_h, \text{sk}_h, \text{ek}_h)$ (where each keys are generated with depth bound h' given by bootstrappability). We set the new public key as $\text{pk} = \text{pk}_0$ and $\text{sk} = \text{sk}_h$. The new evaluation key ek includes all the ek_i , along with additional ciphertexts $\text{ct}_i \leftarrow \text{Enc}_{\text{pk}_i}(\text{sk}_{i-1})$ (where $i \in [h]$). Homomorphic evaluation

proceeds by iteratively homomorphically decrypting the current ciphertext under the next public key and performing a (here single) gate operation over this new key.

We note that the depth of generating the keys of the new scheme is $\text{poly}(\lambda)$: this is because all the h sets of keys can be generated in parallel, after which the ciphertexts ct_i can also be generated in parallel. Furthermore, FHE.Enc (respectively FHE.Dec) corresponds to encrypt with respect to pk_0 (respectively decrypt with respect to sk_0), so that their complexity is a polynomial $\text{poly}(\lambda)$ independent of h .

4 Unique NIZKs

In this section, we define our main notion of unique NIZKs (UNIZKs) and variants.

In the following, we use as a canonical NP-complete language Circuit-SAT, where instances consist of a circuit C of size n with a 1-bit output, witnesses consist of inputs of size ℓ , and a tuple (C, w) is in the relation $R = R_{\text{Circuit-SAT}}$ iff $C(w) = 1$.

Definition 4.1 (Unique NIZK (UNIZK) (Adapted from [LMs05])). *A unique NIZK (UNIZK) for the Circuit-SAT language consists of five PPT algorithms (Setup, Gen, Prove, Verify, Ext) with the following syntax:*

- $\text{Setup}(1^n, 1^\ell, 1^\lambda) \rightarrow (\text{crs}, \text{td}_{\text{Ext}})$: takes as input a circuit size n , an input size ℓ , the security parameter λ and outputs a common reference string crs , and an extraction trapdoor td_{Ext} .²²
- $\text{Gen}(\text{crs}; \rho) \rightarrow (\text{pk}, \text{sk})$: takes as input a common reference string crs and outputs a pair of keys (pk, sk) .
- $\text{Prove}(\text{crs}, \text{sk}, C, w) \rightarrow \pi$: takes as input a common reference string crs , a secret key sk , a statement C and a witness w , and deterministically outputs a proof π .
- $\text{Verify}(\text{crs}, \text{pk}, C, \pi) \rightarrow b$: takes as input a common reference string crs , a public key pk , a statement C and a proof π and outputs a bit $b \in \{0, 1\}$.
- $\text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi) \rightarrow w$: takes as input an extraction trapdoor td_{Ext} , a public key pk and a proof π , and deterministically outputs a witness w .

We require the following properties from an UNIZK.

Completeness. For all polynomials $n = n(\lambda)$ and $\ell = \ell(\lambda)$, for every C, w such that C has size $|C| \leq n$ taking inputs of size $|w| = \ell$ and such that $C(w) = 1$, we have:

$$\Pr[\text{Verify}(\text{crs}, \text{pk}, C, \pi) = 1] \geq 1 - \text{negl}(\lambda),$$

over the randomness of $(\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs})$, $\pi \leftarrow \text{Prove}(\text{crs}, \text{sk}, C, w)$, and the internal randomness of Verify .

Adaptive, Statistical Knowledge Soundness with Malicious Keys. For all polynomials $n = n(\lambda)$ and $\ell = \ell(\lambda)$, for every cheating prover $\mathcal{P}^*$, we have:

$$\Pr \left[\begin{array}{l} (\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, C^*, \pi) \leftarrow \mathcal{P}^*(\text{crs}), \\ w^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1 \\ \wedge C^*(w^*) = 0 \end{array} \right] \leq \text{negl}(\lambda),$$

where we require that $|C^*| = n$ and $|w^*| = \ell$.

²²Note: We make the extraction trapdoor td_{Ext} part of the setup algorithm, because it is more convenient for defining adaptive soundness.

Adaptive, Multi-Theorem Zero-Knowledge. For all polynomials $n = n(\lambda)$ and $\ell = \ell(\lambda)$, there exists a PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ such that the following properties hold.

- CRS indistinguishability: The following two distributions are computationally indistinguishable:

$$\text{crs} \approx_c \widetilde{\text{crs}},$$

where $(\text{crs}, \text{td}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda)$ and $(\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda)$.

- Multi-Theorem Zero-Knowledge with Simulated CRS: For all PPT adversaries $\mathcal{A}$, the two following distributions are computationally indistinguishable:

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\widetilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Prove}}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}) \end{array} : b \right\}$$

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\widetilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sim}_2}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}) \end{array} : b \right\},$$

where $\mathcal{O}_{\text{Prove}}$ and $\mathcal{O}_{\text{Sim}_2}$ are oracles defined as follows:

- $\mathcal{O}_{\text{Prove}}(C, w)$ is an oracle that has hard-coded crs, sk , which, on input (C, w) where $|C| = n, |w| = \ell$ and $C(w) = 1$, outputs $\pi \leftarrow \text{Prove}(\text{crs}, \text{sk}, C, w)$, and otherwise outputs $\perp$;
- $\mathcal{O}_{\text{Sim}_2}(C, w)$ is an oracle that has hard-coded $\text{td}_{\text{Sim}}, \text{pk}$, which, on input (C, w) where $|C| = n, |w| = \ell$ and $C(w) = 1$, outputs $\pi \leftarrow \text{Sim}_2(\text{td}_{\text{Sim}}, \text{pk}, C)$,

and we restrict $\mathcal{A}$ so that, for every C , it makes at most one oracle call on any input of the form (C, w) .

Adaptive, Statistical Uniqueness with Malicious Keys. For all polynomials $n = n(\lambda)$ and $\ell = \ell(\lambda)$, and for every cheating prover $\mathcal{P}^*$, we have:

$$\Pr \left[\begin{array}{l} (\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, C^*, \pi_0, \pi_1) \leftarrow \mathcal{P}^*(\text{crs}), \\ w_0^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi_0), \\ w_1^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi_1) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi_0) = 1 \\ \wedge \text{Verify}(\text{crs}, \text{pk}, C^*, \pi_1) = 1 \\ \wedge \pi_0 \neq \pi_1 \\ \wedge w_0^* = w_1^* \end{array} \right] \leq \text{negl}(\lambda).$$

Remark 4.2 (On the need for an algorithm Gen). Standard constructions for NIZKs (that do not consider uniqueness of proofs) traditionally consist of a trusted Setup algorithm which produces a crs used to generate and verify proofs, which is crucial to achieve both zero-knowledge and soundness (and completeness). In contrast, in this work, we aim to study and build NIZKs with a uniqueness property. As we show in Section 9, this requires to provide the prover with a secret state needed to generate proofs.²³ From this perspective, considering a separate algorithm $\text{Gen} \rightarrow (\text{pk}, \text{sk})$ allows to consider strong forms of security: while we cannot guarantee Gen itself to have some uniqueness property,²⁴ we show that (1) both soundness and uniqueness hold for proofs generated using pk , *even if pk is maliciously generated* and (2) such a state can be safely reused by the prover (in terms of zero-knowledge). In other words, the notion we obtain is a “two-phase” proof system, where in the first (reusable) offline phase, the prover generates a potentially malicious key, which does not have any uniqueness property, and then, in an online phase, the prover generate proofs for many statements of his choice, with guarantees of soundness and uniqueness even if the offline phase was performed maliciously.

²³On a high level, without such secret state, the uniqueness of the proofs would either contradict zero-knowledge, or be sufficiently strong to imply witness encryption.

²⁴Otherwise, this would essentially imply again a unique NIZK without a prover secret state, as provers could then generate and append fresh public keys to every proof.

Remark 4.3 (On Multi-Theorem Zero-Knowledge). In our definition of multi-theorem zero-knowledge, we only allow the verifier to see a single proof of any given statement C . We first note that this is inherent due to uniqueness. Indeed, if a verifier receives two proofs generated using (C, w) for *the same* valid witness w , then it should receive the same proofs (if the underlying language associated to C is hard enough). However, zero-knowledge would then imply that if it instead received two proofs generated using (C, w) , (C, w') for two distinct valid witnesses $w \neq w'$, then the proofs would also have to be the same. This is because in the ideal world, the simulator Sim_2 does *not* see whether the two proofs were generated using the same witnesses or different ones. As a result, a UNIZK satisfying zero-knowledge without any restriction on queries would have uniqueness of proofs *across several witnesses* (meaning that any efficiently valid proofs for C has to be identical independent of the witness), which would imply witness encryption (see Section 9 for more details).

On the other hand, we observe that this restriction is quite natural for us, in that the typical use we envision for UNIZKs is over languages with computationally unique witnesses, so that verifiers never see proofs corresponding to different witnesses (and thus, by uniqueness, verifiers only ever see a single proof corresponding to any given statement).

Remark 4.4 (Strong Uniqueness and Multi-Theorem Zero-Knowledge from Stateful Provers.). In contrast to the remark above, we observe that we can actually obtain strong multi-theorem zero-knowledge where verifiers are allowed to see proofs on many pairs C, w , and strong uniqueness where all accepting proofs for C are statistically unique (regardless of the witness used), if the prover is allowed to be *stateful*. This is because the prover can then remember proofs it has issued: on input (C, w) , it checks whether it has previously issued a proof corresponding to C , and if so issues the same proof. It otherwise generates a fresh proof π for (C, w) , and stores in its state that it has produced a proof π for the statement C .

Remark 4.5 (Computational Soundness and Uniqueness). While we define in Definition 4.1 knowledge soundness and uniqueness (with the latter ensuring uniqueness per valid witness for the statement) to hold against arbitrarily powerful cheating provers, one could also meaningfully define UNIZKs to only guarantee these properties only against efficient cheating provers. Looking ahead, we will achieve the stronger versions of statistical knowledge soundness and uniqueness in our main theorem, and thus by default consider these notions throughout the paper.

4.1 Relaxations of UNIZKs and Additional Properties

As stepping stones towards realizing UNIZKs, we consider the following relaxations of the properties defined above, along with a useful efficiency property that we will leverage in our constructions.

Definition 4.6 (Relaxed Properties of UNIZKs). *For a tuple of five PPT algorithms (Setup, Gen, Prove, Verify, Ext) with the same syntax as a UNIZK Definition 4.1, we consider, along with perfect completeness, the following variants of soundness, zero-knowledge, and uniqueness respectively.*

Adaptive, Statistical Knowledge Soundness with Semi-Malicious Keys. *For all polynomials $n = n(\lambda)$ and $\ell = \ell(\lambda)$, for every cheating prover $\mathcal{P}^*$, we have:*

$$\Pr \left[\begin{array}{l} (\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\rho, C^*, \pi) \leftarrow \mathcal{P}^*(\text{crs}), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho), \\ w^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1 \\ \wedge C^*(w^*) = 0 \end{array} \right] \leq \text{negl}(\lambda),$$

where we require that $|C^*| = n$ and $|w^*| = \ell$.

Adaptive, Single-Theorem Zero-Knowledge with Reusable CRS. *For all polynomials $n = n(\lambda)$ and $\ell = \ell(\lambda)$, there exists a two-stage PPT simulator $(\text{Sim}_1, \text{Sim}_2)$ such that the following properties hold.*

- CRS indistinguishability: The following two distributions are computationally indistinguishable:

$$\text{crs} \approx_c \widetilde{\text{crs}},$$

where $(\text{crs}, \text{td}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda)$ and $(\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda)$.

- Single-Theorem Zero-Knowledge given Simulation Trapdoor: For all PPT adversaries $\mathcal{A}$, the two following distributions are computationally indistinguishable:

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\widetilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Prove}}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}), \end{array} : b \right\}$$

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\widetilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sim}_2}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}), \end{array} : b \right\},$$

where $\mathcal{O}_{\text{Prove}}$ and $\mathcal{O}_{\text{Sim}_2}$ are defined similarly as in Definition 4.1, with the restriction that $\mathcal{A}$ is only allowed to make a single oracle query.

Adaptive, Statistical Uniqueness with Semi-Malicious Keys. For all polynomials $n = n(\lambda)$ and $\ell = \ell(\lambda)$, and for every cheating prover $\mathcal{P}^*$, we have:

$$\Pr \left[\begin{array}{l} (\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\rho, C^*, \pi_0, \pi_1) \leftarrow \mathcal{P}^*(\text{crs}), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho), \\ w_0^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi_0), \\ w_1^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi_1) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi_0) = 1 \\ \wedge \text{Verify}(\text{crs}, \text{pk}, C^*, \pi_1) = 1 \\ \wedge \pi_0 \neq \pi_1 \\ \wedge w_0^* = w_1^* \end{array} \right] \leq \text{negl}(\lambda).$$

Next, we consider a useful efficiency property of (relaxed) UNIZKs.

Bootstrappability. We consider here a slightly different syntax, where

- $\text{Setup}(1^h, 1^\ell, 1^\lambda)$ takes instead as input a depth bound h and an input size bound ℓ and outputs $(\text{crs}, \text{td}_{\text{Ext}})$.

With this syntax, all the other properties are modified so that the restrictions on the statements C become that the depth of C is at most h (instead of their size being n) and their input length at most ℓ .

Let $C_{\text{Gen}, \text{crs}, \text{pk}}$ denote the circuit that has crs and pk hard-coded, takes as input ρ and outputs 1 if and only if $\text{Gen}(\text{crs}; \rho) = (\text{pk}, \text{sk})$ for some sk . We say that a UNIZK is bootstrappable if there exist large enough polynomials $h = h(\lambda)$ and $\ell = \ell(\lambda)$ such that the depth of $C_{\text{Gen}, \text{crs}, \text{pk}}$ is at most h and $|\rho| \leq \ell$.

Definition 4.7 (NIZK). A non-interactive proof system $(\text{Setup}, \text{Prove}, \text{Verify})$ is a UNIZK such that Gen outputs an empty string, and without the requirement of (adaptive) uniqueness.

5 Single Theorem UNIZK

In this section, we build a single-theorem version of UNIZKs with semi-malicious soundness and uniqueness.

Definition 5.1. A 1-time UNIZK is a relaxed UNIZK (Definition 4.6) satisfying completeness, adaptive knowledge soundness with semi-malicious keys, single-theorem zero-knowledge, and adaptive uniqueness with semi-malicious keys.

Theorem 5.2. *Assuming the existence of a perfectly correct, statistically extractable/computationally equivocal commitment scheme with statistically unique openings and a perfectly correct garbled circuit, there exists a 1-time UNIZK.*

Remark 5.3 (Computational uniqueness). Theorem 5.2 refers to our default definition of UNIZK (Definition 4.6), where uniqueness is statistical. Starting with a commitment scheme with computationally unique openings instead, we obtain an analogue of Theorem 5.2, where uniqueness only holds against efficient adversaries. The construction and proof are identical.

Construction. We use the following building blocks:

- A perfectly correct garbled circuit (Garble, Eval);
- A perfectly correct, statistically extractable/computationally equivocal commitment scheme with statistically unique openings (Commit.Gen, Commit, Commit.Verify, Commit.Ext) (Definition 3.7).

We define our 1-time UNIZK as follows:

- Setup($1^n, 1^\ell, 1^\lambda$): Compute $(ck, td_{Ext}) \leftarrow \text{Commit.Gen}(1^\lambda)$. Set $crs = ck$ and output (crs, td_{Ext}) .
- Gen(crs): Parse $crs = ck$. Let $U = U(C, w)$ to denote the universal circuit that which takes as input a circuit C of size n and a string w of size ℓ , and outputs $C(w)$.
 - Compute $(\tilde{U}, \{lab_{i,b}\}_{i \in [n+\ell], b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, U; r_{gb})$.
 - Commit to the garbled circuit labels $(com_{lab_{i,b}}, decom_{lab_{i,b}}) \leftarrow \text{Commit}(ck, \{lab_{i,b}\})$ for $i \leq n + \ell, b \in \{0, 1\}$.
 - Commit to the garbling randomness $(com_{r_{gb}}, decom_{r_{gb}}) \leftarrow \text{Commit}(ck, r_{gb})$.
 - Commit to the garbled circuit $(com_{GC}, decom_{GC}) \leftarrow \text{Commit}(ck, \tilde{U})$.
 - Sample a random string $\phi \leftarrow \{0, 1\}^\ell$
 - Permute the commitments to the labels corresponding to the w , namely for every $j \leq \ell$, set:

$$\overline{com}_{lab_{(n+j), \phi_j}} = com_{lab_{(n+j), 0}}$$

$$\overline{com}_{lab_{(n+j), 1-\phi_j}} = com_{lab_{(n+j), 1}}.$$

In words, this puts $(i, 0)$ label (corresponding to if $w_i = 0$) in position (i, ϕ) and the $(i, 1)$ label (corresponding to if $w_i = 1$) in position $1 - \phi$.

- Set $pk = (com_{GC}, com_{r_{gb}}, \{com_{lab_{i,b}}\}_{i \leq n, b}, \{\overline{com}_{lab_{n+i,b}}\}_{i \leq \ell, b})$ and $sk = (\{lab_{i,b}, decom_{lab_{i,b}}\}_{i,b}, \phi, \tilde{U}, decom_{GC})$ and output (pk, sk) .
- Prove(crs, sk, C, w). Parse $sk = (\{lab_{i,b}, decom_{lab_{i,b}}\}_{i,b}, \phi, \tilde{U}, decom_{GC})$. Output

$$\pi = \{\tilde{U}, decom_{GC}, (lab_{1,C_1}, decom_{lab_{1,C_1}}), \dots, (lab_{n,C_n}, decom_{lab_{n,C_n}}), (lab_{n+1,w_1}, decom_{lab_{n+1,w_1}}), \dots, (lab_{(n+\ell),w_\ell}, decom_{lab_{(n+\ell),w_\ell}})\}$$

where C_i is the i -th bit in the description of C and w_i is the i -th bit of w .

- Verify(crs, pk, C, π). Parses $pk = (com_{GC}, com_{r_{gb}}, \{com_{lab_{i,b}}\}_{i \leq n, b}, \{\overline{com}_{lab_{n+i,b}}\}_{i \leq \ell, b})$ and $\pi = (\tilde{U}, decom_{GC}, \{lab_i, decom_{lab_i}\}_i)$ then, outputs 1 if,

- $\text{Eval}(1^\lambda, \tilde{U}, \{\text{lab}_i\}_{i \in [n+\ell]}) = 1$,
 - and, $\text{Commit.Verify}(\text{ck}, \tilde{U}, \text{com}_{\text{GC}}, \text{decom}_{\text{GC}}) = 1$,
 - and, for all $i \in \{1, \dots, n\}$:

$$\text{Commit.Verify}(\text{ck}, \text{lab}_i, \text{com}_{\text{lab}_i, C_i}, \text{decom}_{\text{lab}_i}) = 1,$$
 - and, for all $i \in \{(n+1), \dots, (n+\ell)\}$,

$$\text{Commit.Verify}(\text{ck}, \text{lab}_i, \overline{\text{com}}_{\text{lab}_i, 0}, \text{decom}_{\text{lab}_i}) = 1$$
- or
- $$\text{Commit.Verify}(\text{ck}, \text{lab}_i, \overline{\text{com}}_{\text{lab}_i, 1}, \text{decom}_{\text{lab}_i}) = 1.$$

Otherwise output 0.

- $\text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi)$:
 - Parse $\text{pk} = (\text{com}_{\text{GC}}, \text{com}_{r_{\text{gb}}}, \{\text{com}_{\text{lab}_{i,b}}\}_{i \leq n, b}, \{\overline{\text{com}}_{\text{lab}_{n+i,b}}\}_{i \leq \ell, b})$ and $\pi = (\tilde{U}, \text{decom}_{\text{GC}}, \{\text{lab}_i, \text{decom}_{\text{lab}_i}\}_i)$.
 - Compute $r_{\text{gb}} \leftarrow \text{Commit.Ext}(\text{td}_{\text{Ext}}, \text{com}_{r_{\text{gb}}})$, where Commit.Ext is the extractor for the commitment scheme.
 - Compute $(\tilde{U}', \{\text{lab}'_{i,b}\}_{i \in [n+\ell], b \in \{0,1\}}) = \text{Garble}(1^\lambda, U; r_{\text{gb}})$
 - For each $i \in \{(n+1), \dots, (n+\ell)\}$ and $j \in [\ell]$
 - * If $\text{lab}'_{i,0} = \text{lab}_i$ then, set $w_j = 0$,
 - * Else if $\text{lab}'_{i,1} = \text{lab}_i$ then, set $w_j = 1$,
 - * Otherwise abort and output $\perp$.
 - Output $w = \{w_j\}_{j \in [\ell]}$.

Claim 5.4 (Completeness). *Assuming the perfect correctness of the garbling scheme ($\text{Garble}, \text{Eval}$) and the perfect correctness of the commitment scheme ($\text{Commit.Gen}, \text{Commit}, \text{Commit.Verify}, \text{Commit.Ext}$), the construction satisfies perfect completeness.*

Proof. Assume that $C(w) = 1$. Let $(\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho)$ where $\text{pk} = (\text{com}_{\text{GC}}, \text{com}_{r_{\text{gb}}}, \{\text{com}_{\text{lab}_{i,b}}\}_{i \leq n, b}, \{\overline{\text{com}}_{\text{lab}_{n+i,b}}\}_{i \leq \ell, b})$ and $\text{sk} = (\{\text{lab}_{i,b}, \text{decom}_{\text{lab}_{i,b}}\}_{i,b}, \phi, \tilde{U}, \text{decom}'_{\text{GC}})$, and $\pi \leftarrow \text{Prove}(\text{crs}, \text{pk}, \text{sk}, C, w)$ where $\pi = (\tilde{U}, \text{decom}'_{\text{GC}}, \{\text{lab}_i, \text{decom}_{\text{lab}_i}\}_i)$. Now we have $\text{Verify}(\text{crs}, \text{pk}, C, \pi) = 1$ when all of the following conditions are met:

- $\text{Eval}(1^\lambda, \tilde{U}, \{\text{lab}_i\}_{i \in [n+\ell]}) = 1$,
 - $\text{Commit.Verify}(\text{ck}, \tilde{U}, \text{com}_{\text{GC}}, \text{decom}'_{\text{GC}}) = 1$,
 - For all $i \in \{1, \dots, n\}$, $\text{Commit.Verify}(\text{ck}, \text{lab}_i, \text{com}_{\text{lab}_i, C_i}, \text{decom}_{\text{lab}_i}) = 1$
 - For all $i \in \{(n+1), \dots, (n+\ell)\}$,

$$\text{Commit.Verify}(\text{ck}, \text{lab}_i, \overline{\text{com}}_{\text{lab}_i, 0}, \text{decom}_{\text{lab}_i}) = 1$$
- or
- $$\text{Commit.Verify}(\text{ck}, \text{lab}_i, \overline{\text{com}}_{\text{lab}_i, 1}, \text{decom}_{\text{lab}_i}) = 1$$

By perfect correctness of ($\text{Garble}, \text{Eval}$), we have $\text{Eval}(1^\lambda, \tilde{U}, \{\text{lab}_i\}_{i \in [n+\ell]}) = 1$ as we assumed $C(w) = 1$. Similarly, by perfect correctness of the commitment scheme Commit , the commitment to the garbled circuit $\tilde{U}$ will verify, and all commitments for the labels of the statement and witness bits will also verify. Namely, we have $\text{Commit.Verify}(\text{ck}, \tilde{U}, \text{com}_{\text{GC}}, \text{decom}'_{\text{GC}}) = 1$, and, for all $i \in \{1, \dots, n\}$, $\text{Commit.Verify}(\text{ck}, \text{lab}_i, \text{com}_{\text{lab}_i, C_i}, \text{decom}_{\text{lab}_i}) = 1$.

$\text{decom}_{\text{lab}_i} = 1$ where lab_{i,C_i} is the label for i -th bit of the statement C , and, for $i \in \{(n+1), \dots, (n+\ell)\}$ we have

$$\text{Commit.Verify}(\text{ck}, \text{lab}_i, \overline{\text{com}}_{\text{lab}_i, w_i + \phi_i \bmod 2}, \text{decom}_{\text{lab}_i}) = 1,$$

where lab_{i,w_i} is the label for the i -th bit of the witness w . □

Claim 5.5 (Adaptive, Statistical Knowledge Soundness with Semi-Malicious Keys). *Assuming perfect correctness of the garbling scheme (Garble, Eval) and that the commitment scheme (Commit.Gen, Commit, Commit.Verify, Commit.Ext) is statistically extractable and perfectly correct, the construction satisfies adaptive knowledge soundness with semi-malicious keys.*

Proof. Let $\mathcal{P}^*$ be the cheating prover against soundness with semi-malicious keys and assume that

$$\Pr \left[\begin{array}{l} (\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\rho, C^*, \pi) \leftarrow \mathcal{P}^*(\text{crs}), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho), \\ w^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1 \\ \wedge C^*(w^*) = 0 \end{array} \right] \geq 1/p(\lambda),$$

where $p = p(\lambda)$ is a polynomial. Parse $\pi = \{\tilde{U}, \text{decom}'_{\text{GC}}, \text{lab}_{i,b_i}, \text{decom}_{\text{lab}_{i,b_i}}\}_{i \leq n+\ell}$. In the following, we condition on the event that $\text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1$.

Note that (pk, sk) being semi-maliciously generated implies that there exists randomness $r_{\text{gb}}, r_{\text{com},i,b}, r_{\text{com}}, r_{\text{GC}}$ such that:

$$\begin{aligned} (\tilde{U}', \{\text{lab}'_{i,b}\}_{i \in [n+\ell], b \in \{0,1\}}) &= \text{Garble}(1^\lambda, U; r_{\text{gb}}) \\ (\text{com}_{\text{lab}_{i,b}}, \text{decom}_{\text{lab}_{i,b}}) &= \text{Commit}(\text{ck}, \text{lab}'_{i,b}; r_{\text{com},i,b}) \text{ for } i \leq n+\ell, b \in \{0,1\} \\ (\text{com}_{r_{\text{gb}}}, \text{decom}_{r_{\text{gb}}}) &= \text{Commit}(\text{ck}, r_{\text{gb}}; r_{\text{com}}), \\ (\text{com}_{\text{GC}}, \text{decom}'_{\text{GC}}) &= \text{Commit}(\text{ck}, \tilde{U}'; r_{\text{GC}}). \end{aligned}$$

Let $i \leq n$. By perfect correctness of Commit, we have for both $b \in \{0,1\}$,

$\text{Commit.Verify}(\text{ck}, \text{lab}'_{i,b}, \text{com}_{\text{lab}_{i,b}}, \text{decom}_{\text{lab}_{i,b}}) = 1$. Thus, by statistical extractability of Commit, we have that $\text{lab}_{i,b_i} = \text{lab}'_{i,C_i}$ (except with negligible probability).

Let $i \in [n+1, n+\ell]$. Again, by perfect correctness and statistical extractability of Commit, a similar argument gives that either $\text{lab}_{i,b_i} = \text{lab}'_{i,0}$ or $\text{lab}_{i,b_i} = \text{lab}'_{i,1}$ (except with negligible probability). Moreover, (again by perfect correctness and extractability of Commit), we have that $\text{Commit.Ext}(\text{td}_{\text{Ext}}, \text{com}_{r_{\text{gb}}}) = r_{\text{gb}}$. Thus, computing $(\tilde{U}', \{\text{lab}'_{i,b}\}_{i \in [n+\ell], b \in \{0,1\}}) = \text{Garble}(1^\lambda, U; r_{\text{gb}})$, Ext correctly matches a bit β_i such that $\text{lab}_{i,b_i} = \text{lab}'_{i,\beta_i}$ (except with negligible probability).

By perfect correctness and statistical extractability of Commit (again, this time over the commitment com_{GC}), we have that $\tilde{U} = \tilde{U}'$.

We now argue that the witness w^* output by Ext satisfies $C(w^*) = 1$ except with negligible probability. This is because we have overall extracted labels $\{\text{lab}'_{i,\beta_i}\}_i$ (except with negligible probability) such that

- For $i \leq n$, $\beta_i = C_i$ corresponds to the i -th bit of C ,
- For $1 \leq i \leq \ell$, $\beta_{n+i} = w_i^*$ corresponds to the i -th bit of w^* by definition of w^* .

Thus, by perfect correctness of the garbled circuit, given that $\text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1$ implies $\text{Eval}(1^\lambda, \tilde{U}, \{\text{lab}'_{i,\beta_i}\}_{i \in [n+\ell]}) = 1$ (except with negligible probability), we conclude that $C(w^*) = 1$. □

Claim 5.6 (Adaptive, Statistical Uniqueness with Semi-Malicious Keys). *Suppose that the commitment scheme (Commit.Gen, Commit.Verify, Commit.Ext) is statistically extractable and has statistically unique openings, and that the garbling scheme (Garble, Eval) is perfectly correct. Then, the construction satisfies adaptive statistical uniqueness with semi-malicious keys.*

Proof. Let $\mathcal{P}^*$ be the cheating prover against adaptive uniqueness and assume that

$$\Pr \left[\begin{array}{l} (\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\rho, C^*, \pi_0, \pi_1) \leftarrow \mathcal{P}^*(\text{crs}), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho), \\ w_0^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi_0), \\ w_1^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi_1) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi_0) = 1 \\ \wedge \text{Verify}(\text{crs}, \text{pk}, C^*, \pi_1) = 1 \\ \wedge \pi^{(0)} \neq \pi^{(1)} \\ \wedge w_0^* = w_1^* \end{array} \right] \geq 1/p(\lambda),$$

where $p = p(\lambda)$ is a polynomial. Parse for $\beta \in \{0, 1\}$, $\pi^{(\beta)} = ((\tilde{U}^{(\beta)}, \text{decom}_U^{(\beta)}), \{\text{lab}_{i,b_i}^{(\beta)}, \text{decom}_{\text{lab}_{i,b_i}}^{(\beta)}\}_{i \leq n+\ell})$, and condition on the event above holding.

Suppose $i \leq n$. For $\beta \in \{0, 1\}$, we have that (note that the commitment does not depend on β):

$$\text{Commit.Verify}(\text{ck}, \text{lab}_{i,b_i}^{(\beta)}, \text{com}_{\text{lab}_{i,b_i}}, \text{decom}_{\text{lab}_{i,b_i}}^{(\beta)}) = 1.$$

By statistical extractability of com , we have $\text{lab}_{i,b_i}^{(0)} = \text{lab}_{i,b_i}^{(1)}$ (except with negligible probability). But then, by statistical uniqueness of openings of the commitment, we have $\text{decom}_{\text{lab}_{i,b_i}}^{(0)} = \text{decom}_{\text{lab}_{i,b_i}}^{(1)}$ (except with negligible probability).

Similarly, for $\beta \in \{0, 1\}$, we have that $\text{Commit.Verify}(\text{ck}, \tilde{U}^{(\beta)}, \text{com}_{\text{GC}}, \text{decom}_U^{(\beta)}) = 1$, and the same argument gives that $\tilde{U}^{(0)} = \tilde{U}^{(1)}$ and $\text{decom}_U^{(0)} = \text{decom}_U^{(1)}$.

Suppose now that $n+1 \leq i \leq n+\ell$. Similarly as before, if both $\text{lab}_{i,b_i}^{(\beta)}, \text{decom}_{\text{lab}_{i,b_i}}^{(\beta)}$ open to the same commitment $\overline{\text{com}}_{\text{lab}_{i,b_i}}$, then $\text{lab}_{i,b_i}^{(0)} = \text{lab}_{i,b_i}^{(1)}$ and $\text{decom}_{\text{lab}_{i,b_i}}^{(0)} = \text{decom}_{\text{lab}_{i,b_i}}^{(1)}$ except with negligible probability.

Thus, $\pi^{(0)} \neq \pi^{(1)}$ implies (by the argument above) that for some index i where $n+1 \leq i \leq n+\ell$, the proofs provide openings to different commitments $\overline{\text{com}}_{\text{lab}_{i,b_i}}$, $b = 0, 1$ (except with negligible probability). We argue that in this case, Ext extracts a different bit $w_i^{(0)} \neq w_i^{(1)}$ except with negligible probability, and thus that $w_0^* \neq w_1^*$. This is because, if they open to different commitments, by extractability we have that one, say $\pi^{(\beta)}$, opens to $\text{lab}_{i,0}$ and the other $\pi^{(1-\beta)}$ to $\text{lab}_{i,1}$ (except with negligible probability). By perfect correctness of the garbled circuit, where we use that public keys and thus garbled circuits are semi-maliciously generated, and extractability of Commit , we have that $\text{lab}_{i,0} \neq \text{lab}_{i,1}$ except with negligible probability — this is because this would otherwise contradict garbled circuit correctness on any $C, w^{(0)}, w^{(1)}$ such that $w^{(0)}, w^{(1)}$ only differ on their i -th bit, but $C(w^{(0)}) \neq C(w^{(1)})$. But then, this means Ext will set the i -th bits of the witnesses as $w_{\beta,i}^* = 0$ and $w_{\beta,i}^* = 1$, so that $w_0^* \neq w_1^*$. This contradicts our initial assumption on $\mathcal{P}^*$. $\square$

Claim 5.7 (Adaptive, Single-Theorem Zero-Knowledge). *Assuming the security of the garbling scheme (Garble, Eval) and that the commitment scheme Commit is equivocal, hiding, and satisfies indistinguishability of modes, the construction satisfies adaptive, single-theorem zero-knowledge.*

Proof. Let $\text{Gen}_{\text{Equiv}}, \text{Equiv}$ be the equivocability simulators for the commitment scheme and Sim_{GC} be the simulator for the garbled circuit. Then we define the simulator algorithms $(\text{Sim}_1, \text{Sim}_2)$ below.

- $\text{Sim}_1(1^n, 1^\ell, 1^\lambda)$:
 - Compute $(\tilde{\text{ck}}, \text{td}_{\text{Equiv}}) \leftarrow \text{Gen}_{\text{Equiv}}(1^\lambda)$ and output $(\tilde{\text{crs}}, \text{td}_{\text{Sim}})$ where $\tilde{\text{crs}} = \tilde{\text{ck}}$ and $\text{td}_{\text{Sim}} = \text{td}_{\text{Equiv}}$.
- $\text{Sim}_2(\text{td}_{\text{Sim}}, \text{pk}, C)$:
 - Parses $\text{td}_{\text{Sim}} = \text{td}_{\text{Equiv}}$ and $\text{pk} = (\text{com}_{\text{GC}}, \text{com}_{r_{\text{gb}}}, \{\overline{\text{com}}_{\text{lab}_{i,b}}\})$
 - Compute $(\tilde{U}, \{\text{lab}'_{i,b}\}) \leftarrow \text{Sim}_{\text{GC}}(1^\lambda, 1^{|U|}, U)$.
 - Compute $\text{decom}'_{\text{GC}} \leftarrow \text{Equiv}(\text{td}_{\text{Equiv}}, \text{com}_{\text{GC}}, \text{decom}_{\text{GC}}, \tilde{U})$.

- For each $i \in [n + \ell], b \in \{0, 1\}$, sample a random $\phi_i \leftarrow \{0, 1\}$ and compute:

$$\text{decom}'_{\text{lab}'_{i,b}} \leftarrow \text{Equiv}(\text{td}_{\text{Equiv}}, \text{com}_{\text{lab}_{i,\phi_i}}, \text{decom}_{\text{lab}_{i,b}}, \text{lab}'_{i,b}).$$

- Sample uniformly random string $w \leftarrow \{0, 1\}^\ell$
- Output $\pi_{\text{Sim}} = \left(\tilde{U}, \text{decom}'_{\text{GC}}, \{\text{lab}'_{i,C_i}, \text{decom}'_{\text{lab}'_{1,C_i}}\}_{i \leq n}, \{\text{lab}'_{n+i,w_i}, \text{decom}'_{\text{lab}'_{n+i,w_i}}\}_{i \leq \ell} \right).$

We first argue CRS indistinguishability: Let $(\text{crs}, \text{td}) \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda)$ where $\text{crs} = \text{ck} \leftarrow \text{Commit.Gen}(1^\lambda)$, and $(\tilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda)$ where $\tilde{\text{crs}} = \tilde{\text{ck}} \leftarrow \text{Gen}_{\text{Equiv}}(1^\lambda)$. Then, $\text{crs} = \text{ck}$ is computationally indistinguishable from $\tilde{\text{crs}} = \tilde{\text{ck}}$ by indistinguishability of modes of the commitment scheme.

Next we argue single-theorem zero-knowledge given the simulation trapdoor. Let $\mathcal{A}$ be a PPT adversary. We show that the two following two distributions are computational indistinguishable:

$$\left\{ \begin{array}{l} (\tilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\tilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Prove}}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}), \end{array} : b \right\}$$

$$\left\{ \begin{array}{l} (\tilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\tilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sim}_2}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}), \end{array} : b \right\},$$

To do so, we argue by a hybrid argument that the experiments induced by $\mathcal{O}_{\text{Prove}}(\cdot)$ and $\mathcal{O}_{\text{Sim}_2}(\cdot)$ are indistinguishable.

Hybrid $\mathcal{H}_0$. This is the distribution induced by $\mathcal{O}_{\text{Prove}}(\cdot)$ where queries for (C, w) are answered by $\text{Prove}(\text{crs}, \text{sk}, C, w)$.

Hybrid $\mathcal{H}_1$. In this hybrid, we change how $\text{decom}'_{\text{GC}}$ and $\text{decom}_{\text{lab}_{i,b}}$ provided in the proof are computed. We now instead compute them as $\text{Equiv}(\text{td}_{\text{Equiv}}, \text{com}_{\text{GC}}, \text{decom}_{\text{GC}}, \tilde{U})$ and $\text{decom}'_{\text{lab}'_{i,b}} \leftarrow \text{Equiv}(\text{td}_{\text{Equiv}}, \text{com}_{\text{lab}_{i,b}}, \text{decom}_{\text{lab}_{i,b}}, \text{lab}'_{i,b})$.

By equivocability of the commitment *given the equivocation trapdoor*, we obtain that the distributions induced in $\mathcal{H}_0$ and $\mathcal{H}_1$ are indistinguishable.

Hybrid $\mathcal{H}_2$. We change how $\tilde{U}, \{\text{lab}_{i,b}\}$ provided in the proof are computed. We now generate them as $\tilde{U}, \{\text{lab}_{i,b}\} \leftarrow \text{Sim}_{\text{GC}}(1^\lambda, |U|, 1)$.

Security of the garbled circuit gives that the distributions induced in $\mathcal{H}_1$ and $\mathcal{H}_2$ are indistinguishable. More precisely, we crucially use here that the input C to the garbled circuit is chosen before the garbled circuit is generated and w is implicitly generated as a random string.

Notice that this distribution induced by this hybrid is identical to that induced by $\mathcal{O}_{\text{Sim}_2}(\cdot)$ since, now the modified Prove behaves exactly as Sim_2 — this is because on labels corresponding to w , the adversary only sees the opening of a randomly chosen commitment (namely over the choice of ϕ_i). This concludes the proof. $\square$

6 Adding Bootstrappability

In this section, we show how to additionally confer 1-time UNIZKs with bootstrappability.

Theorem 6.1. *Assuming the existence of a 1-time UNIZK (Definition 5.1), statistically extractable commitment scheme with statistically unique openings, and leveled fully homomorphic encryption, there exists a 1-time UNIZK that is bootstrappable (Definition 4.1).*

Remark 6.2 (Computational uniqueness). Similarly to Remark 5.3, if we start with a computationally unique 1-time UNIZK along with a commitment scheme with computationally unique openings instead, Theorem 6.1 yields instead a computationally unique 1-time UNIZK that is bootstrappable. The construction and proof are identical.

Construction. We use the following building blocks:

- A 1-time UNIZK $1T.UNIZK = (1T.Setup, 1T.Gen, 1T.Prove, 1T.Verify, 1T.Ext)$ (Definition 5.1);
- A (leveled) perfectly correct fully homomorphic encryption scheme $FHE = (FHE.Gen, FHE.Enc, FHE.Dec)$ satisfying the efficiency conditions in Claim 3.17,
- A statistically extractable commitment scheme $(Commit.Gen, Commit, Commit.Verify, Commit.Ext)$ with statistically unique opening.
- A PRG $PRG_{Enc} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{r(\lambda)}$, where r is a large enough polynomial to be set later.
- A PRG $PRG_{Gen} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{t(\lambda)}$, where t is a large enough polynomial to be set later.
- A PRG $PRG_{1T} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{u(\lambda)}$, where u is a large enough polynomial to be set later.

We use the following parameters. Given a polynomial $\ell = \ell(\lambda)$, We set $n_{1T} = \text{poly}(\ell, \lambda)$ and $\ell_{1T} = \ell + \text{poly}(\lambda)$ for some sufficiently large poly to be determined later.

We define our new 1-time UNIZK with bootstrappability as follows. Note that $Bootstrap.Setup$ here takes as input a depth bound h , where the syntax is introduced in the definition of bootstrappability in Definition 4.6.

- $Bootstrap.Setup(1^h, 1^\ell, 1^\lambda)$: Call $(1T.crs, 1T.td_{Ext}) \leftarrow 1T.Setup(1^{n_{1T}}, 1^{\ell_{1T}}, 1^\lambda)$ and $(ck, Commit.td_{Ext}) \leftarrow Commit.Setup(1^\lambda)$ sets $crs = (1T.crs, ck)$, $td_{Ext} = 1T.td_{Ext}$ and outputs (crs, td_{Ext}) .
- $Bootstrap.Gen(crs; \rho)$: Parse $crs = (1T.crs, ck)$ and does the following,
 - Parse $\rho = \rho_{FHE} \parallel \rho_{Enc} \parallel \rho_{Commit,1} \parallel \rho_{Commit,2} \parallel \rho_{1T}$.
 - Expand the randomness: compute $r_{FHE} = PRG_{Gen}(\rho_{FHE}) \in \{0, 1\}^r$, $r_{1T} = PRG_{1T}(\rho_{1T})$.
 - Compute $(FHE.pk, FHE.sk, FHE.ek) \leftarrow FHE.Gen(1^\lambda, 1^{h_{1T}}; r_{FHE})$.
 - Compute the commitments
 $(com_{\rho_{Enc}}, decom_{\rho_{Enc}}) \leftarrow Commit(ck, \rho_{Enc}; \rho_{Commit,1})$ and
 $(com_{FHE.sk}, decom_{FHE.sk}) \leftarrow Commit(ck, FHE.sk; \rho_{Commit,2})$.
 - Compute $(1T.pk, 1T.sk) \leftarrow 1T.Gen(1T.crs; r_{1T})$.
 - Output:
$$pk = (1T.pk, FHE.pk, FHE.ek, com_{\rho_{Enc}}, com_{FHE.sk})$$

$$sk = (pk, 1T.sk, \rho_{Enc}, decom_{\rho_{Enc}}, FHE.sk, decom_{FHE.sk}).$$
- $Bootstrap.Prove(crs, sk, C, w)$: Parse $crs = (1T.crs, ck)$, $sk = (1T.sk, \rho_{Enc}, decom_{\rho_{Enc}}, FHE.sk, decom_{FHE.sk})$, and $pk = (1T.pk, FHE.pk, FHE.ek, com_{\rho_{Enc}}, com_{FHE.sk})$, and perform the following:
 - Compute $r_{Enc} = PRG_{Enc}(\rho_{Enc}) \in \{0, 1\}^r$.
 - Encrypt the witness $ct_w \leftarrow FHE.Enc_{FHE.pk}(w; r_{Enc})$ and homomorphically evaluate C on it: $ct_{C(w)} = FHE.Eval_{FHE.ek}(C, ct_w)$.

- Consider the circuit C' defined in Fig. 1. Let $w' = (\rho_{\text{Enc}}, \text{decom}_{\rho_{\text{Enc}}}, \text{FHE.sk}, \text{decom}_{\text{FHE.sk}}, w)$. Compute $1\text{T}.\pi = 1\text{T}.\text{Prove}(1\text{T}.\text{crs}, 1\text{T}.\text{sk}, C', w')$, and output

$$\pi = (\text{ct}_w, 1\text{T}.\pi).$$

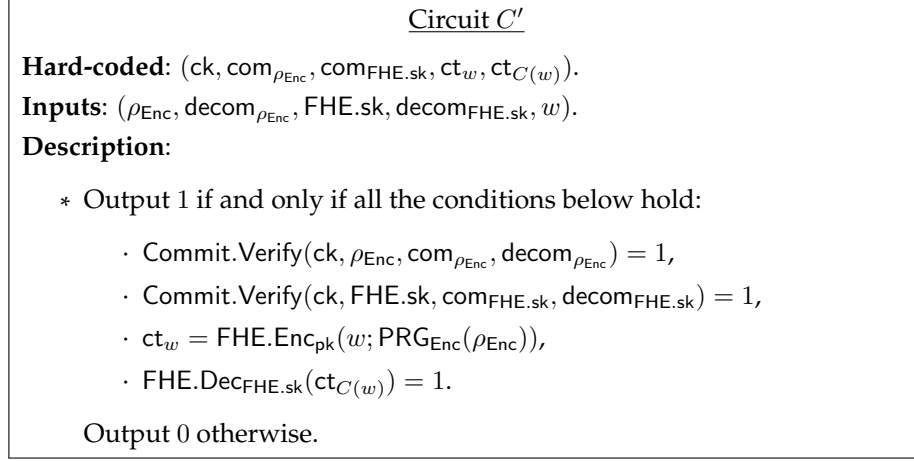


Figure 1: Definition of the circuit C' .

- $\text{Bootstrap}.\text{Verify}(\text{crs}, \text{pk}, C, \pi)$: Parse $\text{crs} = (1\text{T}.\text{crs}, \text{ck})$, $\text{pk} = (1\text{T}.\text{pk}, \text{FHE.pk}, \text{FHE.ek}, \text{com}_{\rho_{\text{Enc}}}, \text{com}_{\text{FHE.sk}})$ and $\pi = (\text{ct}_w, 1\text{T}.\pi)$. Compute $\text{ct}_{C'(w)} = \text{FHE}.\text{Eval}_{\text{FHE.ek}}(C, \text{ct}_w)$. Consider the circuit C' defined in Fig. 1. Output:

$$b = 1\text{T}.\text{Verify}(1\text{T}.\text{crs}, 1\text{T}.\text{pk}, C', 1\text{T}.\pi).$$

- $\text{Bootstrap}.\text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi)$: Parse $\text{td}_{\text{Ext}} = 1\text{T}.\text{td}_{\text{Ext}}$, $\text{pk} = (1\text{T}.\text{pk}, \text{FHE.pk}, \text{FHE.ek}, \text{com}_{\rho_{\text{Enc}}}, \text{com}_{\text{FHE.sk}})$ and $\pi = (\text{ct}_w, 1\text{T}.\pi)$. Compute $(\rho_{\text{Enc}}, \text{decom}_{\rho_{\text{Enc}}}, \text{FHE.sk}, \text{decom}_{\text{FHE.sk}}, w) = 1\text{T}.\text{Ext}(1\text{T}.\text{td}_{\text{Ext}}, 1\text{T}.\text{pk}, 1\text{T}.\pi)$, and output w .

Parameters. Recall that we denote by r the output length of PRG_{FHE} . Given polynomials $h = h(\lambda)$, $\ell = \ell(\lambda)$, we set $r = r(\lambda, \ell) = \text{poly}(\lambda, \ell)$ such that the total amount of randomness used to run $\text{FHE}.\text{Enc}$ over a message of size ℓ is at most r . Note that such PRG can be computed with a circuit of size $\text{poly}(\lambda, \ell)$.

Similarly, denoting t the output length of PRG_{Gen} , we set t such that the total amount of randomness to run an FHE setup that supports computations of depth h is at most t , so that $t = \text{poly}(h, \lambda)$. Note that such a PRG_{Gen} can be computed with a circuit of *depth* $\text{poly}(\lambda, \log h) = \text{poly}(\lambda)$.

Let n, ℓ be polynomials, and suppose C has size n and input size ℓ . Note that by construction, $|\rho_{\text{Enc}}| = \lambda$. Note that the size of commitments and openings of messages of size $\text{poly}(\lambda)$ is $\text{poly}(\lambda)$, and verification time for such commitments is also $\text{poly}(\lambda)$ for a fixed polynomial. Furthermore, assuming FHE satisfies the efficiency properties of Claim 3.17, then both FHE.sk and the time to decrypt ciphertexts that encrypt 1 bit is at most $\text{poly}(\lambda)$. Thus, the size of the circuit C' is at most $n_{1\text{T}} = \text{poly}(\ell, \lambda)$ (due to the complexity of evaluating PRG_{Enc}), and has input size $\ell_{1\text{T}} = \ell + \text{poly}(\lambda)$. Furthermore, the *depth* of C' is $\text{poly}(\lambda)$, given the instantiation of PRG_{Gen} above.

Denoting u the output length of $\text{PRG}_{1\text{T}}$, we set u so that the total amount of randomness to compute $1\text{T}.\text{Gen}(1^{n_{1\text{T}}}, 1^{\ell_{1\text{T}}}, 1^\lambda)$ is at most u (where $n_{1\text{T}} = \text{poly}(\ell, \lambda)$, $\ell_{1\text{T}} = \ell + \text{poly}(\lambda)$). Note that such a $\text{PRG}_{1\text{T}}$ can be computed with a circuit of *depth* $\text{poly}(\lambda, \log \ell) = \text{poly}(\lambda)$.

Claim 6.3 (Completeness). *Assuming the completeness of the 1-time UNIZK $1\text{T}.\text{UNIZK}$ (Definition 4.6), (leveled) fully homomorphic encryption scheme FHE is perfectly correct and satisfies the efficiency properties of Claim 3.17, and that the commitment scheme has perfect correctness. Then, the construction above satisfies perfect completeness.*

Proof. By the previous observations, C' has size at most $n_{1T} = \ell \cdot \text{poly}(\lambda)$ and input size $\ell_{1T} = \ell + \text{poly}(\lambda)$, for fixed polynomials independent of n .

Assume that $C(w) = 1$, and assume that C has depth at most h and input length at most ℓ . Let $\text{crs} \leftarrow \text{Bootstrap.Setup}(1^h, 1^\ell, 1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{Bootstrap.Gen}(\text{crs}; \rho)$ where $\text{pk} = (1T.\text{pk}, \text{FHE}.\text{pk}, \text{FHE}.\text{ek}, \text{com}_{\rho_{\text{Enc}}}, \text{com}_{\text{FHE}.\text{sk}})$, $\text{sk} = (1T.\text{sk}, \rho_{\text{Enc}}, \text{decom}_{\rho_{\text{Enc}}}, \text{FHE}.\text{sk}, \text{decom}_{\text{FHE}.\text{sk}})$, and $\pi \leftarrow \text{Bootstrap.Prove}(\text{crs}, \text{pk}, \text{sk}, C, w)$ where $\pi = (\text{ct}_w, 1T.\pi)$. Then by correctness of the commitment, we have $\text{Commit.Verify}(\text{ck}, \rho_{\text{Enc}}, \text{com}_{\rho_{\text{Enc}}}, \text{decom}_{\rho_{\text{Enc}}}) = 1$, and similarly $\text{Commit.Verify}(\text{ck}, \text{FHE}.\text{sk}, \text{com}_{\text{FHE}.\text{sk}}, \text{decom}_{\text{FHE}.\text{sk}}) = 1$. Furthermore, by perfect correctness of FHE, the assumption that $C(w) = 1$ and, as h is larger than the depth of C , we have $\text{FHE}.\text{Dec}_{\text{FHE}.\text{sk}}(\text{ct}_{C(w)}) = 1$ (even though r_{Enc} and r_{FHE} are only pseudorandom). Thus, letting $w' := (\rho_{\text{Enc}}, \text{decom}_{\rho_{\text{Enc}}}, \text{FHE}.\text{sk}, \text{decom}_{\text{FHE}.\text{sk}}, w)$, we have $C'(w') = 1$, and so by perfect correctness of $1T.\text{UNIZK}$ (to handle that r_{1T} is only pseudorandom), and using that C' has size at most n_{1T} and input size at most ℓ_{1T} , we obtain $1T.\text{Verify}(1T.\text{crs}, 1T.\text{pk}, C', 1T.\pi) = 1$, which concludes the proof. $\square$

Claim 6.4 (Adaptive, Statistical Knowledge Soundness with Semi-Malicious Keys). *Assuming that $1T.\text{UNIZK}$ satisfies adaptive statistical knowledge soundness with semi-malicious keys and that FHE is perfectly correct and satisfies the efficiency constraints of Claim 3.17, and the commitment is statistically binding, the construction satisfies adaptive knowledge soundness with semi-malicious keys.*

Proof. Let $\mathcal{P}^*$ be a cheating prover for the adaptive soundness with semi-malicious keys of the construction, namely given $(\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Bootstrap.Setup}(1^h, 1^\ell, 1^\lambda)$, $(\rho, C^*, \pi) \leftarrow \mathcal{P}^*(\text{crs})$, $(\text{pk}, \text{sk}) \leftarrow \text{Bootstrap.Gen}(\text{crs}; \rho)$, and $w^* \leftarrow \text{Bootstrap.Ext}(\text{pk}, \text{td}_{\text{Ext}}, \pi)$, we have

$$\Pr \left[\begin{array}{c} \text{Bootstrap.Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1 \\ \wedge \quad C^*(w^*) = 0 \end{array} \right] \geq 1/p(\lambda),$$

where $p = p(\lambda)$ is a polynomial. We argue that this contradicts adaptive statistical knowledge soundness with semi-malicious keys of $1T.\text{UNIZK}$.

The reduction runs $(\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Bootstrap.Setup}(1^h, 1^\ell, 1^\lambda)$, and receives $(\rho, C^*, \pi) \leftarrow \mathcal{P}^*(\text{crs})$. It runs $(\text{pk}, \text{sk}) \leftarrow \text{Bootstrap.Gen}(\text{crs}, \rho)$ and computes C' (as defined in Fig. 1, with underlying circuit C^*), and submits $(\text{PRG}_{1T}(\rho_{1T}), C', 1T.\pi)$ in the $1T.\text{UNIZK}$ knowledge soundness experiment (where $\text{PRG}_{1T}(\rho_{1T})$ is the randomness used in Bootstrap.Gen to generate $1T.\text{Gen}$, and is defined by ρ). Let

$$w' = (\rho_{\text{Enc}}, \text{decom}_{\rho_{\text{Enc}}}, \text{FHE}.\text{sk}, \text{decom}_{\text{FHE}.\text{sk}}, w^*) = 1T.\text{Ext}(1T.\text{td}_{\text{Ext}}, 1T.\text{pk}, 1T.\pi)$$

be the extracted witness from $1T.\pi$, where the last component of w' is w^* by definition of Bootstrap.Ext . By definition, $\text{Bootstrap.Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1$ implies that $1T.\text{Verify}(1T.\text{crs}, 1T.\text{pk}, C', 1T.\pi) = 1$. But by perfect correctness of the FHE (and the setting of h), and statistical extractability of the commitment (in fact, binding is enough, as to ensure that $\text{FHE}.\text{sk}$ that is decommitted to is indeed in the support of FHE.Gen), we have $C^*(w^*) = 0$ implies $C'(w') = 0$ (as $\text{FHE}.\text{Dec}_{\text{FHE}.\text{sk}}(\text{ct}_{C(w^*)})$ will then always be 0), and thus the reduction wins the adaptive statistical knowledge soundness with semi-malicious keys experiment of $1T.\text{UNIZK}$ with probability $1/p(\lambda) - \text{negl}(\lambda)$ (where we implicitly use that C' has size at most n_{1T} and input size at most ℓ_{1T}). $\square$

Claim 6.5 (Adaptive, Statistical Uniqueness with Semi-Malicious Keys). *Assuming that $1T.\text{UNIZK}$ satisfies adaptive statistical soundness and uniqueness with semi-malicious keys, and the commitment scheme Commit is binding and has statistically unique openings, the construction satisfies adaptive, statistical uniqueness against semi-malicious keys.*

Proof. Let $\mathcal{P}^*$ be an adversary against adaptive, statistical uniqueness with semi-malicious keys of the construction, namely given $(\text{crs}, \text{td}_{\text{Ext}}) \leftarrow \text{Bootstrap.Setup}(1^h, 1^\ell, 1^\lambda)$, $(\rho, C^*, \pi_0, \pi_1) \leftarrow \mathcal{P}^*(\text{crs})$, $(\text{pk}, \text{sk}) \leftarrow \text{Bootstrap.Gen}(\text{crs}; \rho)$, $w_0 \leftarrow \text{Bootstrap.Ext}(\text{pk}, \text{td}_{\text{Ext}}, \pi_0)$, and $w_1 \leftarrow \text{Bootstrap.Ext}(\text{pk}, \text{td}_{\text{Ext}}, \pi_1)$

$$\Pr \left[\begin{array}{l} \text{Bootstrap.Verify}(\text{crs}, \text{pk}, C^*, \pi_0) = 1 \\ \wedge \text{Bootstrap.Verify}(\text{crs}, \text{pk}, C^*, \pi_1) = 1, \\ \wedge \pi_0 \neq \pi_1, \\ \wedge w_0 = w_1, \end{array} \right] \geq 1/p(\lambda),$$

Parse $\pi_0 = (\text{ct}_{w_0}, 1\text{T}.\pi_0)$ and $\pi_1 = (\text{ct}_{w_1}, 1\text{T}.\pi_1)$. Let $1\text{T}.w_b = 1\text{T}.\text{Ext}(\text{td}_{\text{Ext}}, 1\text{T}.\text{pk}, \pi_b)$, and parse $1\text{T}.w_b = (\rho_{\text{Enc}}, \text{decom}_{\rho_{\text{Enc}}}, \text{FHE.sk}, \text{decom}_{\text{FHE.sk}}, w_b)$; in the following we condition on $\mathcal{P}^*$ succeeding.

By binding of Commit, we have ρ_{Enc} (resp. FHE.sk) is statistically uniquely defined by $\text{com}_{\rho_{\text{Enc}}}$ (resp. $\text{com}_{\text{FHE.sk}}$) included in the public key. As a result, if Commit additionally has statistically unique openings, then $\text{decom}_{\rho_{\text{Enc}}}$ (resp. $\text{decom}_{\text{FHE.sk}}$) is statistically uniquely defined by $\text{com}_{\rho_{\text{Enc}}}$ (resp. $\text{com}_{\text{FHE.sk}}$). Thus, we obtain by adaptive statistical soundness of $1\text{T}.\text{UNIZK}$ with semi-malicious keys that $w_0 = w_1$ in the experiment above implies that $1\text{T}.w_0 = 1\text{T}.w_1$ except with negligible probability. Then, relying again on adaptive statistical soundness of $1\text{T}.\text{UNIZK}$ with semi-malicious keys implies that, assuming $w_0 = w_1$ in the experiment above, we have $\text{ct}_{w_0} = \text{ct}_{w_1} = \text{FHE.Enc}(w_b; \rho_{\text{Enc}})$ except with negligible probability. We thus conclude that $\pi_0 \neq \pi_1$ in the experiment above implies that $1\text{T}.\pi_0 \neq 1\text{T}.\pi_1$ except with negligible probability.

We now argue that this contradicts adaptive uniqueness against semi-malicious keys of $1\text{T}.\text{UNIZK}$. Very similarly to the reduction in Claim 6.5, upon receiving $(\rho, C^*, \pi_0, \pi_1)$ from $\mathcal{P}^*$, the reduction runs $(\text{pk}, \text{sk}) \leftarrow \text{Bootstrap.Gen}(\text{crs}, \rho)$ and computes C' (as defined in Fig. 1, with underlying circuit C^*), and submits $(\text{PRG}(\rho_{1\text{T}}), C', 1\text{T}.\pi_0, 1\text{T}.\pi_1)$ in the $1\text{T}.\text{UNIZK}$ experiment (where $\rho_{1\text{T}}$ is included in ρ , and $1\text{T}.\pi_b$ is included in π_b). Let $1\text{T}.w_b = 1\text{T}.\text{Ext}(1\text{T}.\text{td}_{\text{Ext}}, 1\text{T}.\text{pk}, 1\text{T}.\pi_b)$ be the extracted witness from $1\text{T}.\pi_b$ where $b \in \{0, 1\}$. By definition, for both $b \in \{0, 1\}$, $\text{Bootstrap.Verify}(\text{crs}, \text{pk}, C^*, \pi_b) = 1$ implies that $1\text{T}.\text{Verify}(1\text{T}.\text{crs}, 1\text{T}.\text{pk}, C', 1\text{T}.\pi_b) = 1$. By the arguments above (where we implicitly use that C' has size at most $n_{1\text{T}}$ and input size at most $\ell_{1\text{T}}$), we have that $1\text{T}.\pi_0 \neq 1\text{T}.\pi_1$ and $1\text{T}.w_0 = 1\text{T}.w_1$ except with negligible probability, and thus this reduction contradicts adaptive, statistical uniqueness against semi-malicious keys of $1\text{T}.\text{UNIZK}$. $\square$

Claim 6.6 (Adaptive, Single-Theorem Zero-Knowledge). *Assuming that $1\text{T}.\text{UNIZK}$ satisfies adaptive, single-theorem zero-knowledge (Definition 4.6), that the (leveled) fully homomorphic encryption scheme FHE is secure, and that the commitment scheme is hiding, then the construction satisfies adaptive, single-theorem zero-knowledge.*

Proof. Let $1\text{T}.\text{Sim}_1, 1\text{T}.\text{Sim}_2$ be the simulators for the 1-time UNIZK. Then we define the simulator $(\text{Sim}_1, \text{Sim}_2)$: $\text{Sim}_1(1^h, 1^\ell, 1^\lambda)$:

- Call $(1\text{T}.\text{crs}, 1\text{T}.\text{td}_{\text{Sim}}) \leftarrow 1\text{T}.\text{Sim}_1(1^{n_{1\text{T}}}, 1^{\ell_{1\text{T}}}, 1^\lambda)$ and $(\text{ck}, \text{Commit}.\text{td}_{\text{Ext}}) \leftarrow \text{Commit.Gen}(1^\lambda)$.
- Output $(\widetilde{\text{crs}}, \text{td}_{\text{Sim}})$ where $\widetilde{\text{crs}} = (1\text{T}.\text{crs}, \text{ck})$ and $\text{td}_{\text{Sim}} = 1\text{T}.\text{td}_{\text{Sim}}$.

$\text{Sim}_2(\text{td}_{\text{Sim}}, \text{pk}, C)$:

- Parse $\text{td}_{\text{Sim}} = 1\text{T}.\text{td}$ and $\text{pk} = (1\text{T}.\text{pk}, \text{FHE.pk}, \text{FHE.ek}, \text{com}_{\rho_{\text{Enc}}}, \text{com}_{\text{FHE.sk}})$.
- Compute $\text{ct}_w \leftarrow \text{FHE.Enc}(0^\ell)$, and $\text{ct}_{C(w)} = \text{FHE.Eval}_{\text{FHE.ek}}(C, \text{ct}_w)$.
- Call $1\text{T}.\pi \leftarrow 1\text{T}.\text{Sim}_2(\text{td}_{\text{Sim}}, 1\text{T}.\text{pk}, C')$ where C' is defined as in Fig. 1.
- Output $\pi = (\text{ct}_w, 1\text{T}.\pi)$.

We first argue CRS indistinguishability: this follows directly by CRS indistinguishability of $1\text{T}.\text{UNIZK}$ (as a reduction can generate $(\text{ck}, \text{Commit}.\text{td}_{\text{Ext}}) \leftarrow \text{Commit.Gen}(1^\lambda)$ itself).

Next we argue single-theorem zero-knowledge given the simulation trapdoor. Let $\mathcal{A}$ be a PPT adversary. We want to show that the following two distributions are computationally indistinguishable:

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^h, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\widetilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Prove}}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}), \end{array} : b \right\}$$

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^h, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\widetilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sim}_2}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}), \end{array} : b \right\},$$

where $\mathcal{A}$ is restricted to make at most one oracle query.

To do so, we argue by a hybrid argument that the experiments induced by $\mathcal{O}_{\text{Prove}}(\cdot)$ and $\mathcal{O}_{\text{Sim}_2}(\cdot)$ are indistinguishable.

Hybrid $\mathcal{H}_0$. This is the distribution induced by $\mathcal{O}_{\text{Prove}}(\cdot)$, namely calls to (C, w) are answered as $\text{Bootstrap.Prove}(\text{crs}, \text{sk}, C, w)$.

Hybrid $\mathcal{H}_1$. We change how pk is computed, and more specifically the way $r_{\text{FHE}}, r_{1\text{T}}$ are computed in Bootstrap.Gen . Instead of computing $r_{\text{FHE}} = \text{PRG}_{\text{Gen}}(\rho_{\text{FHE}})$ and $r_{1\text{T}} = \text{PRG}_{1\text{T}}(\rho_{1\text{T}})$, we now sample them as $r_{\text{FHE}} \leftarrow \{0, 1\}^r$ and $r_{1\text{T}} \leftarrow \{0, 1\}^r$.

The distributions induced by $\mathcal{H}_0$ and $\mathcal{H}_1$ are computationally indistinguishable by security of the PRGs PRG_{Gen} and $\text{PRG}_{1\text{T}}$.

Hybrid $\mathcal{H}_2$. In this hybrid, we modify Bootstrap.Prove to generate the 1-time-UNIZK proof as follows $1\text{T}.\pi \leftarrow 1\text{T}.\text{Sim}_2(1\text{T}.\text{td}_{\text{Sim}}, \text{pk}, C')$ where C' is the one defined in Fig. 1.

The distributions induced by $\mathcal{H}_1$ and $\mathcal{H}_2$ are computationally indistinguishable by adaptive single-theorem zero-knowledge given simulation trapdoors of 1T.UNIZK: this follows by a direct reduction that computes the other components of pk except $1\text{T}.\text{pk}$ itself, and computes $\text{ct}_w = \text{FHE.Enc}(w; \rho_{\text{Enc}})$ itself (where ρ_{Enc} is sampled by the reduction and implicitly included in sk , and where we implicitly use that C' has size at most $n_{1\text{T}}$ and input size at most $\ell_{1\text{T}}$).

Hybrid $\mathcal{H}_3$. In this hybrid, we modify Bootstrap.Gen so that the commitments $\text{com}_{\text{FHE.sk}}, \text{com}_{\rho_{\text{Enc}}}$ included in the public key pk are instead generated as $\text{com}_{\text{FHE.sk}} \leftarrow \text{Commit}(\text{ck}, \mathbf{0})$, $\text{com}_{\rho_{\text{Enc}}} \leftarrow \text{Commit}(\text{ck}, \mathbf{0})$ where $\mathbf{0}$ is of appropriate size $\text{poly}(\lambda)$.

The distributions induced by $\mathcal{H}_2$ and $\mathcal{H}_3$ are computationally indistinguishable by hiding of the commitment.

Hybrid $\mathcal{H}_4$. In this hybrid, we change how r_{Enc} , used to encrypt w under the FHE, is computed. Instead of generating it as $r_{\text{Enc}} = \text{PRG}_{\text{Enc}}(\rho_{\text{Enc}})$ we now sample $r_{\text{Enc}} \leftarrow \{0, 1\}^r$.

The distributions induced by $\mathcal{H}_3$ and $\mathcal{H}_4$ are computationally indistinguishable by security of the PRG PRG_{Enc} .

Hybrid $\mathcal{H}_5$. In this hybrid, we change how we compute ct_w and $\text{ct}_{C(w)}$. Namely, we now compute $\text{ct}_w \leftarrow \text{FHE.Enc}(0^\ell)$, and $\text{ct}_{C(w)} = \text{FHE.Eval}_{\text{FHE.ek}}(C, \text{ct}_w)$.

Thus, the distributions induced by $\mathcal{H}_4$ and $\mathcal{H}_5$ are computationally indistinguishable by the security of FHE (noting that the randomness r_{Enc} and the secret key FHE.sk are otherwise not used by the oracle in either of these hybrids).

Hybrid $\mathcal{H}_6$. We switch back how the commitments $\text{com}_{\text{FHE.sk}}, \text{com}_{\rho_{\text{Enc}}}$ included in the public key pk are generated: we compute now $\text{com}_{\text{FHE.sk}} \leftarrow \text{Commit}(\text{ck}, \text{FHE.sk})$, $\text{com}_{\rho_{\text{Enc}}} \leftarrow \text{Commit}(\text{ck}, \rho_{\text{Enc}})$.

Thus, the distributions induced by $\mathcal{H}_5$ and $\mathcal{H}_6$ are computationally indistinguishable by the hiding of the commitment.

Hybrid $\mathcal{H}_7$. We now switch back r_{FHE} and $r_{1\text{T}}$ to be computed as $r_{\text{FHE}} = \text{PRG}(\rho_{\text{FHE}})$ and $r_{1\text{T}} = \text{PRG}(\rho_{1\text{T}})$. The distributions induced by $\mathcal{H}_6$ and $\mathcal{H}_7$ are computationally indistinguishable by the security of the PRGs PRG_{Gen} and $\text{PRG}_{1\text{T}}$.

Observe now that the experiment in $\mathcal{H}_6$ is exactly the one induced by using $\text{Sim}_2(\text{td}_{\text{Sim}}, \text{pk}, C)$, which concludes the proof. $\square$

Claim 6.7 (Bootstrappability). *Assuming the (leveled) fully homomorphic encryption scheme FHE satisfies the efficiency conditions of Claim 3.17, the construction satisfies bootstrappability.*

Proof. First, we recall the discussion of parameters from above, where we observed that if FHE satisfies Claim 3.17, and with the setting of r above, then for C of size n and input size ℓ , we have that C' has size $n_{1\text{T}} = \text{poly}(\ell, \lambda)$, input size $\ell_{1\text{T}} = \ell + \text{poly}(\lambda)$, and depth $\text{poly}(\lambda)$ for some fixed, large enough polynomials poly .

As a result, $\text{Bootstrap.Setup}(1^h, 1^\ell, 1^\lambda)$ supports statements/witnesses (C, w) as long as $|w| \leq \ell_{1\text{T}}$ and the depth of C is at most h .

Let ρ be the input to $C_{\text{Gen}, \text{crs}, \text{pk}}$ (defined in Definition 4.6, paragraph Bootstrappability). Because ρ consists of randomness used to compute two evaluations of Commit of $\text{poly}(\lambda)$ -length messages, along with three PRG seeds (to PRG_{Gen} , $\text{PRG}_{1\text{T}}$, PRG_{Enc} respectively), we obtain that $C_{\text{Gen}, \text{crs}, \text{pk}}$ has input length $\text{poly}(\lambda)$.

Next, we claim that the *depth* of $C_{\text{Gen}, \text{crs}, \text{pk}}$ is $\text{depth}_{\text{Gen}} = \text{poly}(\ell, \lambda)$ for a large enough poly independent of h . This is because by Claim 3.17, FHE.Gen can be computed in depth $\text{poly}(\lambda)$ independent of h , and by inspection (and the settings of $n_{1\text{T}}$, $\ell_{1\text{T}}$), other sub-procedures can be computed in depth $\text{poly}(\ell, \lambda)$.

Therefore, setting $h = \text{poly}(\ell, \lambda)$ for a large enough poly ensures bootstrappability. $\square$

7 Multi-Theorem UNIZK

In this section, we show how to construct a multi-theorem UNIZK starting from a 1-time UNIZK with bootstrappability.

Definition 7.1. *A multi-theorem UNIZK is a relaxed UNIZK satisfying completeness, adaptive knowledge soundness with semi-malicious keys, multi-theorem zero-knowledge, and adaptive uniqueness with semi-malicious keys (Definition 4.1, Definition 4.6).*

Theorem 7.2. *Assuming the existence of a 1-time UNIZK with bootstrappability (Definition 5.1, Definition 4.1), a statistically extractable commitment scheme and collision-resistant hash functions, there exists a multi-theorem UNIZK (Definition 4.1).*

Remark 7.3 (Computational uniqueness). Similarly to Remarks 5.3 and 6.2, if we start with a computationally unique 1-time UNIZK with bootstrappability instead, Theorem 7.2 yields instead a computationally unique multi-theorem UNIZK, with essentially the same construction and proof.

Construction. We use the following building blocks:

- A bootstrappable 1-time DP-UNIZK $\text{Bootstrap.UNIZK} = (\text{Bootstrap.Setup}, \text{Bootstrap.Gen}, \text{Bootstrap.Prove}, \text{Bootstrap.Verify})$,
- A statistically extractable commitment scheme $(\text{Commit.Gen}, \text{Commit}, \text{Commit.Verify}, \text{Commit.Ext})$ with statistically unique openings.
- A collision resistant hash function $(\text{CRHF.Setup}, H)$ with input size $n = n(\lambda)$ and output size λ .²⁵
- A pseudorandom function $(\text{PRF.Gen}, \text{PRF.Eval})$ with output size defined below.

²⁵We assume without loss of generality $n \leq \lambda$, up to padding the n -bits input with sufficiently many 0's. In fact if $n < \lambda$ we do not even need to rely on a collision resistant hash function.

Parameters. Let $d = d(\lambda)$ denote (an upper bound on) the depth of circuit statement. We use the following parameters:

- $h_{\text{Bt}} = h + d + \text{poly}(\lambda)$, where h is given by bootstrappability of Bootstrap.UNIZK, and for some fixed large enough polynomial poly .
- We will suppose that Bootstrap.Gen and Commit take randomness of size (at most) $r(\lambda) = \text{poly}(\lambda)$, and that PRF has output size $2r$.

We define our NIZK as follows.

- $\text{Setup}(1^n, 1^\ell, 1^\lambda)$: Compute $(\text{Bootstrap.crs}, \text{Bootstrap.td}_{\text{Ext}}) \leftarrow \text{Bootstrap.Setup}(1^{h_{\text{Bt}}}, 1^\lambda)$, where h_{Bt} is defined above.
Compute $\text{ck} \leftarrow \text{Commit.Gen}(1^\lambda)$ and $\text{hk} \leftarrow \text{CRHF.Setup}(1^\lambda)$. Output:

$$\text{crs} = (\text{Bootstrap.crs}, \text{ck}, \text{hk}), \quad \text{td}_{\text{Ext}} = (\text{Bootstrap.td}_{\text{Ext}}, \text{hk})$$

- $\text{Gen}(\text{crs}; \rho)$: Compute for $i = 1, \dots, \lambda$, $K_i \leftarrow \text{PRF.Gen}(1^\lambda)$, and compute $(\text{com}_{\text{PRF}, i}, \text{decom}_{\text{PRF}, i}) \leftarrow \text{Commit}(\text{ck}, K_i)$.

Compute $(\text{Bootstrap.pk}_0, \text{Bootstrap.sk}_0) \leftarrow \text{Bootstrap.Gen}(\text{Bootstrap.crs})$

Output:

$$\text{pk} = (\{\text{com}_{\text{PRF}, i}\}_{1 \leq i \leq \lambda}, \text{Bootstrap.pk}_0)$$

and

$$\text{sk} = (\text{pk}, \text{Bootstrap.sk}_0, \{K_i, \text{com}_{\text{PRF}, i}, \text{decom}_{\text{PRF}, i}\}_{1 \leq i \leq \lambda}).$$

- $\text{Prove}(\text{crs}, \text{sk}, C, w)$: Parse $\text{crs} = (\text{Bootstrap.crs}, \text{ck}, \text{hk})$, $\text{sk} = (\text{pk}, \text{Bootstrap.sk}_0, \{K_i, \text{com}_{\text{PRF}, i}, \text{decom}_{\text{PRF}, i}\}_{1 \leq i \leq \lambda})$, and $\text{pk} = (\{\text{com}_{\text{PRF}, i}\}_{1 \leq i \leq \lambda}, \text{Bootstrap.pk}_0)$.
Compute $\text{path} = H(\text{hk}, C) \in \{0, 1\}^\lambda$. Compute, for $1 \leq i \leq \lambda$ and $b \in \{0, 1\}$:

$$\rho_{\text{path}_{\leq i} \| b}^{\text{Gen}} = \text{PRF}(K_i, \text{path}_{\leq i} \| b) \in \{0, 1\}^r. \text{ }^{26}$$

For $i = 1, \dots, \lambda$:

- For $b \in \{0, 1\}$, compute $(\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}, \text{Bootstrap.sk}_{\text{path}_{\leq i} \| b}) = \text{Bootstrap.Gen}(\text{crs}; \rho_{\text{path}_{\leq i} \| b}^{\text{Gen}})$ using randomness $\rho_{\text{path}_{\leq i} \| b}^{\text{Gen}}$.
- Consider the circuit C^i defined in Fig. 2.
Compute $\pi_i = \text{Bootstrap.Prove}(\text{Bootstrap.crs}, \text{Bootstrap.sk}_{\text{path}_{\leq i-1}}, C^i, w_i)$, where

$$w_i = (K_i, \text{decom}_{\text{PRF}, i})$$

and with the convention that $\text{Bootstrap.sk}_\emptyset = \text{Bootstrap.sk}_0$.

Compute $\pi_C = \text{Bootstrap.Prove}(\text{Bootstrap.crs}, \text{Bootstrap.sk}_{\text{path}}, C, w)$.

Output:

$$\pi = (\{\pi_i\}_{1 \leq i \leq \lambda}, \pi_C, \{\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}\}_{1 \leq i \leq \lambda, b \in \{0, 1\}}).$$

- $\text{Verify}(\text{crs}, \text{pk}, C, \pi)$: Parse $\text{crs} = (\text{Bootstrap.crs}, \text{ck}, \text{hk})$, $\text{pk} = (\{\text{com}_{\text{PRF}, i}\}_{1 \leq i \leq \lambda}, \text{Bootstrap.pk}_0)$, and $\pi = (\{\pi_i\}_{1 \leq i \leq \lambda}, \pi_C, \{\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}\}_{1 \leq i \leq \lambda, b \in \{0, 1\}})$.
Compute $\text{path} = H(\text{hk}, C)$.
Output 1 if and only if:

²⁶Recall that $\text{path}_{\leq i}$ denotes the first i bits of path .

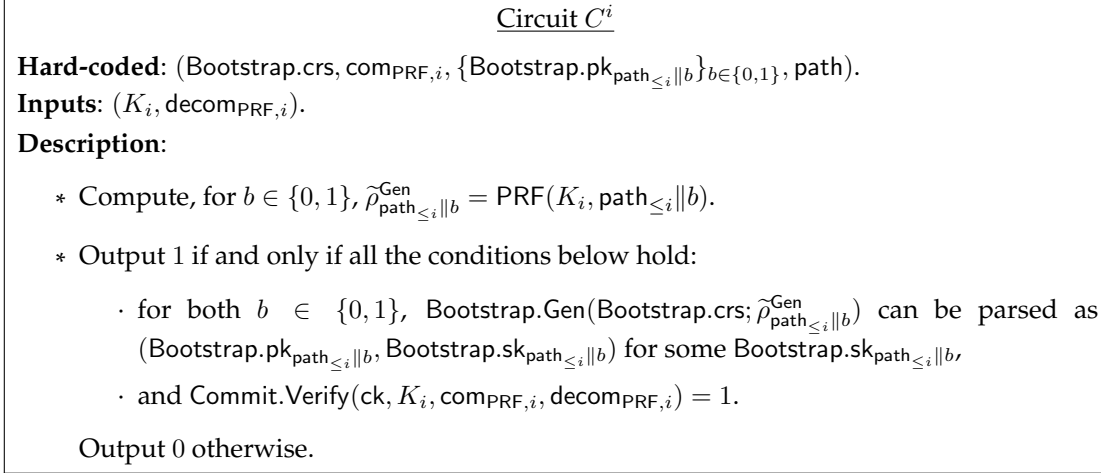


Figure 2: Definition of the circuit C^i .

- $\text{Bootstrap.Verify}(\text{Bootstrap.crs}, \text{Bootstrap.pk}_{\text{path}_{\leq i-1}}, C^i, \pi_i) = 1$ for all $1 \leq i \leq \lambda$, with the convention that $\text{Bootstrap.pk}_{\emptyset} = \text{Bootstrap.pk}_0$, and where C^i is defined in Fig. 2;
- and $\text{Bootstrap.Verify}(\text{Bootstrap.crs}, \text{Bootstrap.pk}_{\text{path}}, C, \pi_C) = 1$.

and output 0 otherwise.

- $\text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi)$: Parse $\pi = (\{\pi_i\}_{1 \leq i \leq \lambda}, \pi_C, \{\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}\}_{1 \leq i \leq \lambda, b \in \{0,1\}})$. Compute $\text{path} = H(\text{hk}, C)$. Output

$$w = \text{Bootstrap.Ext}(\text{Bootstrap.td}_{\text{Ext}}, \text{Bootstrap.pk}_{\text{path}}, \pi_C).$$

Claim 7.4 (Completeness). *Suppose $h_{\text{Bt}} = h + d + \text{poly}(\lambda)$, where d is (an upper bound) on the depth of the circuits C considered, and h is given by bootstrappability of Bootstrap.UNIZK , for some fixed large enough polynomial poly . Assuming the perfect completeness and bootstrappability of the 1-time DP-UNIZK Bootstrap.UNIZK (Definition 4.6), and the perfect correctness of the commitment scheme $(\text{Commit.Gen}, \text{Commit}, \text{Commit.Verify})$, the construction above satisfies perfect completeness (Definition 4.1).*

Proof. By bootstrappability of Bootstrap.UNIZK , for all $1 \leq i \leq \lambda$, the circuit C^i has depth at most $h(\lambda) + \text{poly}(\lambda)$ (as evaluating PRF and Commit.Verify only induce an additive $\text{poly}(\lambda)$ overhead on the depth of the circuit, and where h is given by bootstrappability of Bootstrap.UNIZK , see Definition 4.6). Let d be (an upper bound on) the depth of the statement considered. Setting $h_{\text{Bt}} = h(\lambda) + \text{poly}(\lambda) + d$, we have that both C^i and C have size at most h_{Bt} .

We claim that for all $1 \leq i \leq \lambda$, $\text{Verify}(\text{Bootstrap.crs}, \text{Bootstrap.pk}_{\text{path}_{\leq i-1}}, C^i, \pi_i) = 1$ and that $\text{Bootstrap.pk}_{\text{path}_{\leq i}}$ is in the support of Bootstrap.Gen . The claim for index i follows using that $C^i(w_i) = 1$ (where w_i is defined in the construction), that Commit is (perfectly) correct, and that $\text{Bootstrap.pk}_{\text{path}_{\leq i-1}}$ is in the support of Bootstrap.Gen , so that invoking the perfect completeness of Bootstrap.UNIZK yields the claim.

Almost identically, assuming $C(w) = 1$, we obtain that $\text{Verify}(\text{Bootstrap.crs}, \text{Bootstrap.pk}_{\text{path}}, C, \pi_C) = 1$: this again follows from perfect completeness of Bootstrap.UNIZK (as Bootstrap.pk is in the support of Bootstrap.Gen), and the conclusion follows. □

Claim 7.5 (Adaptive, Statistical Knowledge Soundness with Semi-Malicious Keys). *Suppose that Bootstrap.UNIZK satisfies adaptive statistical knowledge-soundness against semi-malicious keys Definition 4.6. Then the construction above satisfies adaptive knowledge soundness against semi-malicious keys Definition 4.6.*

Proof. Let $\mathcal{P}^*$ be a cheating prover against adaptive knowledge soundness with semi-malicious keys of the construction above, that is:

$$\Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\rho, C^*, \pi) \leftarrow \mathcal{P}^*(\text{crs}), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho), \\ w^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1 \\ \wedge C^*(w^*) = 0 \end{array} \right] \geq 1/p(\lambda).$$

for some polynomial $p(\lambda)$. Parse $\pi = (\{\pi_i\}_{1 \leq i \leq \lambda}, \pi_{C^*}, \{\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}\}_{1 \leq i \leq \lambda, b \in \{0,1\}})$. For $1 \leq i \leq \lambda$, let $w_i = \text{Bootstrap.Ext}(\text{Bootstrap.td}_{\text{Ext}}, \text{Bootstrap.pk}_{\text{path}_{\leq i-1}}, \pi_i)$, and let $w_{C^*} = \text{Bootstrap.Ext}(\text{Bootstrap.td}_{\text{Ext}}, \text{Bootstrap.pk}_{\text{path}}, \pi_{C^*})$ (where Bootstrap.Ext is the knowledge extractor of Bootstrap.UNIZK). Let $\text{path} = H(\text{hk}, C^*)$ and parse $w_i = (K_i, \text{decomp}_{\text{PRF}, i})$.

By adaptive knowledge soundness against semi-malicious keys of Bootstrap.UNIZK , the probability that $\text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1$ and $C^1(w_1) = 0$ in the experiment above is negligible — this is because the randomness of Gen includes the randomness used to generate $(\text{Bootstrap.pk}_0, \text{Bootstrap.sk}_0)$, so that producing w_1 such that $\text{Bootstrap.Verify}(\text{Bootstrap.crs}, \text{Bootstrap.pk}_0, C^1, \pi_1) = 1$ (which is implied by $\text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1$) and $C^1(w_1) = 0$ would contradict adaptive knowledge soundness against semi-malicious keys of Bootstrap.UNIZK with key pk_0 (by a direct reduction that provides as randomness for Bootstrap.Gen the relevant part of the randomness of Gen , set the proof for Bootstrap.UNIZK as π_1 and ignore the other parts of the proof, very similarly to Claim 6.4). In other words:

$$\Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\rho, C^*, \pi) \leftarrow \mathcal{P}^*(\text{crs}), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho), \\ w^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1 \\ \wedge C^*(w^*) = 0 \\ \wedge C^1(w_1) = 1 \end{array} \right] \geq 1/p(\lambda) - \text{negl}(\lambda).$$

This implies by definition of C^1 that with probability at least $1/p(\lambda) - \text{negl}(\lambda)$, we have that for both $b \in \{0, 1\}$, $(\text{Bootstrap.pk}_{\text{path}_1 \| b}, \text{Bootstrap.sk}_{\text{path}_1 \| b}) = \text{Bootstrap.Gen}(\text{Bootstrap.crs}; \rho_{\text{path}_1 \| b}^{\text{Gen}})$ for some $\text{Bootstrap.sk}_{\text{path}_1 \| b}$, where $\rho_{\text{path}_1 \| b}^{\text{Gen}}$ is defined as $\rho_{\text{path}_{\leq 1} \| b}^{\text{Gen}} = \text{PRF}(K_1, \text{path}_{\leq 1} \| b)$ (where we recall that K_1 is included in the extracted witness w_1). We can thus again invoke adaptive knowledge soundness against semi-malicious keys of Bootstrap.UNIZK (implicitly using $\text{Bootstrap.pk}_{\text{path}_{\leq 1}}$ as a public key) to argue that the probability that $\text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1$ and $C^2(w_2) = 0$ is negligible, where now the randomness of Bootstrap.Gen is given by $\rho_{\text{path}_1 \| b}^{\text{Gen}}$, and an almost identical reduction.

Iterating this argument λ times over the (C^i, w_i) , $i \leq \lambda$ gives that

$$\Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda), \\ (\rho, C^*, \pi) \leftarrow \mathcal{P}^*(\text{crs}), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho), \\ w^* \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi) \end{array} : \begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi) = 1 \\ \wedge C^*(w^*) = 0 \\ \wedge \bigwedge_{i=1}^{\lambda} C^i(w_i) = 1 \end{array} \right] \geq 1/p(\lambda) - \text{negl}(\lambda).$$

Invoking adaptive knowledge soundness with malicious keys one last time (implicitly using $\text{Bootstrap.pk}_{\text{path}}$ and randomness $\rho_{\text{path}} = \text{PRF}(K_\lambda, \text{path})$) gives that the probability that $\text{Bootstrap.Verify}(\text{crs}, \text{pk}_\lambda, C^*, \pi_{C^*}) = 1$ and $C^*(w^*) = 0$ is negligible, which overall contradicts the initial assumption that $C^*(w^*) = 0$ with probability $1/p(\lambda)$. $\square$

Claim 7.6 (Adaptive, Statistical Uniqueness with Semi-Malicious Keys). *Suppose that Bootstrap.UNIZK satisfies adaptive, statistical uniqueness against semi-malicious keys (Definition 4.6), and that Commit is statistically extractable and has statistically unique openings. Then the construction above satisfies adaptive, statistical uniqueness against semi-malicious keys (Definition 4.6).*

Proof. Suppose $\mathcal{P}^*$ is a cheating prover against uniqueness with semi-malicious keys in the construction above, that is given $\text{crs} \leftarrow \text{Setup}(1^n, 1^\ell, 1^\lambda)$, $(\rho, C^*, \pi^{(0)}, \pi^{(1)}) \leftarrow \mathcal{P}^*(\text{crs})$, $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}; \rho)$, $w^{(0)} \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi^{(0)})$, and $w^{(1)} \leftarrow \text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi^{(1)})$:

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{crs}, \text{pk}, C^*, \pi^{(0)}) = 1 \\ \wedge \text{Verify}(\text{crs}, \text{pk}, C^*, \pi^{(1)}) = 1 \\ \wedge \pi^{(0)} \neq \pi^{(1)} \\ \wedge w^{(0)} = w^{(1)} \end{array} \right] \geq 1/p(\lambda)$$

for some polynomial $p(\lambda)$. We will say that $\mathcal{P}^*$ *wins* if the event above occurs.

Let us denote, for $\beta \in \{0, 1\}$, $\pi^{(\beta)} = (\{\pi_i^{(\beta)}\}_{1 \leq i \leq \lambda}, \pi_{C^*}^{(\beta)}, \{\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}\}_{1 \leq i \leq \lambda, b \in \{0, 1\}})$. Let us furthermore denote, for $1 \leq i \leq \lambda$ and $\beta \in \{0, 1\}$, $w_i^{(\beta)} = \text{Bootstrap.Ext}(\text{Bootstrap.td}_{\text{Ext}}, \text{Bootstrap.pk}_{\text{path}_{\leq i-1}}, \pi_i^{(\beta)})$ and $w_{C^*}^{(\beta)} = \text{Bootstrap.Ext}(\text{Bootstrap.td}_{\text{Ext}}, \text{Bootstrap.pk}_{\text{path}}, \pi_{C^*}^{(\beta)})$. We first prove the following lemma, which states statistical uniqueness of the “intermediate” proofs included in $\pi^{(\beta)}$.

Lemma 7.7. *Assume Commit is statistically extractable and has statistically unique openings, and that Bootstrap.UNIZK satisfies both adaptive, statistical uniqueness and adaptive, statistical knowledge soundness against semi-malicious keys. Then conditioned on $\mathcal{P}^*$ winning, we have that $\pi_{C^*}^{(0)} \neq \pi_{C^*}^{(1)}$ except with negligible probability.*

Proof. We prove the claim by showing that conditioned on $\mathcal{P}^*$ winning, all the other components of $\pi^{(\beta)}$ are equal with overwhelming probability.

To do so, suppose that there exists an index $1 \leq j \leq \lambda$ such that, with non-negligible probability conditioned on $\mathcal{P}^*$ winning, $w_j^{(0)} = w_j^{(1)}$, that is:

$$\left(\pi_j^{(0)}, \{\text{Bootstrap.pk}_{\text{path}_{\leq j} \| b}\}_{b \in \{0, 1\}} \right) \neq \left(\pi_j^{(1)}, \{\text{Bootstrap.pk}_{\text{path}_{\leq j} \| b}\}_{b \in \{0, 1\}} \right).$$

Without loss of generality, we will assume that j is the smallest such index; in particular for all $k < j$ we have $w_k^{(0)} = w_k^{(1)}$. Note that $\text{Bootstrap.pk}_{\text{path}_{\leq j-1} \| b}^{(0)} = \text{Bootstrap.pk}_{\text{path}_{\leq j-1} \| b}^{(1)}$ by minimality and definition of j ; we will therefore denote $\text{Bootstrap.pk}_{\text{path}_{\leq j-1} \| b} = \text{Bootstrap.pk}_{\text{path}_{\leq j-1} \| b}^{(\beta)}$ for $\beta \in \{0, 1\}$.

Similar to the proof of Claim 7.5, leveraging adaptive soundness of Bootstrap.UNIZK and extractability of Commit, we obtain that except with negligible probability, $\text{Bootstrap.pk}_{\text{path}_{\leq j-1} \| b}^{(\beta)}$ is in the support of Bootstrap.Gen, with true randomness ρ if $j = 1$, or randomness $\rho_{\text{path}_{\leq j-1} \| b}^{\text{Gen}} = \text{PRF}(K_{j-1}, \text{path}_{\leq j-1} \| b)$ where $K_{j-1} = \text{Commit.Ext}(\text{Commit.td}_{\text{Ext}}, \text{com}_{\text{PRF}, j-1})$ if $j \geq 2$. In particular, because K_{j-1} is defined independently of β (recall that $\text{com}_{\text{PRF}, j-1}$ is included in the public key pk), we obtain overall that except with negligible probability, $\{\text{Bootstrap.pk}_{\text{path}_{\leq j} \| b}^{(0)}\}_{b \in \{0, 1\}} = \{\text{Bootstrap.pk}_{\text{path}_{\leq j} \| b}^{(1)}\}_{b \in \{0, 1\}}$.²⁷

Thus, with non-negligible probability, we have $\pi_j^{(0)} \neq \pi_j^{(1)}$.

We then claim that this would imply $w_j^{(0)} \neq w_j^{(1)}$ except with negligible probability. This follows by adaptive, statistical uniqueness with semi-malicious keys of Bootstrap.UNIZK via a routine reduction, that calls $\mathcal{P}^*$, submits C^j as the circuit, $\rho_{\text{path}_{\leq j-1} \| b}^{\text{Gen}} = \text{PRF}(K_{j-1}, \text{path}_{\leq j-1} \| b)$ as randomness, and $\pi_j^{(0)} \neq \pi_j^{(1)}$ as the proofs (and ignores the components $k \neq j$ of the proofs provided by $\mathcal{P}^*$).

Last, we claim that $w_j^{(0)} = w_j^{(1)}$ except with negligible probability, which contradicts the above. Let us write, for $\beta \in \{0, 1\}$,

$$w_j^{(\beta)} = \left(K_j^{(\beta)}, \{\text{decom}_{\text{PRF}, j}^{(\beta)}\}_{b \in \{0, 1\}} \right).$$

Conditioned on $\mathcal{P}^*$ winning, for both $\beta \in \{0, 1\}$, we have $\text{Verify}(\text{crs}, \text{pk}, C^*, \pi^{(\beta)}) = 1$, thus

$\text{Bootstrap.Verify}(\text{Bootstrap.crs}, \text{Bootstrap.pk}_{\text{path}_{\leq i-1}}, C^i, \pi_i^{(\beta)}) = 1$, by definition of $\mathcal{P}^*$ winning and of Verify,

²⁷For Claim 7.5, we didn't need to rely on extractability of Commit, as it was enough to argue there that the public key was in the support of Bootstrap.Gen. Here we additionally rely on extractability to ensure that both keys $\text{Bootstrap.pk}_{\text{path}_{\leq j-1} \| b}^{(\beta)}$, $\beta \in \{0, 1\}$ are generated using the same randomness $\text{PRF}(K_{j-1}, \text{path}_{\leq j-1} \| b)$ and therefore are equal.

respectively. Thus, adaptive (knowledge) soundness with semi-malicious keys of Bootstrap.UNIZK gives that except with negligible probability, $C^i(w_i^{(\beta)}) = 1$, and in particular:

$$\text{Commit.Verify}(\text{ck}, K_j^{(\beta)}, \text{com}_{\text{PRF},j}, \text{decom}_{\text{PRF},j}^{(\beta)}) = 1$$

with overwhelming probability. Recall that we have previously argued by extractability of Commit, that except with negligible probability, $K_j^{(0)} = K_j^{(1)} = \text{Commit.Ext}(\text{Commit.td}_{\text{Ext}}, \text{com}_{\text{PRF},j})$.

By the statistical uniqueness of openings of Commit, the verification equation above implies that except with negligible probability, $\text{decom}_{\text{PRF},j}^{(0)} = \text{decom}_{\text{PRF},j}^{(1)}$.

Overall, we obtain that $w_j^{(0)} = w_j^{(1)}$ except with negligible probability, which contradicts the previous conclusion that $w_j^{(0)} \neq w_j^{(1)}$ except with negligible probability. We conclude that the initial assumption is false, and thus for every $1 \leq j \leq \lambda$,

$$\left(\pi_j^{(0)}, \{\text{Bootstrap.pk}_{\text{path}_{\leq j}\|b}^{(0)}\}_{b \in \{0,1\}} \right) = \left(\pi_j^{(1)}, \{\text{Bootstrap.pk}_{\text{path}_{\leq j}\|b}^{(1)}\}_{b \in \{0,1\}} \right)$$

except with negligible probability, which concludes the proof of the lemma. $\square$

Given Lemma 7.7, conditioned on $\mathcal{P}^*$ winning, the condition $\pi^{(0)} \neq \pi^{(1)}$ implies $\pi_C^{(0)} \neq \pi_C^{(1)}$ with non-negligible probability. But because $\mathcal{P}^*$ wins with non-negligible probability, this contradicts the adaptive uniqueness of Bootstrap.UNIZK (through a routine reduction that calls $\mathcal{P}^*$, submits C^* as the circuit, $\rho_{\text{path}} = \text{PRF}(K_\lambda, \text{path})$ as randomness, and $\pi_C^{(0)} \neq \pi_C^{(1)}$ as the proofs, and ignores the other components of the proofs provided by $\mathcal{P}^*$, where we use that $\text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi^{(\beta)}) = \text{Bootstrap.Ext}(\text{Bootstrap.td}_{\text{Ext}}, \text{Bootstrap.pk}_{\text{path}}, \pi_C^{(\beta)})$ for both $\beta \in \{0,1\}$), which concludes the proof of uniqueness. $\square$

Claim 7.8 (Adaptive Multi-Theorem Zero-Knowledge). *Assuming that Bootstrap.UNIZK satisfies adaptive single-theorem zero-knowledge (Definition 4.6), that $(\text{PRF.Gen}, \text{PRF.Eval})$ is a secure pseudorandom function, that the commitment scheme Commit is computationally hiding, and that the hash function H is collision resistant, the construction above satisfies multi-theorem zero-knowledge (Definition 4.1).*

Proof. Let $(\text{Bootstrap.Sim}_1, \text{Bootstrap.Sim}_2)$ be the two-stage simulator given by adaptive (single-theorem) zero-knowledge of Bootstrap.UNIZK.

We define our simulator $\text{Sim}_1, \text{Sim}_2$ as follows:

- $\text{Sim}_1(1^n, 1^\ell, 1^\lambda)$:
Compute $(\text{Bootstrap.crs}, \text{Bootstrap.td}_{\text{Sim}}) \leftarrow \text{Bootstrap.Sim}_1(1^{h_{\text{Bt}}}, 1^\lambda)$,
 $\text{ck} \leftarrow \text{Commit.Gen}(1^\lambda)$ and $\text{hk} \leftarrow \text{CRHF.Gen}(1^\lambda)$. Output

$$\widetilde{\text{crs}} = (\text{Bootstrap.crs}, \text{ck}, \text{hk}) \quad , \quad \widetilde{\text{td}_{\text{Sim}}} = \text{Bootstrap.td}_{\text{Sim}}.$$

- $\text{Sim}_2(\widetilde{\text{crs}}, \text{pk}, C)$:
 - Parse $\widetilde{\text{crs}} = (\text{Bootstrap.crs}, \text{ck}, \text{hk})$ and $\text{pk}(\{\text{com}_{\text{PRF},i}\}_{1 \leq i \leq \lambda}, \text{Bootstrap.pk}_0)$.
 - Compute $\text{path} = H(\text{hk}, C)$, and initialize an empty table T .
 - For $i = 1, \dots, \lambda$:
 - * For $b \in \{0,1\}$, compute $(\text{Bootstrap.pk}_{\text{path}_{\leq i}\|b}, \text{Bootstrap.sk}_{\text{path}_{\leq i}\|b}) = \text{Bootstrap.Gen}(\text{crs}; \rho_{\text{path}_{\leq i}\|b}^{\text{Gen}})$ using randomness $\rho_{\text{path}_{\leq i}\|b}^{\text{Gen}}$ if $\rho_{\text{path}_{\leq i}\|b}^{\text{Gen}}$ has been previously defined in T ; otherwise sample a fresh $\rho_{\text{path}_{\leq i}\|b}^{\text{Gen}}$ uniformly at random and record it in a table T ;
 - * Let C^i denote the circuit described in Fig. 2.
Compute $\pi_i = \text{Bootstrap.Sim}_2(\text{Bootstrap.td}_{\text{Sim}}, \text{Bootstrap.pk}_{\text{path}_{\leq i-1}}, C^i)$, with the convention that $\text{Bootstrap.pk}_\emptyset = \text{Bootstrap.pk}_0$.

Compute $\pi_C = \text{Bootstrap.Sim}_2(\text{Bootstrap.td}_{\text{Sim}}, \text{Bootstrap.pk}_{\text{path}}, C)$.

Output:

$$\pi = \left(\{\pi_i\}_{1 \leq i \leq \lambda}, \pi_C, \{\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}\}_{1 \leq i \leq \lambda, b \in \{0,1\}} \right).$$

We first argue CRS indistinguishability for Sim_1 . Note that the only difference is that in the output of Sim_1 , the CRS is now including $\widetilde{\text{crs}} \leftarrow \text{Bootstrap.Sim}_1$ instead of $\text{Bootstrap.crs} \leftarrow \text{Setup}$. CRS indistinguishability then follows from CRS indistinguishability of Bootstrap.UNIZK , via a direct reduction (that either samples $\text{Bootstrap.crs} \leftarrow \text{Bootstrap.Setup}(1^{h_{\text{Bt}}}, 1^\lambda)$ or $\text{Bootstrap.crs} \leftarrow \text{Bootstrap.Sim}_1(1^{h_{\text{Bt}}}, 1^\lambda)$, and computes all other components itself).

Next, we argue that the simulated proofs using $\widetilde{\text{crs}}$ are indistinguishable from real proofs, and we do so by introducing the following sequence of hybrid experiments. Let $\mathcal{A}$ be a PPT adversary.

Hybrid $\mathcal{H}_0$. This corresponds to the setting where proofs are generated using $\mathcal{O}_{\text{Prove}}$, which has $\widetilde{\text{crs}}$, pk hard-coded, and which, on input C, w (with the restriction that $|C| = n, w = \ell, C(w) = 1$), outputs $\pi \leftarrow \text{Prove}(\widetilde{\text{crs}}, \text{pk}, C, w)$. Namely, the output of the experiment is computed as follows:

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^n, 1^\ell, 1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\widetilde{\text{crs}}), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Prove}}(\cdot)}(\text{crs}, \text{pk}, \text{td}_{\text{Sim}}) \end{array} : b \right\}.$$

For $1 \leq i \leq \lambda$, we define the following intermediate experiments:

Hybrid $\mathcal{H}_i$. In this experiment, all components $\{\pi_j\}_{j \leq i}$ and $\{\text{Bootstrap.pk}_{\text{path}_{\leq j} \| b}\}_{1 \leq j \leq i, b \in \{0,1\}}$ are computed as in Sim_2 . In other words, Sim_2 runs its internal “for” loop up to index i , and runs the sub-procedure from Prove for indices $j > i$.

We furthermore define the following sub-hybrid experiments to argue that $\mathcal{H}_i \approx \mathcal{H}_{i+1}$.

Hybrid $\mathcal{H}_{i,1}$. We change how we compute the components π_i in all the issued proofs. We now compute it as $\pi_i = \text{Bootstrap.Sim}_2(\text{Bootstrap.td}_{\text{Sim}}, \text{Bootstrap.pk}_{\text{path}_{\leq i-1}}, C^i)$.

Note that at this stage, all keys $\text{Bootstrap.pk}_{\text{path}_{\leq i-1}}$ are generated using true randomness by definition of $\mathcal{H}_i$. Furthermore, every $\text{Bootstrap.pk}_{\text{path}_{\leq i-1}}$ is only ever used to prove a single statement C^i (that has hard-coded $(\text{Bootstrap.crs}, \text{com}_{\text{PRF}, i}, \{\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}\}_{b \in \{0,1\}}, \text{path})$), where we use that the hard-coded values $\{(\text{Bootstrap.crs}, \text{com}_{\text{PRF}, i}, \{\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}, \text{com}_{\text{sk}, \text{path}_{\leq i} \| b}\}_{b \in \{0,1\}}, \text{path})\}$ are fully determined by the PRF key K_i and path, and the table T .

Therefore by single-theorem, adaptive zero-knowledge of Bootstrap.UNIZK (through a routine reduction, and using that the adversary is given td_{Sim} which we use to simulate proofs corresponding to $j < i$ and using at most Q internal hybrids where Q is the number of oracle queries of the adversary), we have $\mathcal{H}_i \approx_c \mathcal{H}_{i,1}$.

Hybrid $\mathcal{H}_{i,2}$. We change how we compute $\text{com}_{\text{PRF}, i}$ included in the CRS. We now compute it as $\text{Commit}(\text{ck}, \mathbf{0})$.

This follows directly by hiding of Commit (where we use that $\text{decom}_{\text{PRF}, i}$ is neither used in Hybrid $\mathcal{H}_{i,1}$ nor $\mathcal{H}_{i,2}$), yielding $\mathcal{H}_{i,1} \approx_c \mathcal{H}_{i,2}$.

Hybrid $\mathcal{H}_{i,3}$. We change how $\text{Bootstrap.pk}_{\text{path}_{\leq i} \| b}$ is computed. We now use the table T , and use randomness $\rho_{\text{path}_{\leq i} \| b}^{\text{Gen}}$ if it has been previously defined in T , and otherwise sample a fresh $\rho_{\text{path}_{\leq i} \| b}^{\text{Gen}}$ uniformly at random and record it in T .

This follows directly by security of the PRF with key K_i (which is not otherwise used in neither Hybrid $\mathcal{H}_{i,2}$ nor $\mathcal{H}_{i,3}$). This gives $\mathcal{H}_{i,2} \approx_c \mathcal{H}_{i,3}$.

Hybrid $\mathcal{H}_{i,4}$. We change again how we compute $\text{comp}_{\text{PRF},i}$ included in the CRS. We now compute it as $\text{Commit}(\text{ck}, K_i)$.

This again follows directly by hiding of Commit (using that $\text{decomp}_{\text{PRF},i}$ is neither used in Hybrid $\mathcal{H}_{i,2}$ nor $\mathcal{H}_{i,3}$), yielding $\mathcal{H}_{i,2} \approx_c \mathcal{H}_{i,3}$.

Last, observe that the experiment induced in $\mathcal{H}_{i,4}$ is identical to $\mathcal{H}_{i+1}$. We have thus concluded that, under the assumptions of Claim 7.8, for all $1 \leq i < \lambda$, $\mathcal{H}_i \approx_c \mathcal{H}_{i+1}$.

We last define the following hybrids:

Hybrid $\mathcal{H}_{\lambda+1}$. We abort the experiment if $\mathcal{A}$ queries its oracle on two statements $C \neq C'$ such that $\text{path} = H(\text{hk}, C) = H(\text{hk}, C') = \text{path}'$.

By collision resistance of H , the probability that an efficient $\mathcal{A}$ makes the experiment abort this way is negligible, thus $\mathcal{H}_\lambda \approx_c \mathcal{H}_{\lambda+1}$.

Hybrid $\mathcal{H}_{\lambda+2}$. We change how we compute π_C . We now compute it as

$$\pi_C = \text{Bootstrap.Sim}_2(\text{Bootstrap.td}_{\text{Sim}}, \text{Bootstrap.pk}_{\text{path}}, C).$$

Note that this corresponds to the experiment corresponding to answering oracle calls using Sim_2 .

At this stage, all keys $\text{Bootstrap.pk}_{\text{path}}$ are generated using true randomness $\rho_{\text{path}}^{\text{Gen}}$ by definition of $\mathcal{H}_\lambda$ (and are unchanged in further hybrids). Furthermore, every $\text{Bootstrap.pk}_{\text{path}}$ is only ever used to prove a single statement C , thanks to the abort condition introduced in $\mathcal{H}_{\lambda+1}$.

We can thus invoke single-theorem zero-knowledge of Bootstrap.UNIZK through a routine reduction. We use that the adversary is given td_{Sim} which we use to simulate proofs corresponding to π_i for $1 \leq i \leq \lambda$, and using Q internal hybrids where Q is the number of oracle queries of the adversary. Crucially, we use that for any given $\text{Bootstrap.pk}_{\text{path}}$, the adversary only sees a *single* proof associated to $\text{Bootstrap.pk}_{\text{path}}$. This follows from the restriction that, for any statement C , the adversary is only allowed to submit (at most) a single query of the form (C, w) . This yields $\mathcal{H}_{\lambda+1} \approx_c \mathcal{H}_{\lambda+2}$.

Hybrid $\mathcal{H}_{\lambda+3}$. We do not abort the experiment anymore if $\mathcal{V}^*$ queries the simulators on two statements $C \neq C'$ such that $\text{path} = H(\text{hk}, C) = H(\text{hk}, C') = \text{path}'$.

Again, the probability of aborting is negligible by collision resistance of H , and so $\mathcal{H}_{\lambda+2} \approx_c \mathcal{H}_{\lambda+3}$. Last, observe that Hybrid $\mathcal{H}_{\lambda+3}$ corresponds to the experiment induced by $\mathcal{O}_{\text{Sim}_2}$, which concludes the proof. $\square$

8 Security against Fully Malicious Provers

In this section, we discuss a simple transformation from a multi-theorem UNIZK where soundness and uniqueness hold against semi-malicious prover setups, to one where they hold against malicious prover setups: doing so yields a standard UNIZK (Definition 4.1).

Intuitively, we simply attach to the new public key a separate NIZK that the public keys of the semi-malicious multi-theorem UNIZK are well-formed — because we do not require uniqueness out of the public keys, we do not need the NIZK to have any uniqueness properties either. This gives the following.

Theorem 8.1. *Assuming the existence of a (semi-malicious) multi-theorem UNIZK (Definition 7.1), and a NIZK with adaptive statistical soundness and adaptive zero-knowledge, there exists a UNIZK (Definition 4.1).*

Remark 8.2 (Computational uniqueness). Similarly to Remark 5.3, if we start with a computationally unique multi-theorem UNIZK instead, Theorem 8.1 yields instead a computationally unique UNIZK, with essentially the same construction and proof.

Remark 8.3 (Computational soundness). Throughout the technical body, we considered by default statistically (knowledge) sound UNIZKs; however Theorem 8.1 also extends to the computational (knowledge) sound setting assuming an adaptively knowledge sound, adaptively zero-knowledge NIZK argument (which also only requires assuming a computationally knowledge sound semi-malicious UNIZK instead).

Construction. We use the following building blocks:

- A (semi-malicious) multi-theorem UNIZK $\text{MT.UNIZK} = (\text{MT.Setup}, \text{MT.Gen}, \text{MT.Prove}, \text{MT.Verify}, \text{MT.Ext})$ (Definition 7.1),
- A standard NIZK $\text{NIZK} = (\text{NIZK.Setup}, \text{NIZK.Prove}, \text{NIZK.Verify})$.

For polynomials $n = n(\lambda)$, $\ell = \ell(\lambda)$, let $p = p(\lambda)$ and $q = q(\lambda)$ be polynomials that respectively upper bounds the size of the circuit $\text{MT.Gen}(\text{crs}; \cdot)$ and the size of its internal randomness, for all crs in the support of $\text{MT.Setup}(1^n, 1^\ell, 1^\lambda)$.

We define our UNIZK as follows:

- $\text{Setup}(1^n, 1^\ell, 1^\lambda)$: Call $(\text{MT.crs}, \text{MT.td}_{\text{Ext}}) \leftarrow \text{MT.Setup}(1^n, 1^\ell, 1^\lambda)$ and $\text{NIZK.crs} \leftarrow \text{NIZK.Setup}(1^p, 1^q, 1^\lambda)$. Output: $(\text{crs}, \text{td}_{\text{Ext}})$ where $\text{crs} = (\text{MT.crs}, \text{NIZK.crs})$ and $\text{td}_{\text{Ext}} = \text{MT.td}_{\text{Ext}}$.
- $\text{Gen}(\text{crs})$: Parse $\text{crs} = (\text{MT.crs}, \text{NIZK.crs})$ and call $(\text{MT.pk}, \text{MT.sk}) = \text{MT.Gen}(\text{crs}; \rho_{\text{Gen}})$ using randomness that we denote ρ_{Gen} .
Let C_{Gen} be the circuit, with $(\text{MT.crs}, \text{MT.pk})$ hard-coded, that takes as input $(\rho_{\text{Gen}}, \text{MT.sk})$ and checks that $(\text{MT.pk}, \text{MT.sk}) = \text{MT.Gen}(\text{crs}; \rho_{\text{Gen}})$.
Compute $\text{NIZK.}\pi_{\text{Gen}} \leftarrow \text{NIZK.Prove}(\text{NIZK.crs}, C_{\text{Gen}}, (\rho_{\text{Gen}}, \text{MT.sk}))$. Output (pk, sk) where $\text{pk} = (\text{MT.pk}, \text{NIZK.}\pi_{\text{Gen}})$ and $\text{sk} = \text{MT.sk}$.
- $\text{Prove}(\text{crs}, \text{sk}, C, w)$: Parse $\text{crs} = (\text{MT.crs}, \text{NIZK.crs})$ and $\text{sk} = (\text{MT.sk}, \text{NIZK.sk})$, and output
$$\text{MT.}\pi \leftarrow \text{Prove}(\text{MT.crs}, \text{MT.sk}, C, w).$$
- $\text{Verify}(\text{crs}, \text{pk}, C, \pi)$: Parse $\text{crs} = (\text{MT.crs}, \text{NIZK.crs})$, $\text{pk} = (\text{MT.pk}, \text{NIZK.}\pi_{\text{Gen}})$, and $\pi = \text{MT.}\pi$. Output 1 if and only if $\text{MT.Verify}(\text{MT.crs}, \text{MT.pk}, C, \text{MT.}\pi) = 1$ and $\text{NIZK.Verify}(\text{NIZK.crs}, C_{\text{Gen}}, \text{NIZK.}\pi) = 1$ where C_{Gen} is defined above.
- $\text{Ext}(\text{td}_{\text{Ext}}, \text{pk}, \pi)$: Parse $\text{pk} = (\text{MT.pk}, \text{NIZK.}\pi_{\text{Gen}})$ and output
 $w = \text{MT.Ext}(\text{td}_{\text{Ext}}, \text{MT.pk}, \pi)$.

Claim 8.4 (Completeness). *Assuming completeness of MT.UNIZK and of NIZK, the construction satisfies completeness.*

Proof. This follows directly from the fact that $C_{\text{Gen}}(\rho_{\text{Gen}}, \text{MT.sk}) = 1$, and by completeness of MT.UNIZK and of NIZK. $\square$

Claim 8.5 (Adaptive Knowledge Soundness with Malicious Keys). *Assuming that MT.UNIZK satisfies adaptive knowledge soundness with semi-malicious keys and that NIZK is adaptively sound, the construction satisfies adaptive knowledge soundness with malicious keys.*

Proof. By adaptive soundness of NIZK, we have that $\text{NIZK.Verify}(\text{NIZK.crs}, C_{\text{Gen}}, \text{NIZK.}\pi) = 1$ implies that there exists $(\rho_{\text{Gen}}, \text{MT.sk})$ such that $C_{\text{Gen}}(\rho_{\text{Gen}}, \text{MT.sk}) = 1$, that is $(\text{MT.pk}, \text{MT.sk}) = \text{MT.Gen}(\rho_{\text{Gen}})$.

Thus, by adaptive knowledge soundness of MT.UNIZK with semi-malicious keys (where, in the reduction, we set the randomness for MT.Gen as ρ_{Gen}), adaptive knowledge soundness against malicious keys of the construction follows. $\square$

Claim 8.6 (Adaptive Uniqueness with Malicious Keys). *Assuming that MT.UNIZK satisfies adaptive uniqueness with semi-malicious keys and that NIZK is adaptively sound, the construction satisfies adaptive uniqueness with malicious keys.*

Proof. As in the proof of Claim 8.5, adaptive soundness of NIZK implies that there exists ρ_{Gen} such that $(\text{MT.pk}, \text{MT.sk}) = \text{MT.Gen}(\rho_{\text{Gen}})$.

Thus, by adaptive uniqueness of MT.UNIZK with semi-malicious keys (where again, in the reduction, we set the randomness for MT.Gen as ρ_{Gen}), adaptive uniqueness against malicious keys of the construction follows. $\square$

Claim 8.7 (Adaptive, Multi-Theorem Zero-Knowledge). *Assuming that MT.UNIZK satisfies adaptive multi-theorem zero-knowledge, that NIZK is adaptively zero-knowledge, the construction satisfies adaptive multi-theorem zero-knowledge with malicious keys.*

Proof. Let $\text{MT.Sim}_1, \text{MT.Sim}_2$ be the simulators given by adaptive multi-theorem zero-knowledge of MT.UNIZK. We define:

- $\text{Sim}_1(1^n, 1^\ell, 1^\lambda)$: Compute $\text{MT.crs}, \text{td}_{\text{Sim}} = \text{MT.Sim}_1(1^n, 1^\ell, 1^\lambda)$, and $\text{NIZK.crs} \leftarrow \text{NIZK.Setup}(1^p, 1^q, 1^\lambda)$. Output $\text{crs} = (\text{MT.crs}, \text{NIZK.crs}), \text{td}_{\text{Sim}}$.
- $\text{Sim}_2(\text{td}_{\text{Sim}}, \text{pk}, C)$: Parse $\text{pk} = (\text{MT.pk}, \text{NIZK.}\pi_{\text{Gen}})$, and output $\text{MT.Sim}_2(\text{td}_{\text{Sim}}, \text{MT.pk}, C)$.

CRS indistinguishability follows directly by CRS indistinguishability of MT.UNIZK.

To argue multi-theorem zero-knowledge with simulated CRS, we proceed with the following hybrid arguments. First, we switch $\text{NIZK.crs}, \pi_{\text{Gen}}$ to be simulated according to the NIZK simulators. This is indistinguishable by adaptive zero-knowledge of NIZK. Then we switch the oracle to instead compute proofs as $\text{MT.Sim}_2(\text{td}_{\text{Sim}}, \text{MT.pk}, C)$. This is indistinguishable by adaptive multi-theorem zero-knowledge with simulated CRS of MT.UNIZK (where we use that neither ρ_{Gen} nor ρ_{sk} are then needed by the reduction). Last, we switch back $\text{NIZK.crs}, \pi_{\text{Gen}}$ to be generated using $\text{NIZK.Setup}, \text{NIZK.Prove}$: this is again indistinguishable by adaptive zero-knowledge of NIZK. $\square$

Combining Theorems 5.2, 6.1, 7.2 and 8.1, we obtain the following.

Corollary 8.8. *Assuming the existence of perfectly correct garbled circuits, of an extractable/equivocal commitment scheme with statistically unique openings, of a leveled bootstrappable FHE, and an (adaptively statistically sound, adaptively zero-knowledge) NIZK, there exists a UNIZK for all NP.*

Assuming instead a commitment scheme with computationally unique openings (resp. an adaptively, computationally knowledge sound NIZK), then there exists a relaxed UNIZK with computational uniqueness (resp. computational knowledge soundness) for all NP.

Alternatively, assuming the (polynomial) hardness of LWE (with super-polynomial modulus-to-noise ratio), there exists a UNIZK for all NP.

9 Necessity of Relaxations

We start by defining a slightly relaxed notion of unique witness maps (UWMs), that we label UWM^* , from [CPW23]. We additionally consider a zero-knowledge variant thereof.

Definition 9.1. *A UWM^* for an NP language L with relation R consists of the PPT algorithms (Setup, Prove, Verify). We require three properties:*

- *Completeness: There is some negligible function μ such that for all $\lambda \in \mathbb{N}$ all $(x, w) \in R$ we have:*

$$\Pr[\text{Verify}(\text{crs}, x, \pi) = 1 : \text{crs} \leftarrow \text{Setup}(1^\lambda), \pi \leftarrow \text{Prove}(\text{crs}, x, w)] \geq 1 - \mu(\lambda).$$

- *Selective Soundness*: For all PPT adversaries $\mathcal{A}$ all ensembles $x = \{x_\lambda\}$ with $x_\lambda \notin L$ we have:

$$\Pr[\text{Verify}(\text{crs}, x, \pi) = 1 : \text{crs} \leftarrow \text{Setup}(1^\lambda), \pi \leftarrow \mathcal{A}(\text{crs})] = \text{negl}(\lambda).$$

- *Uniqueness*: There is some negligible function μ such that for all $\lambda \in \mathbb{N}$ all x, w, w' with $(x, w) \in R$ and $(x, w') \in R$ we have

$$\Pr \left[\pi = \pi' : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda), \\ \pi \leftarrow \text{Prove}(\text{crs}, x, w), \\ \pi' \leftarrow \text{Prove}(\text{crs}, x, w') \end{array} \right] \geq 1 - \mu(\lambda).$$

We additionally say that a UVM* is zero-knowledge if, furthermore, the following property holds:

- *Adaptive Single-Theorem Zero-Knowledge (Optional)*: There exists a PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ such that the following properties hold.

- CRS indistinguishability: The following two distributions are computationally indistinguishable:

$$\text{crs} \approx_c \widetilde{\text{crs}},$$

where $\text{crs} \leftarrow \text{Setup}(1^\lambda)$ and $(\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^\lambda)$.

- Adaptive Single-Theorem Zero-Knowledge with Simulated CRS: For all oracle-aided PPT adversaries $\mathcal{A}$ making a single oracle query, the two following distributions are computationally indistinguishable:

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^\lambda), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Prove}}(\cdot)}(\text{crs}, \text{td}_{\text{Sim}}) \end{array} : b \right\}$$

$$\left\{ \begin{array}{l} (\widetilde{\text{crs}}, \text{td}_{\text{Sim}}) \leftarrow \text{Sim}_1(1^\lambda), \\ b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sim}_2}(\cdot)}(\text{crs}, \text{td}_{\text{Sim}}) \end{array} : b \right\},$$

where $\mathcal{O}_{\text{Prove}}$ and $\mathcal{O}_{\text{Sim}_2}$ are oracles defined as follows:

- $\mathcal{O}_{\text{Prove}}(x, w)$ is an oracle that has hard-coded crs , which, on input (x, w) where $(x, w) \in R$, outputs $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$, and otherwise outputs $\perp$;
- $\mathcal{O}_{\text{Sim}_2}(x, w)$ is an oracle that has hard-coded $\text{td}_{\text{Sim}}, \text{pk}$, which, on input (x, w) where $(x, w) \in R$, outputs $\pi \leftarrow \text{Sim}_2(\text{td}_{\text{Sim}}, C)$.

Theorem 9.2 ([CPW23]). Assuming there exists a UVM* for NP, then there also exist witness PRFs (Definition 3.3) and witness encryption for NP.

Note that the work of [CPW23] defined a slightly stronger notion of uniqueness, but their proof of the above theorem works with the above notion as is (no changes needed).

We also consider an alternative version of Theorem 9.2, which assumes a zero-knowledge UVM*, but preserves the language of interest (as opposed to requiring an UVM* for all NP). The proof is conceptually very similar to the proof of [CPW23].

Theorem 9.3. Assuming there exists a zero-knowledge UVM* for a language $L \in \text{NP}$, and there exist one-way functions, then there also exists a non-interactive witness PRF (Definition 3.3) for L .²⁸

Proof. Let $(\text{Setup}, \text{Prove}, \text{Verify}, \text{Sim})$ be a zero-knowledge UVM* for L , with proofs of size p for some polynomial p . Let $\{\text{PRF}_{K_1}\}_{K_1}$ and $\{\text{PRF}_{K_2}\}_{K_2}$ and $\{\text{PRF}_{K_3}\}_{K_3}$ be three PRF families.²⁹ We build our witness PRF as follows:

²⁸A non-interactive witness PRF is a relaxation of witness PRF where security only holds against adversaries who do not make any PRF queries [Zha16]. Moreover, the existence of a non-interactive witness PRF for a language L implies the existence of a witness encryption for L . See Definition 3.3 for a formal definition.

²⁹Looking ahead, we use the PRF keys K_1, K_3 solely to make the algorithms deterministic.

- **wPRF.Gen**: on input 1^λ , sample $r \leftarrow \{0,1\}^p$, compute $(\text{crs}, \text{td}_{\text{sim}}) \leftarrow \text{Sim}_1(1^\lambda)$. Sample random PRF keys K_1, K_2 and K_3 for the three PRF families, and output $\text{ek} = (\text{crs}, r, K_3)$ and $\text{fk} = (\text{td}_{\text{sim}}, r, K_1, K_2)$.
- **F_{fk}**: on input an instance x , compute $\tilde{\pi} = \text{Sim}_2(\text{td}_{\text{sim}}, x; \text{PRF}_{K_1}(x))$. If $\text{Verify}(\text{crs}, x, \tilde{\pi}) = 0$, output $y = \text{PRF}_{K_2}(x)$. Otherwise, output $y = \text{hc}(\tilde{\pi}; r)$, where hc is a hardcore bit (for instance the one of [GL89]).
- **wPRF.Eval_{ek}**: on input (x, w) , abort if $(x, w) \notin R$. Otherwise, parse $\text{ek} = (\text{crs}, r)$, compute $\pi = \text{Prove}(\text{crs}, x, w; \text{PRF}_{K_3}(x))$, and output $y' = \text{hc}(\pi; r)$.

We start by arguing correctness of the witness PRF (observe that the witness PRF evaluation algorithms are indeed deterministic). Suppose $(x, w) \in R$.

Firstly, because Verify is efficient, we have by zero-knowledge and completeness of UVM^* , along with security of PRF_{K_1} , that with overwhelming probability, $\text{Verify}(\text{crs}, x, \tilde{\pi}) = 1$, over the randomness of $(\text{crs}, \text{td}_{\text{sim}}) \leftarrow \text{Sim}_1(1^\lambda)$ and K_1 .

Secondly, we argue that $\pi = \tilde{\pi}$ with overwhelmingly probability, where $(\text{crs}, \text{td}_{\text{sim}}) \leftarrow \text{Sim}_1(1^\lambda)$, $\pi = \text{Prove}(\text{crs}, x, w; \text{PRF}_{K_3}(x))$ and $\tilde{\pi} = \text{Sim}_2(\text{crs}, \text{td}_{\text{sim}}, x; \text{PRF}_{K_1}(x))$ where K_1 and K_3 are random. This follows by uniqueness and zero-knowledge of UVM^* , and the security of PRF_{K_1} and PRF_{K_3} . Indeed, if this were not the case, consider the distinguisher for the zero-knowledge experiment of UVM^* which, given x along with a witness w , obtains crs , and π^* by querying its oracle on (x, w) . It computes $\pi^? \leftarrow \text{Prove}(\text{crs}, x, w)$, and outputs 1 if $\pi^? = \pi^*$. By the uniqueness and CRS indistinguishability properties of UVM^* , along with the security of PRF_{K_3} , if $\pi^* = \text{Prove}(\text{crs}, x, w; \text{PRF}_{K_3}(x))$, then $\pi^? = \pi^*$ with overwhelming probability. On the other hand, by assumption, if $\pi^? \leftarrow \text{Prove}(\text{crs}, x, w)$ and $\pi^* \leftarrow \text{Sim}_2(\text{crs}, \text{td}_{\text{sim}}, x)$, then $\pi^? \neq \pi^*$ with non-negligible probability (which is indistinguishable from when $\text{Sim}_2(\text{crs}, \text{td}_{\text{sim}}, x; \text{PRF}_{K_1}(x))$ by security of PRF_{K_1}), which would contradict zero-knowledge of UVM^* .

Next, we argue non-interactive, static security of the witness PRF;³⁰ let A be an adversary for the non-interactive, static witness PRF experiment who wins with non-negligible probability. There are two cases: either A wins with non-negligible probability where the challenge PRF value is generated from $\tilde{\pi}$ such that $\text{Verify}(\text{crs}, x, \tilde{\pi}) = 0$, or wins with non-negligible probability whenever $\text{Verify}(\text{crs}, x, \tilde{\pi}) = 1$ (formally, a reduction would guess which case it is and proceed accordingly).

In the first case, A would break security of the PRF K_2 : the view of the adversary can be simulated given a challenge query for $\text{PRF}_{K_2}(\cdot)$, along with $(\text{crs}, \text{td}_{\text{sim}}) \leftarrow \text{Sim}_1(1^\lambda)$ which the reduction can generate itself.

In the second case, by the properties of the hard-core bit hc , such an adversary A can be used to produce $\tilde{\pi} = \text{Sim}_2(\text{crs}, \text{td}, x; \text{PRF}_{K_1}(x))$ for $x \notin L$ such that $\text{Verify}(\text{crs}, x, \tilde{\pi}) = 1$ where $\text{crs} \leftarrow \text{Sim}_1(1^\lambda)$. By CRS indistinguishability of UVM^* and the fact that x is independent of the crs , this implies that A can also be used to produce $\tilde{\pi}$ such that $\text{Verify}(\text{crs}, x, \tilde{\pi}) = 1$ where $\text{crs} \leftarrow \text{Setup}(1^\lambda)$, which, as $x \notin L$, contradicts soundness of UVM^* . $\square$

UNIZK without Setup. We show that a UNIZK without setup, even with our relaxed notion of uniqueness, implies UWM^* and therefore witness PRFs and witness encryption. It suffices to define a very weak notion of UNIZK without setup where two independent honestly generated proofs of the same statement x with the same witness w need to produce the same proof π w.o.p..

Definition 9.4 (UNIZK Variant 1). *A UNIZK-V1 for an NP language L with relation R is a NIZK with completeness, selective soundness, selective witness indistinguishability and the following uniqueness property:*

- *There is some negligible function μ such that for all $\lambda \in \mathbb{N}$ and all $(x, w) \in R$, we have*

$$\Pr \left[\pi = \pi^* : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda), \\ \pi \leftarrow \text{Prove}(\text{crs}, x, w), \\ \pi^* \leftarrow \text{Prove}(\text{crs}, x, w) \end{array} \right] \geq 1 - \mu(\lambda).$$

³⁰Recall that this refers to witness PRF security when the adversary does not get oracle access to F_{fk} [Zha16], and where the adversary declares some $x \notin L$ before seeing the crs .

Theorem 9.5. *Any UNIZK-V1 for NP is also a UWM* for NP. In particular, the existence of a UNIZK-V1 for NP implies the existence of witness PRFs and witness encryption for NP.*

Proof. Suppose that the scheme is not a UWM*, meaning that it does not satisfy the uniqueness property of UWM*. Then there must exist some non-negligible μ and some ensemble $x_\lambda, w_\lambda, w'_\lambda$ with $(x_\lambda, w_\lambda) \in R$ and $(x_\lambda, w'_\lambda) \in R$ such that

$$\Pr \left[\pi \neq \pi' : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda), \\ \pi \leftarrow \text{Prove}(\text{crs}, x_\lambda, w_\lambda), \\ \pi' \leftarrow \text{Prove}(\text{crs}, x_\lambda, w'_\lambda) \end{array} \right] \geq \mu(\lambda).$$

We show that this leads to an attack on witness indistinguishability of the scheme, leading to a contradiction. In particular, there is an efficient attack to distinguish (crs, π) versus (crs, π') where $\text{crs} \leftarrow \text{Setup}(1^\lambda)$, $\pi \leftarrow \text{Prove}(\text{crs}, x_\lambda, w_\lambda)$ and $\pi' \leftarrow \text{Prove}(\text{crs}, x_\lambda, w'_\lambda)$. The distinguisher $D(\text{crs}, \pi^?)$ generates $\pi^* \leftarrow \text{Prove}(\text{crs}, x_\lambda, w_\lambda)$ and outputs 1 if $\pi^? = \pi^*$. By the uniqueness property of the UNIZK-V1 we have $\Pr[D(\text{crs}, \pi) = 1] \geq 1 - \text{negl}(\lambda)$. On the other hand, by the above we have $\Pr[D(\text{crs}, \pi') = 1] = 1 - \mu(\lambda)$ and therefore the distinguishing advantage is $\mu(\lambda) - \text{negl}(\lambda)$ which is non-negligible. $\square$

Next, we observe that zero-knowledge versions of UNIZK-V1 similarly correspond to zero-knowledge UVM*. Theorem 9.3 then gives the following:

Theorem 9.6. *An (adaptive, single-theorem) zero-knowledge UNIZK-V1 for a language $L \in NP$ is also a zero-knowledge UVM* for L . In particular, the existence of an (adaptive, single-theorem) zero-knowledge UNIZK-V1 for a language $L \in NP$, along with one-way functions, implies the existence of (non-interactive) witness PRFs and witness encryption for L .*

Theorem 9.6 is conceptually very similar to Theorem 9.5; the main difference is that Theorem 9.6 preserves the language L at hand (as opposed to assuming a UNIZK-V1 for all NP), at the cost of requiring zero-knowledge (and the existence of one-way functions), and achieving a weaker security for the resulting witness PRF.³¹

Remark 9.7 (UNIZKs without Setup for UP). There is, to our knowledge, no (non-trivial) construction of witness encryption for UP under weaker assumptions than those implying witness encryption for all NP . Therefore, one consequence of Theorem 9.6 is some indication that building UNIZKs without setup even for UP is also likely to be hard. Note that UVM* are trivially achievable for relations in UP ; consequently, the hardness of building strong UNIZKs for UP stems from the apparent hardness of achieving simultaneously uniqueness and zero-knowledge.

UNIZK with Uniqueness for All Witnesses. We show that a UNIZK with setup that produces unique proofs even for different witnesses implies UWM* and therefore witness PRFs and witness encryption. It suffices to define a very weak notion of uniqueness where, for honestly generated setup (pk, sk) and for any statement x with witnesses w, w' if we generate $\pi \leftarrow \text{Prove}(\text{crs}, \text{sk}, x, w)$, $\pi' \leftarrow \text{Prove}(\text{crs}, \text{sk}, x, w')$ then $\pi = \pi'$ w.o.p.. Moreover, we don't even need the UNIZK to satisfy any ZK property.

Definition 9.8 (UNIZK Variant 2). *A UNIZK-V2 for an NP language L with relation R is a UNIZK with completeness, selective soundness with honest setup, and the following uniqueness property:*

- *There is some negligible function μ such that for all $\lambda \in \mathbb{N}$ all x, w, w' with $(x, w) \in R$ and $(x, w') \in R$ we have,*

$$\Pr \left[\pi = \pi' : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda), \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs}), \\ \pi \leftarrow \text{Prove}(\text{crs}, \text{sk}, x, w), \\ \pi' \leftarrow \text{Prove}(\text{crs}, \text{sk}, x, w') \end{array} \right] \geq 1 - \mu(\lambda).$$

³¹In our construction, achieving a stronger form of security for the witness PRF would require simulation-soundness from the UNIZK-V1.

Theorem 9.9. *Assuming a UNIZK-V2 for NP, there exists a UWM* for NP. In particular, the existence of a UNIZK-V2 for NP implies the existence of witness PRFs and witness encryption for NP.*

Proof. Let $(\text{Setup}, \text{Gen}, \text{Prove}, \text{Verify})$ be a UNIZK-V2 for NP. Define $\text{Setup}'(1^\lambda)$ to generate $\text{crs} \leftarrow \text{Setup}(1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{crs})$ and output $\text{crs}' = (\text{crs}, \text{pk}, \text{sk})$. Define $\text{Prove}'(\text{crs}', x, w) = \text{Prove}(\text{crs}, \text{sk}, x, w)$ and $\text{Verify}'(\text{crs}', x, \pi) = \text{Verify}(\text{crs}, \text{pk}, x, \pi)$. Then $(\text{Setup}', \text{Prove}', \text{Verify}')$ is a UWM*. The completeness and uniqueness properties of the UWM* follow directly from those of the UNIZK-V2. The soundness property of the UWM* follows from the selective soundness with honest setup property of the UNIZK-V2 which ensures that a dishonest prover cannot generate proofs for selectively chosen false statements x , even given an honestly generated $\text{crs}, \text{pk}, \text{sk}$. $\square$

References

- [AFG⁺22] Behzad Abdolmaleki, Nils Fleischhacker, Vipul Goyal, Abhishek Jain, and Giulio Malavolta. Steganography-free zero-knowledge. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022: 20th Theory of Cryptography Conference, Part I*, volume 13747 of *Lecture Notes in Computer Science*, pages 143–172. Springer, Heidelberg, November 2022.
- [Ajt96] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *28th Annual ACM Symposium on Theory of Computing*, pages 99–108. ACM Press, May 1996.
- [BFM88] Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications (extended abstract). In *20th Annual ACM Symposium on Theory of Computing*, pages 103–112. ACM Press, May 1988.
- [BGT20] Zvika Brakerski, Sanjam Garg, and Rotem Tsabary. FHE-based bootstrapping of designated-prover NIZK. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part I*, volume 12550 of *Lecture Notes in Computer Science*, pages 657–683. Springer, Heidelberg, November 2020.
- [BV11] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In Rafail Ostrovsky, editor, *52nd Annual Symposium on Foundations of Computer Science*, pages 97–106. IEEE Computer Society Press, October 2011.
- [CCH⁺19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-Shamir: from practice to theory. In Moses Charikar and Edith Cohen, editors, *51st Annual ACM Symposium on Theory of Computing*, pages 1082–1090. ACM Press, June 2019.
- [CHK03] Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. In Eli Biham, editor, *Advances in Cryptology – EUROCRYPT 2003*, volume 2656 of *Lecture Notes in Computer Science*, pages 255–271. Springer, Heidelberg, May 2003.
- [CPW23] Suhradip Chakraborty, Manoj Prabhakaran, and Daniel Wichs. A map of witness maps: New definitions and connections. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13941 of *Lecture Notes in Computer Science*, pages 635–662. Springer, Heidelberg, May 2023.
- [FLS90] Uriel Feige, Dror Lapidot, and Adi Shamir. Multiple non-interactive zero knowledge proofs based on a single random string (extended abstract). In *31st Annual Symposium on Foundations of Computer Science*, pages 308–317. IEEE Computer Society Press, October 1990.

- [Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st Annual ACM Symposium on Theory of Computing*, pages 169–178. ACM Press, May / June 2009.
- [GGI⁺15] Craig Gentry, Jens Groth, Yuval Ishai, Chris Peikert, Amit Sahai, and Adam D. Smith. Using fully homomorphic hybrid encryption to minimize non-interactive zero-knowledge proofs. *Journal of Cryptology*, 28(4):820–843, October 2015.
- [GGSW13] Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th Annual ACM Symposium on Theory of Computing*, pages 467–476. ACM Press, June 2013.
- [GL89] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *21st Annual ACM Symposium on Theory of Computing*, pages 25–32. ACM Press, May 1989.
- [GMR85] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *17th Annual ACM Symposium on Theory of Computing*, pages 291–304. ACM Press, May 1985.
- [GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In Serge Vaudenay, editor, *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 339–358. Springer, Heidelberg, May / June 2006.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 197–206. ACM Press, May 2008.
- [GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 75–92. Springer, Heidelberg, August 2013.
- [KW18] Sam Kim and David J. Wu. Multi-theorem preprocessing NIZKs from lattices. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part II*, volume 10992 of *Lecture Notes in Computer Science*, pages 733–765. Springer, Heidelberg, August 2018.
- [LMs05] Matt Lepinski, Silvio Micali, and abhi shelat. Fair-zero knowledge. In Joe Kilian, editor, *TCC 2005: 2nd Theory of Cryptography Conference*, volume 3378 of *Lecture Notes in Computer Science*, pages 245–263. Springer, Heidelberg, February 2005.
- [MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718. Springer, Heidelberg, April 2012.
- [NY89] Moni Naor and Moti Yung. Universal one-way hash functions and their cryptographic applications. In *21st Annual ACM Symposium on Theory of Computing*, pages 33–43. ACM Press, May 1989.
- [PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part I*, volume 11692 of *Lecture Notes in Computer Science*, pages 89–114. Springer, Heidelberg, August 2019.

- [Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th Annual ACM Symposium on Theory of Computing*, pages 84–93. ACM Press, May 2005.
- [SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th Annual ACM Symposium on Theory of Computing*, pages 475–484. ACM Press, May / June 2014.
- [Zha16] Mark Zhandry. How to avoid obfuscation using witness PRFs. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A: 13th Theory of Cryptography Conference, Part II*, volume 9563 of *Lecture Notes in Computer Science*, pages 421–448. Springer, Heidelberg, January 2016.